Laboratorul 1 – Implementarea unei functii logice intr-un circuit programabil

Obiective

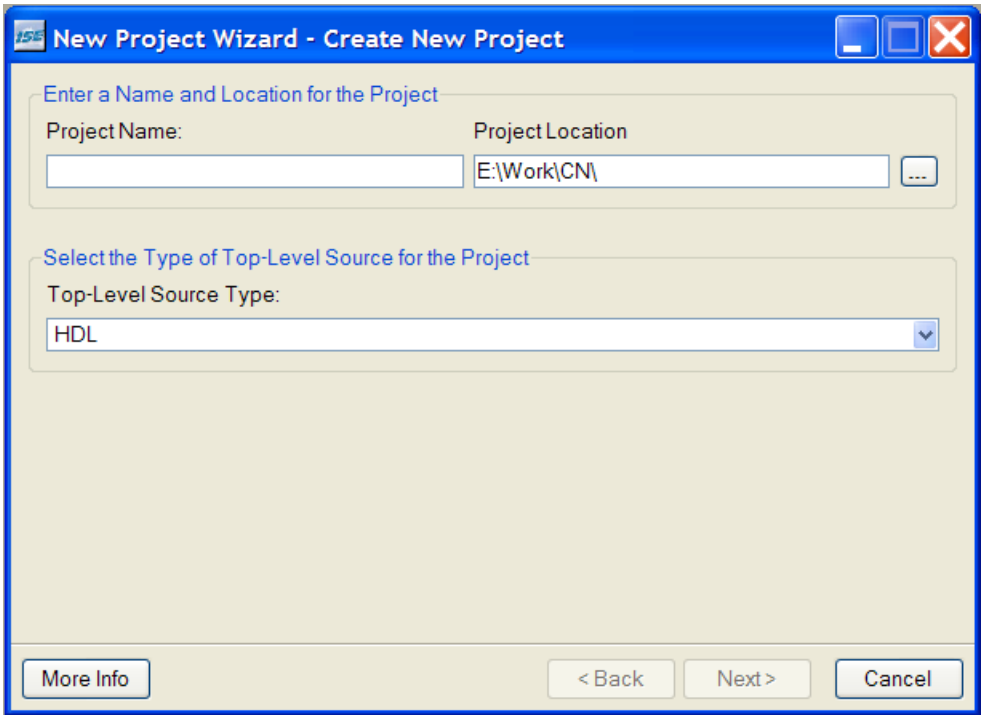
Acest laborator este o introducere in proiectarea logica utilizand Verilog cu ajutorul mediului Xilinx ISE 9.2i. In cadrul laboratorului nu vor fi prezentate noi concepte de proiectare logica. Scopul acestuia este de a va familiariza cu uneltele de proiectare pe care le veti utiliza in acest semestru.

- Xilinx ISE
- Digilent D2SB
- Mentor Graphics ModelSim

Pentru a obtine punctajul acordat acestui laborator, trebuie sa prezentati laborantului functionarea corecta in hardware a proiectului dumneavoastra.

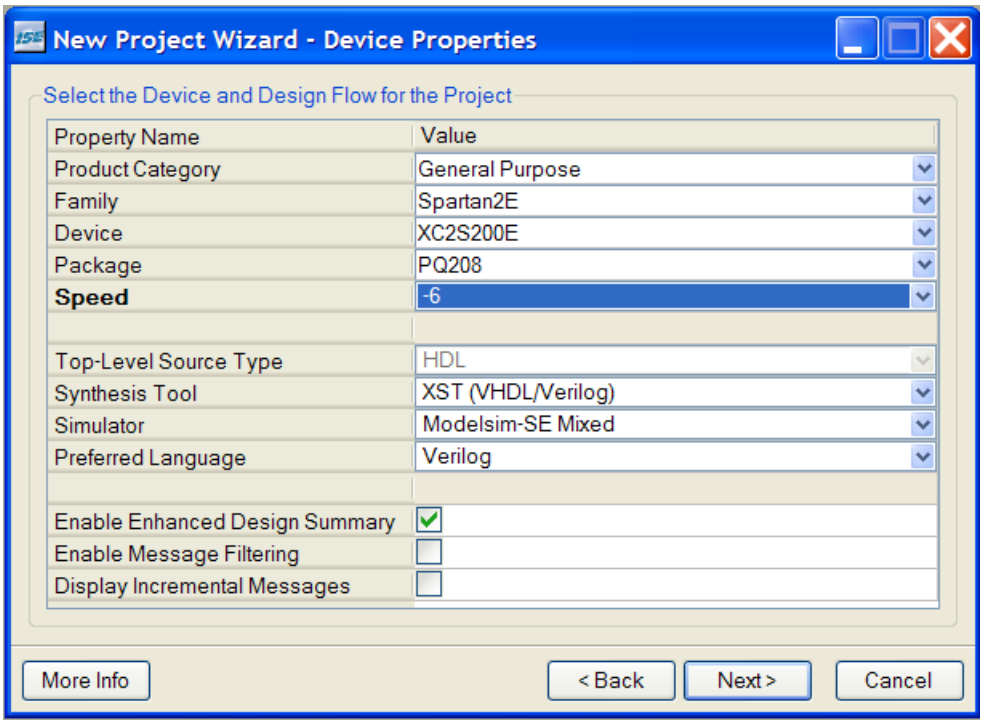
Proiectarea

Proiectul din acest laborator isi propune sa implementeze o functie logica cu doua intrari si o iesire (de exemplu un XOR). Dupa pornirea aplicatiei ISE, selectati meniul File -> New Project. Prima fereastra de dialog New Project va aparea ca in figura de mai jos:

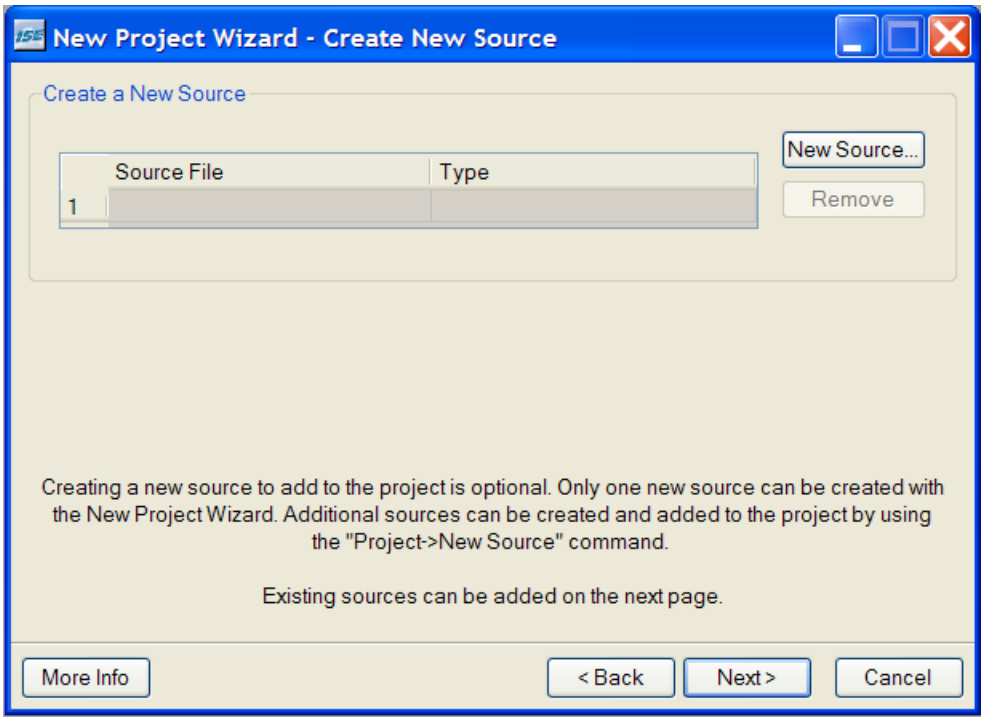


Sunteti solicitati sa introduceti numele proiectului dumneavoastra (lab1), locatia unde acesta va fi salvat si tipul modulului de top. Sunteti sfatuiti sa va salvati proiectele proprii (nu va bazati pe

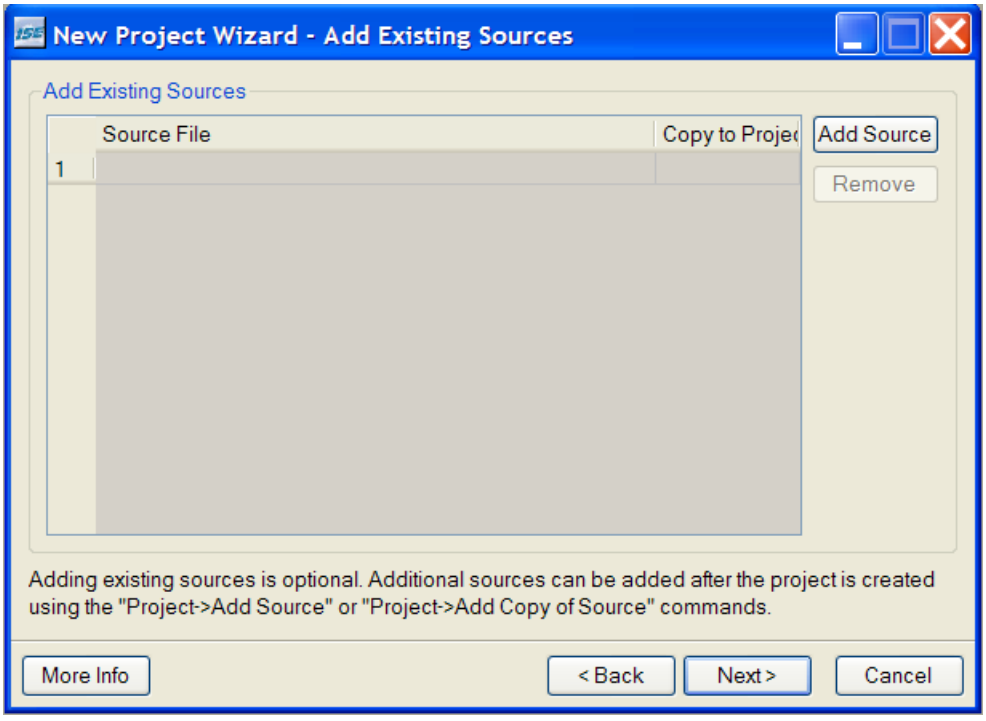
copia de pe calculatorul din laborator deoarece aceasta poate fi stearsa). Dupa introducerea datelor dorite selectati Next.
In urmatoarea fereastra de dialog sunt solicitate datele privitoare la resursele ce sunt la dispozitie pe placa de dezvoltare D2SB si limbajul de descriere, unealta pentru sinteza si cea pentru simulare (ca in figura de mai jos). Dupa selectare obtiunilor dorite selectati Next.



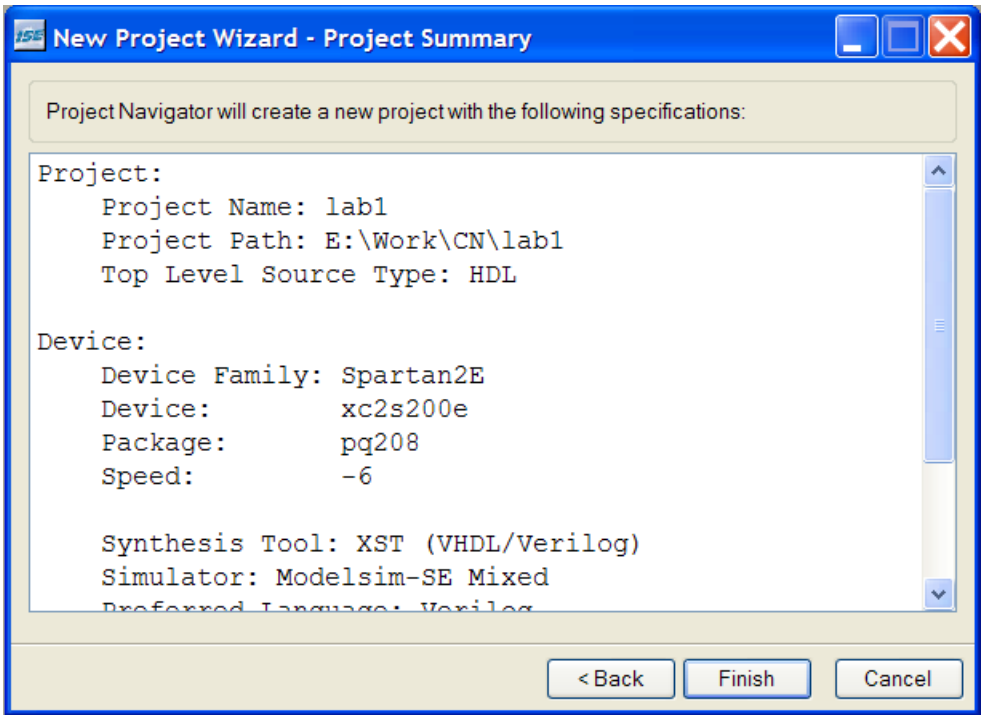
Urmatoarea fereastra de dialog va permite adaugarea unor noi surse in proiectul dumneavoastra. Nu adaugati nici o sursa si selectati Next.



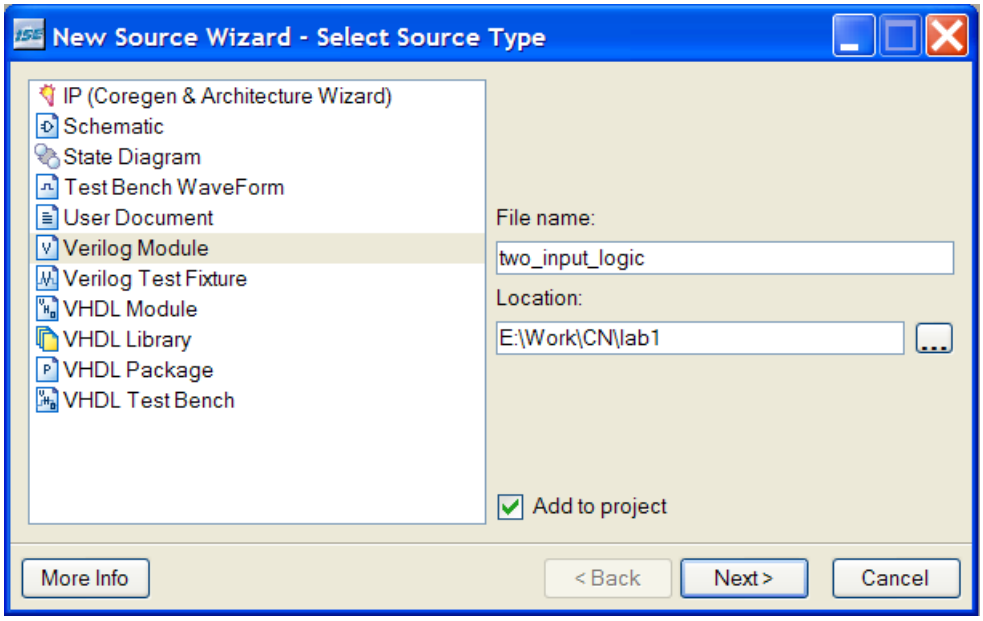
In urmatoarea fereastra de dialog aveti posibilitatea de a adauga surse existente in proiectul dumneavoastra (aceasta facilitate este utila pentru reutilizarea codului). Nu adaugati nici o sursa si selectati Next.



Ultima fereastra de dialog New Project va prezinta sumarul celor selectate anterior. Daca datele prezentate nu corespund dorinteleo dumneavoastra selectati Back si corectati erorile. Daca sunteti multumiti de rezultat selectati Finish ca in figura urmaotare.

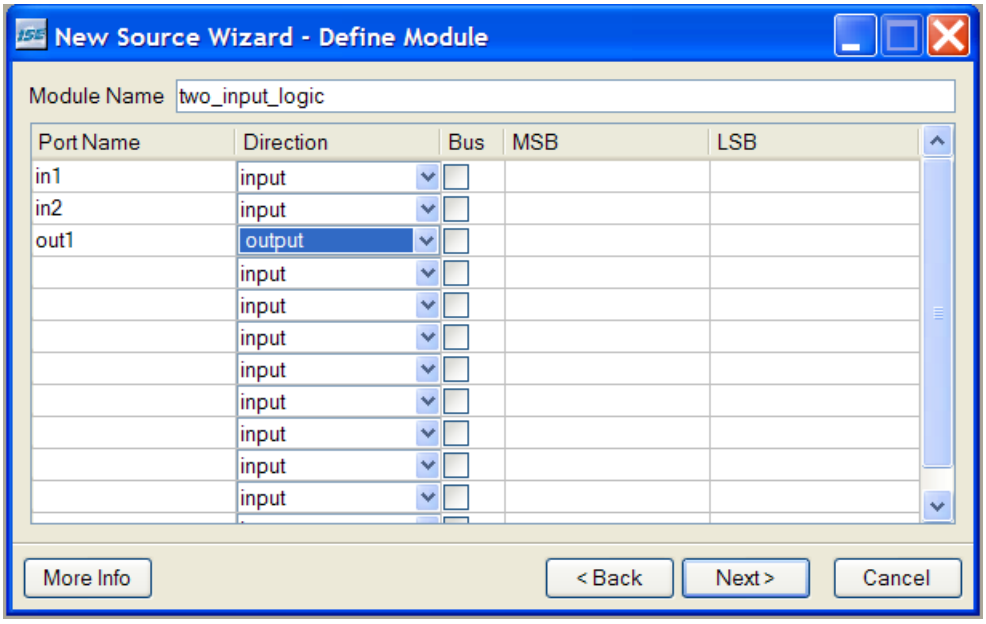


In acest moment ati creat un nou proiect ce nu contine nici o sursa. Pentru a adauga o noua sursa, selectati Create New Souce din meniul Process for: xc2s200e-6pq208 si prima fereastra de dialog New Source va aparea ca in figura de mai jos:

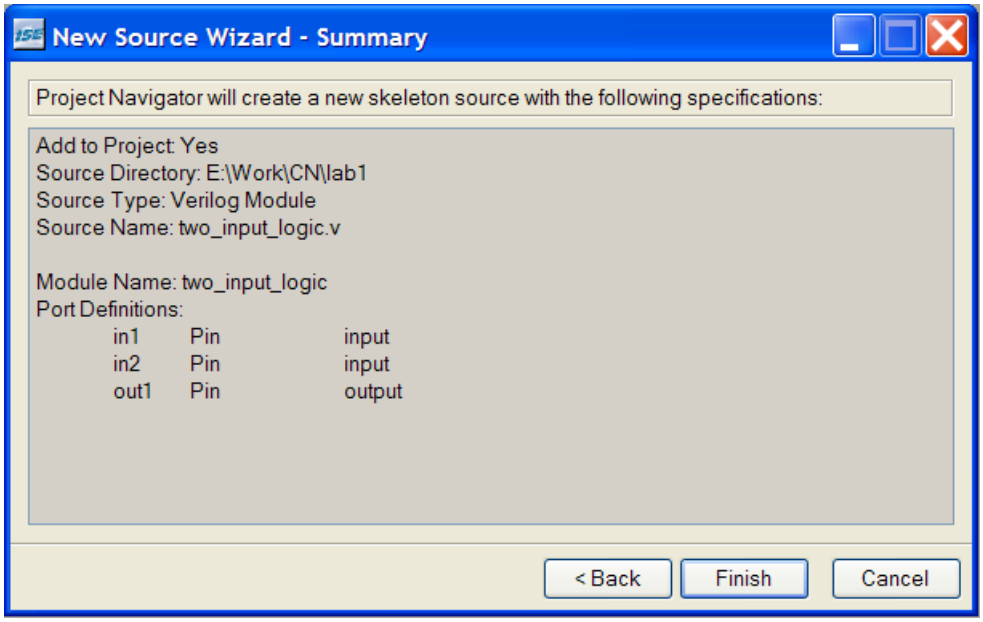


Selectati Verilog Module pentru a indica faptul ca doriti sa creati un modul verilog si atribuiti un nume pentru acesta apoi selectati Next.

Urmatoarea fereastra de dialog va perimite sa definiti intrarile si iesirile modulului dumneavoastra (acestea fot fi schimbate si ulterior in editorul de text). Dupa ce introduceti informatiile necesare selectati Next.



Ultima fereastra de dialog New Source prezinta rezumatul celor selectate anterior. Selectati Finish. Noua sursa va fi deschisa automat in editorul de text.

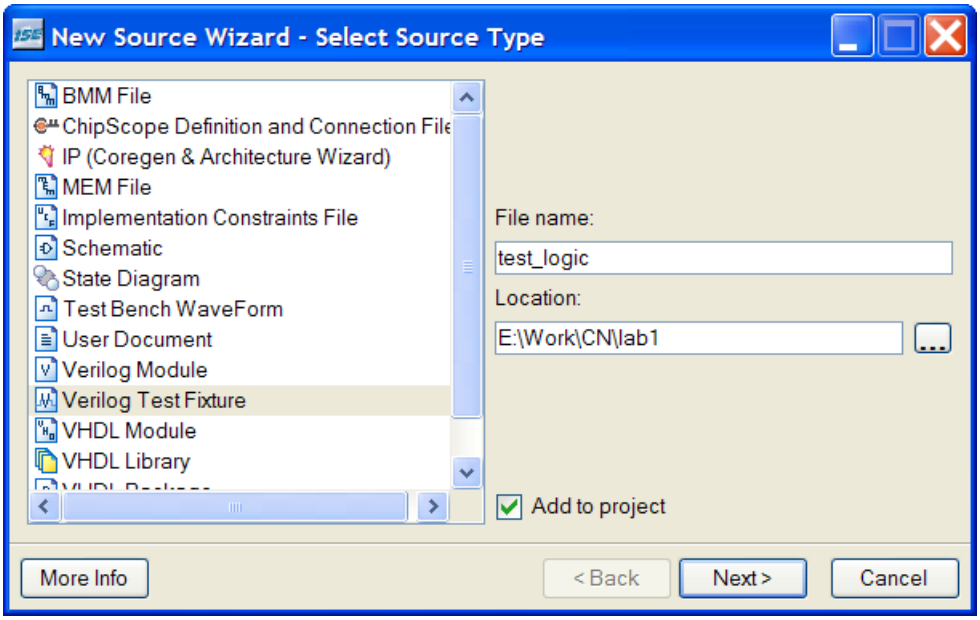


Completati sursa pentru a implementa operatia dorita dupa cum urmeaza:

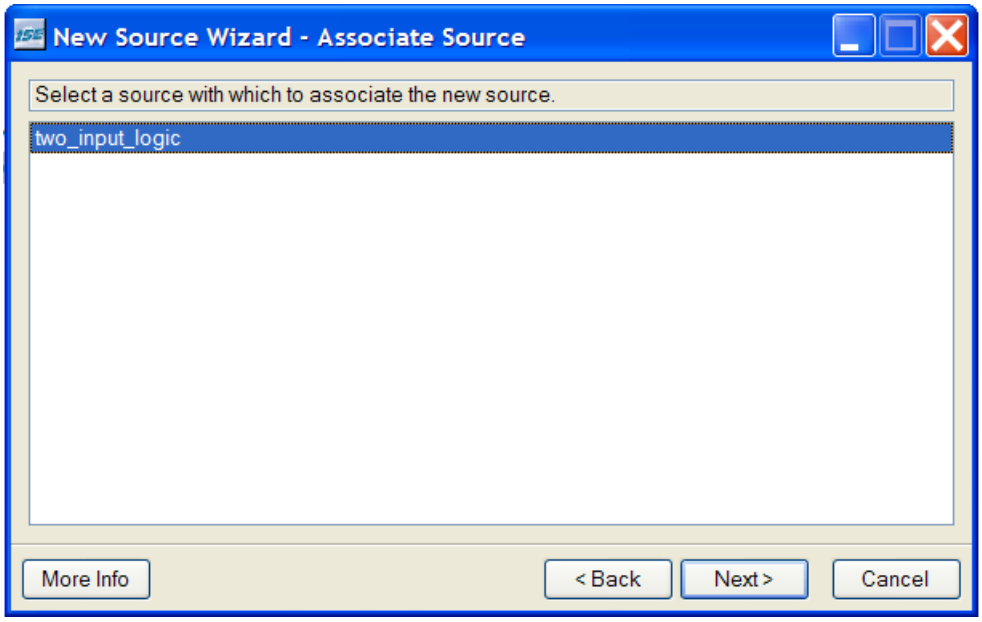
```
21  module two_input_logic(in1, in2, out1);
22      input in1;
23      input in2;
24      output out1;
25
26      assign out1 = in1 ^ in2;
27
28  endmodule
```

Simularea functionala

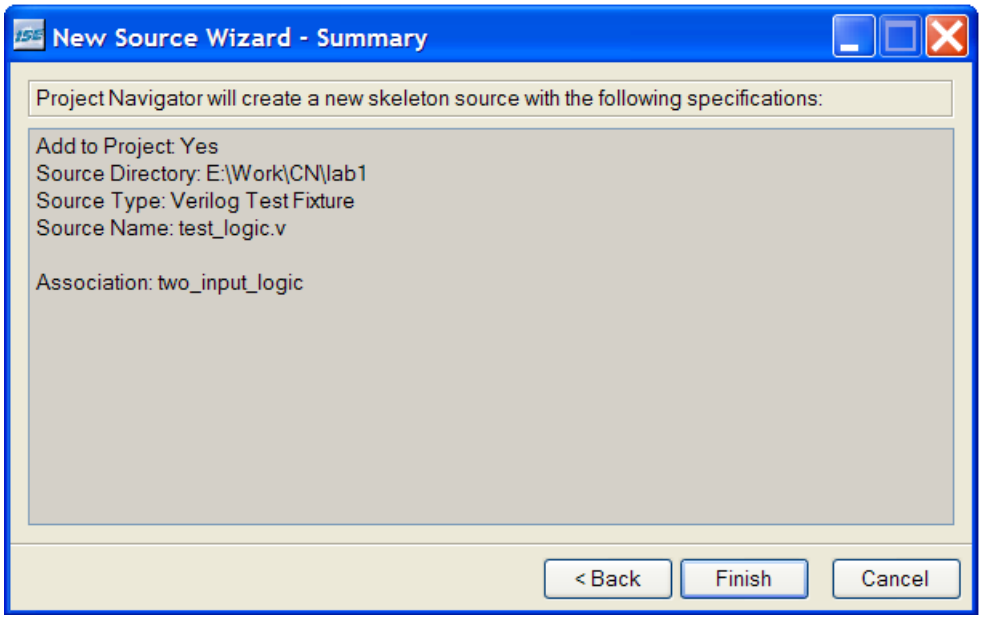
Simularea functionala are rolul de a verifica logica proiectata inainte de sinteza pentru a elimina eventualele greseli sau neconcordante intre specificatie si implementare.
Mediul ISE se integreaza perfect cu simulatorul ModelSim si permite lansarea simularii din mediul de dezvoltare. Pentru a simula un proiect este nevoie sa creati un modul de test. Acest lucru se face similar cu procesul de a crea o sursa noua selectand obtiunile ca in figura urmatoare:



Selectati Verilog Test Fixture pentru a genera un modul de test Verilog. Apoi atribuiti un nume pentru acest modul si selectati Next.
A doua fereastra de dialog va permite sa alegeti modulul pentru care doriti sa creati testul (in acest caz lista contine un singur modul: two_input_logic).



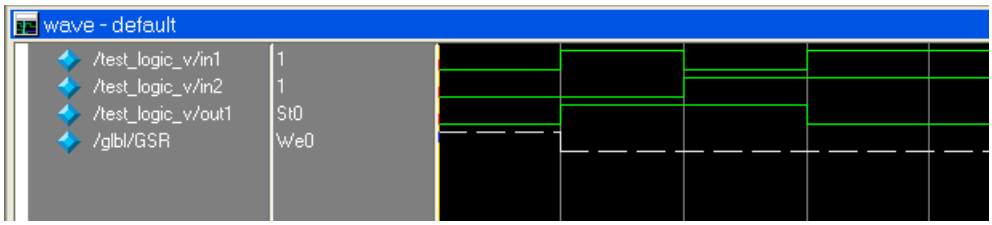
Ultima fereastra de dialog va prezinta sumarul obtiunilor anterioare. Selectati Finish.



Dupa crearea modulului de test, acesta va fi automat descis in editorul de test si trebuie completat cu valorile stimulilor de intrare (in1 si in2) pentru a testa diferitele combinatii. Un exemplu este oferit in cele ce urmeaza:

```
25  module test_logic_v;
26
27     // Inputs
28     reg in1;
29     reg in2;
30
31     // Outputs
32     wire out1;
33
34     // Instantiate the Unit Under Test (UUT)
35     two_input_logic uut (
36        .in1(in1),
37        .in2(in2),
38        .out1(out1)
39     );
40
41     initial begin
42        // Initialize Inputs
43        in1 = 0;
44        in2 = 0;
45
46        // Wait 100 ns for global reset to finish
47        #100;
48
49        // Add stimulus here
50        in1 = 1;
51        in2 = 0;
52
53        // Wait 100 ns for global reset to finish
54        #100;
55        in1 = 0;
56        in2 = 1;
57
58        // Wait 100 ns for global reset to finish
59        #100;
60        in1 = 1;
61        in2 = 1;
62
63     end
64
65  endmodule
```

Pentru a porni simularea selectati Sources for: Behaivioral Simulation pentru a putea selecta modulul de test nou creat. Dupa selectare, in meniul de proces (Processes for: test_logic_v) puteti selecta Simulate Behaivioral Model (de la ModelSim Simulator). Simulatorul va fi pornit automat si va simula circuitul implementat si stimulat de tatele de intrare (conform testului). Rezultatele pot fi urmarite in fereastra Wave din ModelSim.



Sinteza circuitului proiectat

Cu o descriere in Verilog corecta, din punct de vedere functional, se poate trece la transforamrea acesteia intr-un netlist (prin sinteza). Un netlist este o reprezentare a unei scheme logice pe intelesul masinii. In cazul de fata vom folosi XST care este integrat in ISE si poate realiza sinteza pentru dispozitive produse de Xilinx.
Pentru pornirea sintezei trebuie selectat intai modulul principal (two_input_logic) din meniul Sources for: Synthesis/Implementation si apoi din meniul Processes for: two_input_logic selectati Synthesis – XST. In consola mediului de dezvoltare ISE veti putea urmari mesajele generate in urma procesului de sinteza. In mod normal nu ar trebui sa apara nici o eroare in cadrul acestui proces deoarece codul a fost validat anterior. Pentru a analiza raportul sintezei puteti selecta View Synthesis Report.

Release 9.2.02i - xst J.38
Copyright (c) 1995-2007 Xilinx, Inc. All rights reserved.
--> Parameter TMPDIR set to ./xst/projnav.tmp
CPU : 0.00 / 0.81 s | Elapsed : 0.00 / 0.00 s

--> Parameter xsthdpdir set to ./xst
CPU : 0.00 / 0.81 s | Elapsed : 0.00 / 0.00 s

--> Reading design: two_input_logic.prj

TABLE OF CONTENTS
  1) Synthesis Options Summary
  2) HDL Compilation
  3) Design Hierarchy Analysis
  4) HDL Analysis
  5) HDL Synthesis
    5.1) HDL Synthesis Report
  6) Advanced HDL Synthesis
    6.1) Advanced HDL Synthesis Report
  7) Low Level Synthesis
  8) Partition Report
  9) Final Report
    9.1) Device utilization summary
    9.2) Partition Resource Summary
    9.3) TIMING REPORT

```
=========================================================================
*                      Synthesis Options Summary                        *
=========================================================================
```

---- Source Parameters
Input File Name : "two_input_logic.prj"
Input Format : mixed
Ignore Synthesis Constraint File : NO

---- Target Parameters
Output File Name : "two_input_logic"
Output Format : NGC
Target Device : xc2s200e-6-pq208

---- Source Options
Top Module Name : two_input_logic
Automatic FSM Extraction : YES
FSM Encoding Algorithm : Auto
Safe Implementation : No
FSM Style : lut
RAM Extraction : Yes
RAM Style : Auto
ROM Extraction : Yes
Mux Style : Auto
Decoder Extraction : YES
Priority Encoder Extraction : YES
Shift Register Extraction : YES
Logical Shifter Extraction : YES
XOR Collapsing : YES
ROM Style : Auto
Mux Extraction : YES
Resource Sharing : YES
Asynchronous To Synchronous : NO
Multiplier Style : lut
Automatic Register Balancing : No

---- Target Options
Add IO Buffers : YES
Global Maximum Fanout : 100
Add Generic Clock Buffer(BUFG) : 4
Register Duplication : YES
Slice Packing : YES
Optimize Instantiated Primitives : NO
Convert Tristates To Logic : Yes
Use Clock Enable : Yes
Use Synchronous Set : Yes
Use Synchronous Reset : Yes
Pack IO Registers into IOBs : auto
Equivalent register Removal : YES

---- General Options
Optimization Goal : Speed
Optimization Effort : 1
Library Search Order : two_input_logic.lso
Keep Hierarchy : NO
RTL Output : Yes
Global Optimization : AllClockNets
Read Cores : YES
Write Timing Constraints : NO
Cross Clock Analysis : NO

```
Hierarchy Separator              : /
Bus Delimiter                    : <>
Case Specifier                   : maintain
Slice Utilization Ratio          : 100
BRAM Utilization Ratio           : 100
Verilog 2001                     : YES
Auto BRAM Packing                : NO
Slice Utilization Ratio Delta    : 5

=========================================================================


=========================================================================
*                     HDL Compilation                   *
=========================================================================
Compiling verilog file "two_input_logic.v" in library work
Module <two_input_logic> compiled
No errors in compilation
Analysis of file <"two_input_logic.prj"> succeeded.


=========================================================================
*                  Design Hierarchy Analysis            *
=========================================================================
Analyzing hierarchy for module <two_input_logic> in library <work>.


=========================================================================
*                     HDL Analysis                      *
=========================================================================
Analyzing top module <two_input_logic>.
Module <two_input_logic> is correct for synthesis.


=========================================================================
*                     HDL Synthesis                     *
=========================================================================

Performing bidirectional port resolution...

Synthesizing Unit <two_input_logic>.
    Related source file is "two_input_logic.v".
    Found 1-bit xor2 for signal <out1>.
Unit <two_input_logic> synthesized.


=========================================================================
HDL Synthesis Report

Macro Statistics
# Xors                           : 1
 1-bit xor2                      : 1

=========================================================================

=========================================================================
```

```
*                  Advanced HDL Synthesis                  *
=========================================================================

Loading device for application Rf_Device from file '2s200e.nph' in environment C:\Xilinx92i.

=========================================================================
Advanced HDL Synthesis Report

Macro Statistics
# Xors                                : 1
 1-bit xor2                          : 1

=========================================================================

=========================================================================
*                  Low Level Synthesis                  *
=========================================================================

Optimizing unit <two_input_logic> ...

Mapping all equations...
Building and optimizing final netlist ...
Found area constraint ratio of 100 (+ 5) on block two_input_logic, actual ratio is 0.

Final Macro Processing ...

=========================================================================
Final Register Report

Found no macro
=========================================================================

=========================================================================
*                  Partition Report                  *
=========================================================================

Partition Implementation Status
-------------------------------

  No Partitions were found in this design.

-------------------------------

=========================================================================
*                  Final Report                  *
=========================================================================
Final Results
RTL Top Level Output File Name     : two_input_logic.ngr
Top Level Output File Name         : two_input_logic
Output Format                      : NGC
Optimization Goal                  : Speed
Keep Hierarchy                     : NO

Design Statistics
# IOs                              : 3
```

Cell Usage :
BELS : 1
LUT2 : 1
IO Buffers : 3
IBUF : 2
OBUF : 1
==========

Device utilization summary:

Selected Device : 2s200epq208-6

Number of Slices: 1 out of 2352 0%
Number of 4 input LUTs: 1 out of 4704 0%
Number of IOs: 3
Number of bonded IOBs: 3 out of 142 2%

Partition Resource Summary:

No Partitions were found in this design.

==========
TIMING REPORT

NOTE: THESE TIMING NUMBERS ARE ONLY A SYNTHESIS ESTIMATE.
FOR ACCURATE TIMING INFORMATION PLEASE REFER TO THE TRACE REPORT
GENERATED AFTER PLACE-and-ROUTE.

Clock Information:

No clock signals found in this design

Asynchronous Control Signals Information:

No asynchronous control signals found in this design

Timing Summary:

Speed Grade: -6

Minimum period: No path found
Minimum input arrival time before clock: No path found
Maximum output required time after clock: No path found
Maximum combinational path delay: 7.707ns

Timing Detail:

All values displayed in nanoseconds (ns)

==========

```
Timing constraint: Default path analysis
  Total number of paths / destination ports: 2 / 1
-------------------------------------------------------------------------
Delay:               7.707ns (Levels of Logic = 3)
  Source:            in2 (PAD)
  Destination:       out1 (PAD)

  Data Path: in2 to out1
                        Gate     Net
    Cell:in->out      fanout   Delay   Delay  Logical Name (Net Name)
    ----------------------------------------  ------------
     IBUF:I->O             1   0.797   0.920  in2_IBUF (in2_IBUF)
     LUT2:I0->O            1   0.468   0.920  Mxor_out1_Result1 (out1_OBUF)
     OBUF:I->O                 4.602          out1_OBUF (out1)
    ----------------------------------------
    Total                     7.707ns (5.867ns logic, 1.840ns route)
                              (76.1% logic, 23.9% route)

=========================================================================
CPU : 4.17 / 5.14 s | Elapsed : 5.00 / 5.00 s

-->

Total memory usage is 131760 kilobytes

Number of errors   :    0 (   0 filtered)
Number of warnings :    0 (   0 filtered)
Number of infos    :    0 (   0 filtered)
```

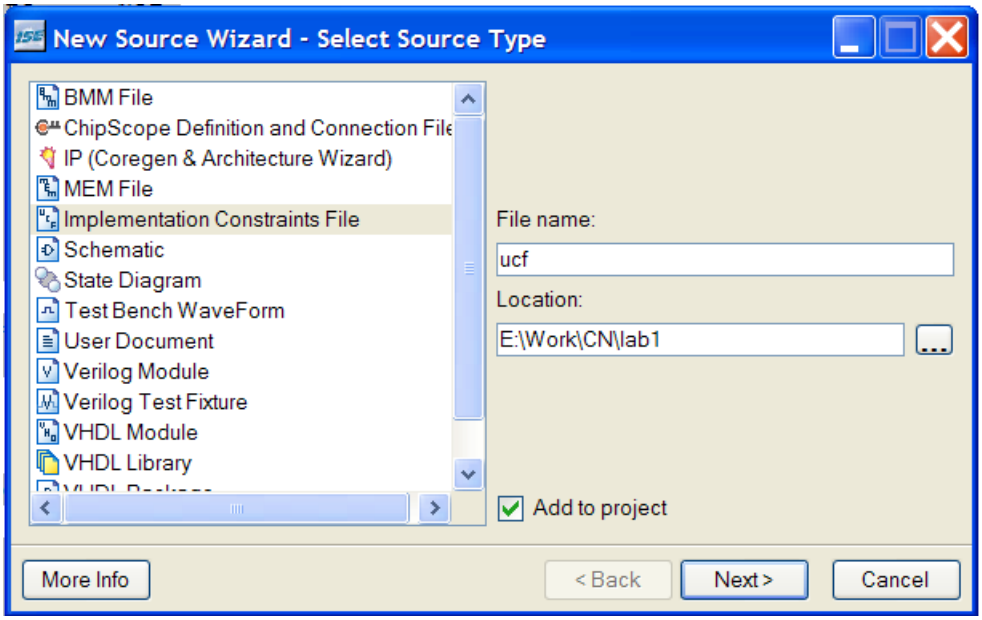
In urma analizei raportului se poate observa cate (si ce tip) de resurse utilizeaza proiectul dumneavoastra. Aceasta informatie este utila in cazul productiei cand se doreste alegerea circuitului care se potriveste cel mai bine pentru respectivul proiect (pentru a nu face risipa de resurse).

Implementarea proiectului

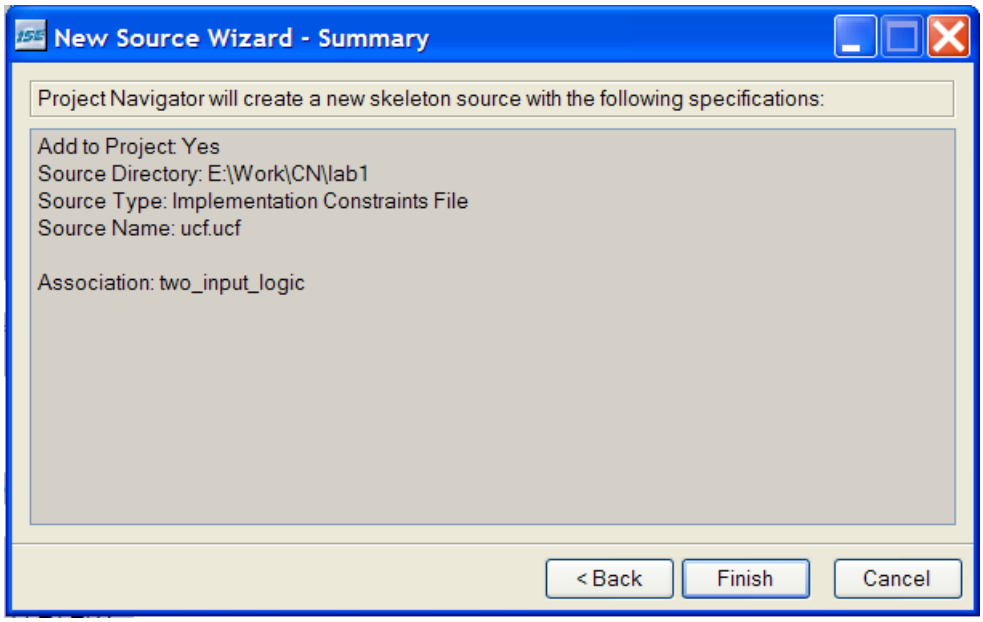
Implementarea proiectului consta intr-o succesiune de evenimente ce conduce la transforamrea codului sintetizat in netlist intr-un fisier de configurare pentru FPGA-ul ales. Uneltele de implementare (incluse in ISE) trebuie sa stie la ce pini ai FPGA-ului sa conecteze intrarile si iesirile proiectului dumneavoastra. Daca acest lucru nu este definit, ele vor fi distribuite aleator pinilor disponibili.
Pentru proiectul propus, vom folosi doua switchuri pentru intrari si un led pentru iesire.
Lista pinilor conectati la diversele resurse ale placi de I/O sunt prezentati in tabelul din anexa I.
Pentru acest proiect vom conecta:

SW1	P23
SW2	P22
LED1	P11

Dupa ce am adunat informatiile necesare, putem trece la realizarea fisierului UCF (fisierul de constrangeri ale proiectului). Acest fisier defineste corespondenta intre semnalele modulului de top si pinii FPGA-ului precum si alte setari de constrangere (cum ar fi tipul iesirilor, etc). Pentru a pastra simplitatea proiectului vom face numai setarile necesare pentru conectarea pinilor.
Este util ca toate constrangerile impuse proiectului sa fie definite intr-un fisier separat ce nu depinde de codul sintetizat.
Pentru a crea fisierul de constrangeri selectati din nou Create New Source.



Selectati Implementation Constraints File si atribuiti un nume (ucf). Apoi selectati Next.



Dupa ce analizati informatiile din sumar, selectati Finish.

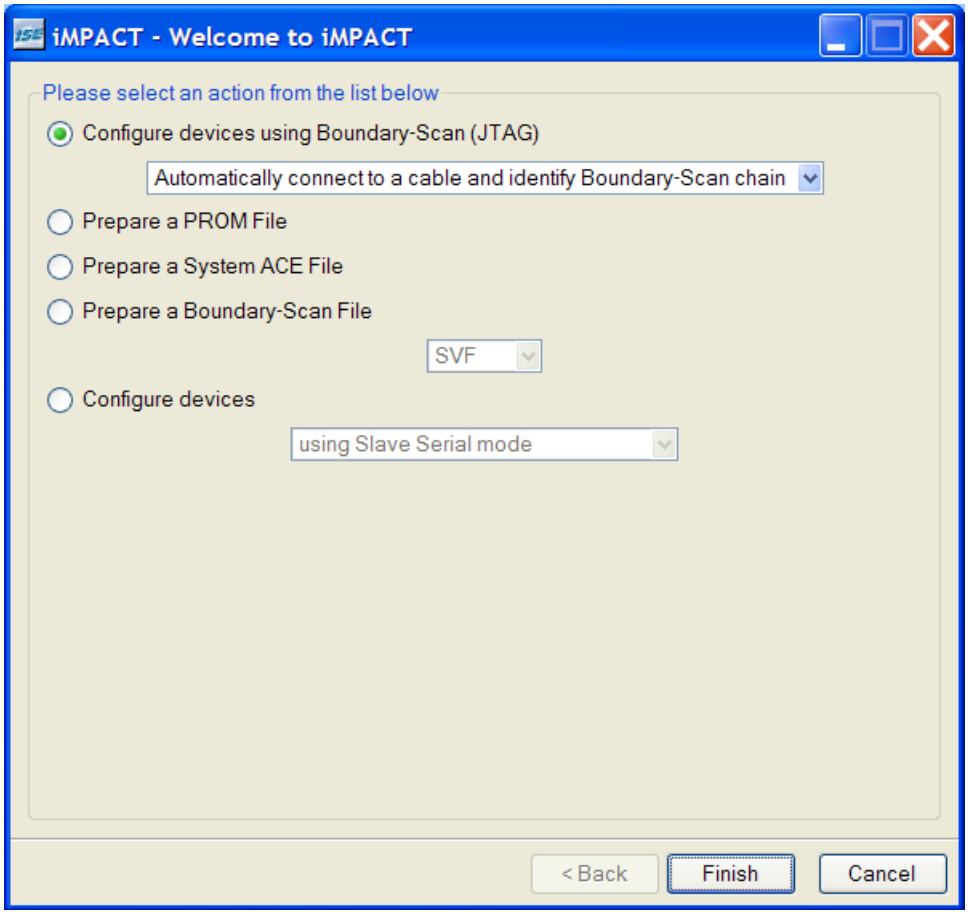
Sub ierarhia modulului de top (two_input_logic) va aparea un nou fisier (ucf.ucf). Fisierul UCF este un fisier text (ca si sursele Verilog). Pentru simplitatea exemplului il vom ediata direct. Selectati fisierul ucf.ucf din meniul Sources for: Synthesis/Implementation si apoi din meniul Processes for: ucf.ucf, Edit Constraints (Text).
Fisierul rezultat va contine urmatoarele trei linii:

```
1  NET in1 LOC="P23";
2  NET in2 LOC="P22";
3  NET out1 LOC="P11";
```

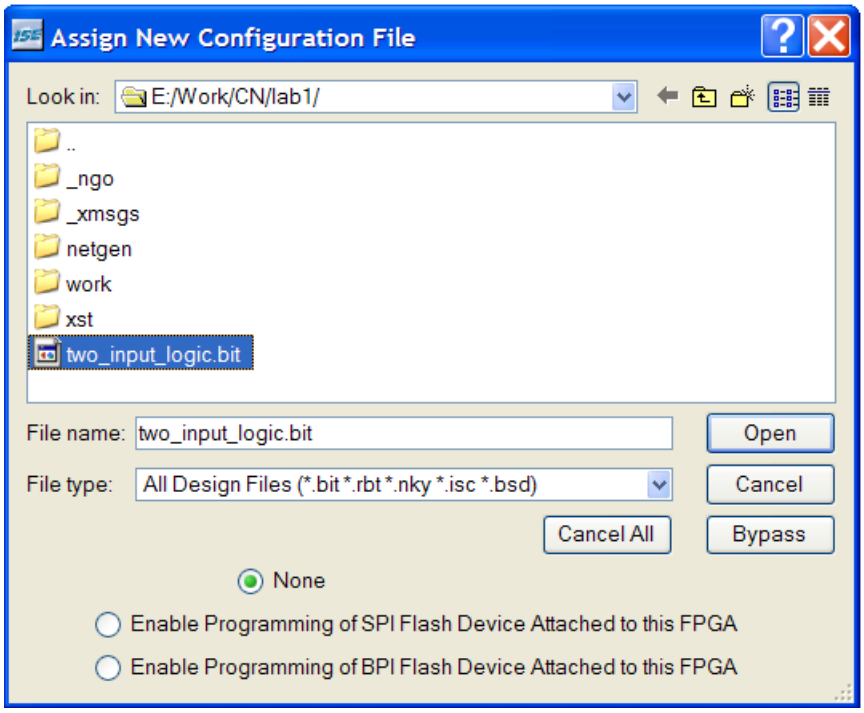
Semnificatia lor este urmaorarea: semnalului in1/in2/out1 i se atribiue pinul cu de la locatia P23/P22/P11 al FPGA-ului.
Dupa ce fisierul de constrangeri este complet, se poate trece la implementarea proiectului. Selectati din nou modulul de top (two_input_logic) si apoi Implement Design in meniul Processes for: two_input_logic.

Programarea FPGA-ului

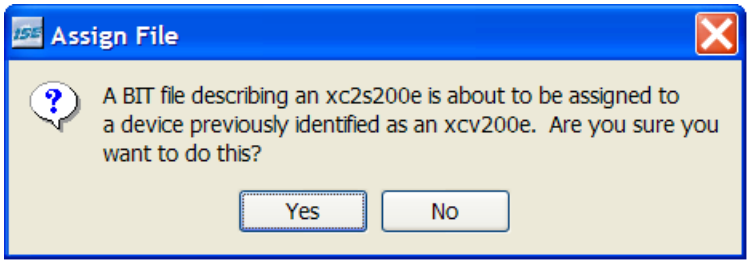
Pentru a putea programa FPGA-ul trebuie creat intai fisierul de configurare. Acesta se creaza selectand Generate Programming File din meniul Processes for: two_input_logic.
Dupa generare, fisierul (cu extensia .bit) trebuie transferat pe FPGA-ul de pe placa de test. Aceasta trebuie sa fie alimentata si conectata la calculator prin cablul de programare (JTAG).
Utilitarul ce se ocupa de configurare se numeste iMPACT si se porneste selectand Configure Device (iMPACT).



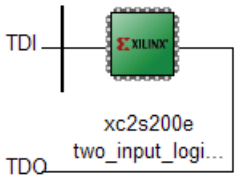
Pentru a detecta circuitul selectati Finish.



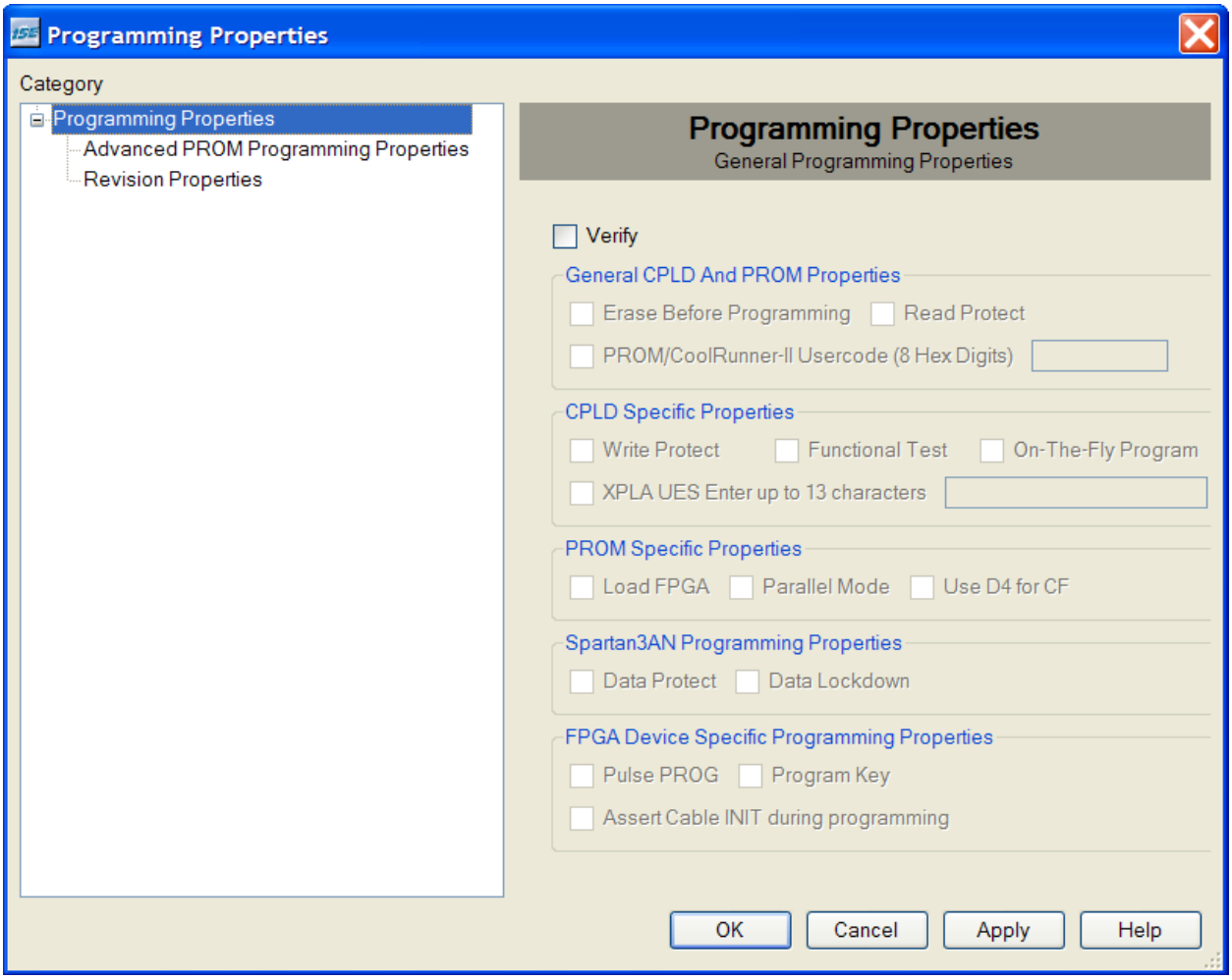
Selectati fisierul .bit ce doriti sa il transferati si apoi selectati Open. Confirmati ca doriti sa programati FPGA-ul cu fisierul .bit selectand Yes.



In acest moment lantul de programare este gata. Tot ce mai trebuie facut este sa porniti programarea selectand circuitul din lant si optiunea Program.



In fereastra de dialog deschisa, selectati OK.



Dupa programare se poate trece la verificarea functionarii prin actionarea switchurilor de pe placa de test si observarea ledului corespunzator.



Exercitii suplimentare

Pentru exersarea celor invatate in acest laborator va propunem implementarea altor functii logice (mai complexe) avand ca intrare cele 8 switchuri si ca iesire cele 8 leduri disponibile pe placa de test.

Început la	vineri, 19 iunie 2020, 08:05
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:35
Timp luat	29 min 49 secs
Notează	**8,00** din maxim 10,00 (**80**%) posibil

1 întrebare

Corect

Marcat 1,00 din 1,00

Adevarat sau fals: O intrerupere ne-mascabila se executa intotdeauna de catre procesor.

Selectați o opțiune:

- Adevărat ✔
- Fals

Răspunsul corect este "Adevărat".

2 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor petrece 50% din timp cu operatii UAL si 40% in operatii cu memoria. Daca puteti imbunatati timpul de acces la memorie de 4 ori, dar, ca rezultat veti incetini executia celorlalte instructiuni cu 50%, ce speedup obtineti?

Alegeți o opțiune:

- a. **3.33**
- b. **nu poate fi determinat**
- c. **1** ✔
- d. **5.3**
- e. **2**

Răspunsul corect este: **1**

3 întrebare

Incorect

Marcat 0,00 din 1,00

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	0x01	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **8**
- b. **B**
- c. **9** ✖
- d. **A**
- e. **Nu se poate determina**

Răspunsul corect este: **Nu se poate determina**

4 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că un procesor fără b.a. execută următoarea operație: *addi r14, 0xda*. În care etapă este actualizat *r14*?

Alegeți o opțiune:

- a. **Decode**
- b. **Execute**
- c. **Write Back** ✔
- d. **Instruction Fetch**
- e. **Memory**

Răspunsul corect este: **Write Back**

5 întrebare

Incorect

Marcat 0,00 din 1,00

Întreruperile mascabile:

Alegeți o opțiune:

- a. **Produc bule de exact o instrucțiune lungime în b.a.** ✖
- b. **Sunt întotdeauna sincrone cu ceasul sistemului**
- c. **Poartă și numele de excepții**
- d. **Pot fi implementate doar de procesoarele RISC**
- e. **Pot să nu fie tratate de procesor**

Răspunsul corect este: **Pot să nu fie tratate de procesor**

6 întrebare

Corect

Marcat 1,00 din 1,00

Numărul -1.75, poate fi reprezentat în cod IEEE 754 single precision ca:

Alegeți o opțiune:

- a. **0xcfe00000**
- b. **Nicio opțiune**
- c. **0xdfe00000**
- d. **0xafe00000**
- e. **0xbfe00000** ✔

Răspunsul corect este: **0xbfe00000**

7 întrebare

Corect

Marcat 1,00 din 1,00

Dacă trebuie să proiectați un procesor performant (frecvență mare de ceas, superscalar) folosind tehnologia actuală, ce fel de ISA ați alege pentru acesta?

Alegeți o opțiune:

- a. **CISC**
- b. **Mașină virtuală Java**
- c. **RISC** ✔
- d. **MISC**
- e. **VLIW**

Răspunsul corect este: **RISC**

8 întrebare

Corect

Marcat 1,00 din 1,00

De ce este important sa stim daca o masina este big-endian sau little-endian?

Alegeți o opțiune:

- a. **Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare**
- b. **Pentru ca anumite programe scriu date in memorie intr-o anumita ordine**
- c. **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare** ✔
- d. **Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers**

Răspunsul corect este: **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare**

9 întrebare

Corect

Marcat 1,00 din 1,00

Hazardele de control:

Alegeți o opțiune:

- a. **Sunt determinate în etapa de Decode sau Execute**
- b. **Sunt produse doar de operații tip JMP**
- c. **Influențează adresa următoare din Instruction Register** ✔
- d. **Necesită o unitate dedicată de control**
- e. **Apar doar la arhitecturile RISC**

Răspunsul corect este: **Influențează adresa următoare din Instruction Register**

10 întrebare

Corect

Marcat 1,00 din 1,00

Pentru a putea rula la 0.5GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 2V, iar curentul de mers în gol al acestuia este 10mA. Știind că procesorul are o capacitate parazită echivalentă de 300nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:

- a. **300.02W** ✔
- b. **100.1W**
- c. **300.03W**
- d. **50W**
- e. **300W**

Răspunsul corect este: **300.02W**

◄ Inregistrare curs 11&12 Salt la... Examen - Partea II ►

Început la	vineri, 19 iunie 2020, 08:08
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:46
Timp luat	38 min
Notează	**9,00** din maxim 10,00 (**90**%) posibil

1 întrebare

Corect

Marcat 1,00 din 1,00

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	HLT	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **9** ✔
- b. **Nu se poate determina**
- c. **8**
- d. **A**
- e. **B**

Răspunsul corect este: **9**

2 întrebare

Corect

Marcat 1,00 din 1,00

Adevarat sau fals: Procesorul cu banda de asamblare executa toate instructiunile intr-un singur ciclu de ceas.

Selectați o opțiune:

- Adevărat
- Fals ✔

Răspunsul corect este "Fals".

3 întrebare

Incorect

Marcat 0,00 din 1,00

Dacă îmbunătățim timpul de acces la memorie, atunci în ce sens putem opera asupra frecvenței procesorului?

Alegeți o opțiune:

- a. **Rămâne neschimbată**
- b. **Nu se poate determina** ✖
- c. **Se apropie de 3GHz**
- d. **Creștere**
- e. **Scădere**

Răspunsul corect este: **Creștere**

4 întrebare

Corect

Marcat 1,00 din 1,00

De ce este important sa stim daca o masina este big-endian sau little-endian?

Alegeți o opțiune:

- a. **Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare**
- b. **Pentru ca anumite programe scriu date in memorie intr-o anumita ordine**
- c. **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare** ✔
- d. **Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers**

Răspunsul corect este: **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare**

5 întrebare

Corect

Marcat 1,00 din 1,00

Numarul 0xC000FEFE, stocat ca *signed int (32 de biți)* este:

Alegeți o opțiune:

- a. **Nicio opțiune**
- b. **Pozitiv**
- c. **Prea mare pentru a putea fi reprezentat**
- d. **Negativ** ✔

Răspunsul corect este: **Negativ**

6 întrebare

Corect

Marcat 1,00 din 1,00

Care este, în medie, efectul pe care dublarea numărului de registre generale îl are asupra ratei apariției hazardelor de date?

Alegeți o opțiune:

- a. **Va crește**
- b. **Ramâne constantă**
- c. **Va scade** ✔
- d. **Imposibil de determinat**
- e. **Le elimină complet**

Răspunsul corect este: **Va scade**

7 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că un procesor în b.a. execută următoarea operație: *addi r1, 0xbeef*. În ce etapă este actualizat *r1*?

Alegeți o opțiune:

- a. **Execute**
- b. **Decode**
- c. **Memory**
- d. **Write Back** ✔
- e. **Instruction Fetch**

Răspunsul corect este: **Write Back**

8 întrebare

Corect

Marcat 1,00 din 1,00

Dacă trebuie să proiectați un procesor performant (frecvență mare de ceas, superscalar) folosind tehnologia actuală, ce fel de ISA ați alege pentru acesta?

Alegeți o opțiune:

- a. **Mașină virtuală Java**
- b. **RISC** ✔
- c. **MISC**
- d. **CISC**
- e. **VLIW**

Răspunsul corect este: **RISC**

9 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor petrece 40% din timp cu operații UAL, 30% în operații cu memoria și 30% din timp cu alte operații. Dacă puteți îmbunătăți performanța UAL-ului de 5 ori, accesul la memorie de 10 ori dar, ca rezultat veți încetini execuția celorlalte instrucțiuni cu 30%, ce speedup obțineți?

Alegeți o opțiune:

- a. **2** ✔
- b. **1.5**
- c. **3.33**
- d. **5**
- e. **10**

Răspunsul corect este: **2**

10 întrebare

Corect

Marcat 1,00 din 1,00

Pentru a putea rula la 1GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 1V, iar curentul de mers în gol al acestuia este 50mA. Știind că procesorul are o capacitate parazită echivalentă de 100nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:

- a. **100.1W**
- b. **50.05W** ✔
- c. **50W**
- d. **100.05W**
- e. **100W**

Răspunsul corect este: **50.05W**

◄ Inregistrare curs 11&12 Salt la... Examen - Partea II ►

Început la	vineri, 19 iunie 2020, 08:08
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:43
Timp luat	35 min 24 secs
Notează	**7,00** din maxim 10,00 (**70**%) posibil

1 întrebare

Incorect

Marcat 0,00 din 1,00

Dacă îmbunătățim timpul de acces la memorie, atunci în ce sens putem opera asupra frecvenței procesorului?

Alegeți o opțiune:

- a. **Se apropie de 3GHz**
- b. **Scădere**
- c. **Creștere**
- d. **Nu se poate determina** ✖
- e. **Rămâne neschimbată**

Răspunsul corect este: **Creștere**

2 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor petrece 60% din timp cu operații UAL. De câte ori mai rapidă trebuie să fie unitatea aritmetică logică pentru ca procesorul să ruleze de două ori mai repede?

Alegeți o opțiune:

- a. **5**
- b. **3**
- c. **2**
- d. **1.5**
- e. **6** ✔

Răspunsul corect este: **6**

3 întrebare

Corect

Marcat 1,00 din 1,00

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	HLT	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **9** ✔
- b. **Nu se poate determina**
- c. **B**
- d. **A**
- e. **8**

Răspunsul corect este: **9**

4 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor cu arhitectura Harvard:

Alegeți o opțiune:

- a. **Toate de mai sus**
- b. **Are memorii de date si instructiuni separate** ✔
- c. **Executa orice operatie intr-un singur ciclu de ceas**
- d. **Are o banda de asamblare cu cinci etape**

Răspunsul corect este: **Are memorii de date si instructiuni separate**

5 întrebare

Corect

Marcat 1,00 din 1,00

Hazardele de date:

Alegeți o opțiune:

- a. **Au latenta minima**
- b. **Sunt cauzate de UAL**
- c. **Apar doar la arhitecturile RISC**
- d. **Pot fi solutionate prin bypassing** ✔

Răspunsul corect este: **Pot fi solutionate prin bypassing**

6 întrebare

Corect

Marcat 1,00 din 1,00

Pentru a putea rula la 0.5GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 2V, iar curentul de mers în gol al acestuia este 10mA. Știind că procesorul are o capacitate parazită echivalentă de 200nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:

- a. **200.02W** ✔
- b. **200W**
- c. **100.1W**
- d. **100.01W**
- e. **50W**

Răspunsul corect este: **200.02W**

7 întrebare

Incorect

Marcat 0,00 din 1,00

Dacă CPU-ul este implementat în logică rapidă, în timp ce accesul la memorie este de cel puțin 10x mai lent, ce fel de ISA alegi pentru a contracara pe cât de mult posibil limitările hardware?

Alegeți o opțiune:

- a. **Mașină virtuală Java**
- b. **VLIW**
- c. **CISC**
- d. **RISC** ✖
- e. **MISC**

Răspunsul corect este: **CISC**

8 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că un procesor fără b.a. execută următoarea operație: *ldi r16, 0xdeadc0de*. În care etapă este actualizat *r16*?

Alegeți o opțiune:

- a. **Instruction Fetch**
- b. **Decode**
- c. **Memory**
- d. **Execute**
- e. **Write Back** ✔

Răspunsul corect este: **Write Back**

9 întrebare

Corect

Marcat 1,00 din 1,00

Numărul 2.25, poate fi reprezentat în cod IEEE 754 single precision ca:

Alegeți o opțiune:

- a. **0x40100000** ✔
- b. **0x40200000**
- c. **0x40600000**
- d. **Nicio opțiune**
- e. **0x40800000**

Răspunsul corect este: **0x40100000**

10 întrebare

Incorect

Marcat 0,00 din 1,00

Adevarat sau fals: Un procesor CISC este în medie mai lent decât un procesor RISC.

Selectați o opțiune:

- Adevărat ✖
- Fals

Răspunsul corect este "Fals".

◄ Inregistrare curs 11&12

Salt la...

Examen - Partea II ►

Teorie Examen CN

1. De ce sunt folosite tranzistoare MOSFET pentru procesoarele moderne?
   a) **Consumul de putere este mai mic decat la tranzistoarele bipolare**
   b) Au un factor de amplificare mai mare
   c) Pot sa aiba doar doua stari: 1 si 0 logic
   d) Toate de mai sus

2. Adevarat sau fals: Procesorul cu banda de asamblare executa toate instructiunile intr-un singur ciclu de ceas (aici as zice fals pt ca daca le-ar executa pe toate intr-un ciclu nu ar mai avea ce tipuri de instructiuni sa separe a.i. sa se execute in paralel) **FALS**

3. De ce este important sa stim daca o masina este big-endian sau little-endian?
   a) Pentru ca anumite programe scriu date in memorie intr-o anumita ordine (toate scriu
   b) dupa cum vrea procesoru; ma rog, daca esti tu smecher si faci un program c care sa iti scrie rezultatele in ce endianness vrei tu e altceva)
   c) Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers (asta ar putea fi, dar procesorul nu o sa adune el gresit, el aduna corect dar interpreteaza numerele invers)
   d) **Pentru ca partajarea folderelor si informatiei intre doua calculatoare poate cauza erori de interpretare - cred ca asta e**
   e) Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare

4. Numarul 0xC000FEFE(covfefe lel), stocat ca signed int este:
   a) Prea mare pentru a putea fi reprezentat (are 32 biti deci nu e prea mare)
   b) Poztiv (C e 1100 deci incepe cu 1 si e negativ)
   c) Impar (E este 1010 deci se termina cu 0 si e par)
   d) **Nicio optiune**

5. Presupuneti ca aveti de ales intre doua operatii echivalente= {op1}:mul r1, 4 si {op2}: shl r1,2. Din punctul de vedere al timpului de executie, pentru majoritatea procesoarelor:
   a) **t(op1) > t(op2)** **Explicatie: shiftarea e mereu mai rapida**
   b) t(op1) < t(op2)
   c) Ambele dureaza la fel
   d) Nu poate fi determinat

6 . Un procesor petrece 50% din timp cu operatii UAL si 40% in operatii cu memoria. Daca puteti imbunatati timpul de acces la memorie de 4 ori, dar, ca rezultat veti incetini executia celorlate instructiuni cu 50%, ce speedup obtineti?

a) 5.3
b) 1 Explicatie: S = 1 / (0.6 * 1.5 + 0.4 / 4) = 1 / 1 = 1
c) 2
d) 3.33
e) Nu poate fi determinat

7. Hazardele de date:

a) Sunt cauzate de UAL
b) Pot fi solutionate prin bypassing
c) Au latenta minima
d) Apar numai la RISC

8 . De ce se foloseste deseori pentru procesoarele RISC denumirea de “arhitecturi load/store”?

Arhitectura load/store are caracteristica de a imparti instructiunile in 2 categorii: operatii cu memoria si operatii cu UAL. Arhitectura procesoarelor RISC face posibile operatiile de orice fel doar intre valori care se afla in registrii(operanzii trb sa fie adusi intr-un registru pentru a putea fi folositi), spre deosebire de CISC(operatiile memorie-registru sunt posibile).

9. Dati cel putin trei caracteristici ale ISA pentru un procesor RISC.

Complet, de mici dimensiuni, open source.s

10. Este obligatorie ca o exceptie sa fie tratata de catre procesor atunci cand se produce? Este acelasi lucru valabil si pentru intreruperi? Justificati raspunsurile voastre.

Exceptie este o intrerupere sincrona: tratata dupa terminarea rutinei actuale, lanseaza rutina de tratare dupa ce a golit toata banda de asamblare(exceptia este o intrerupere sincrona)

Intreruperi asioncrone: este tratata in momentul in care e detectata, se salveaza current state-ul, se executa rutina de tratare, se revine in current state(intrerupere asincrona)

Examen CN1 11.09.2015

1 . Descrieti functionalitatea unui circuit logic programabil de tipul PLA sau PAL (schema si scurta explicatie a functionalitatii).

Un circuit combinaţional programabil poate să înlocuiască funcţionalitatea mai multor porţi logice. Programate prin întreruperea unor conexiuni existente (fuses) sau stabilirea de noi conexiuni(antifuses).

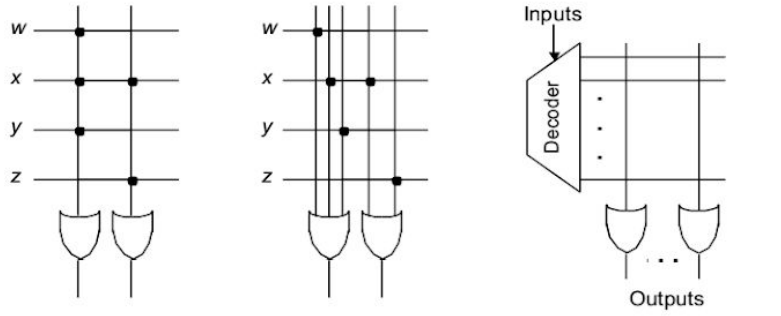
Logică combinațională programabilă

Un circuit combinațional programabil poate să înlocuiască funcționalitatea mai multor porți logice

Programate prin întreruperea unor conexiuni existente (*fuses*) sau stabilirea de noi conexiuni(*antifuses*)

- Programmable ROM (PROM)
- Programmable array logic (PAL)
- Programmable logic array (PLA)

Circuite PROM



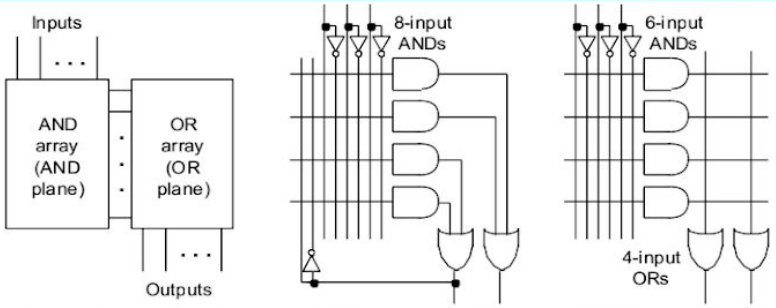
(a) Programmable OR gates

(b) Logic equivalent of part a

(c) Programmable read-only memory (PROM)

Conexiuni programabile și folosirea lor într-un circuit PROM.

Circuite PAL & PLA



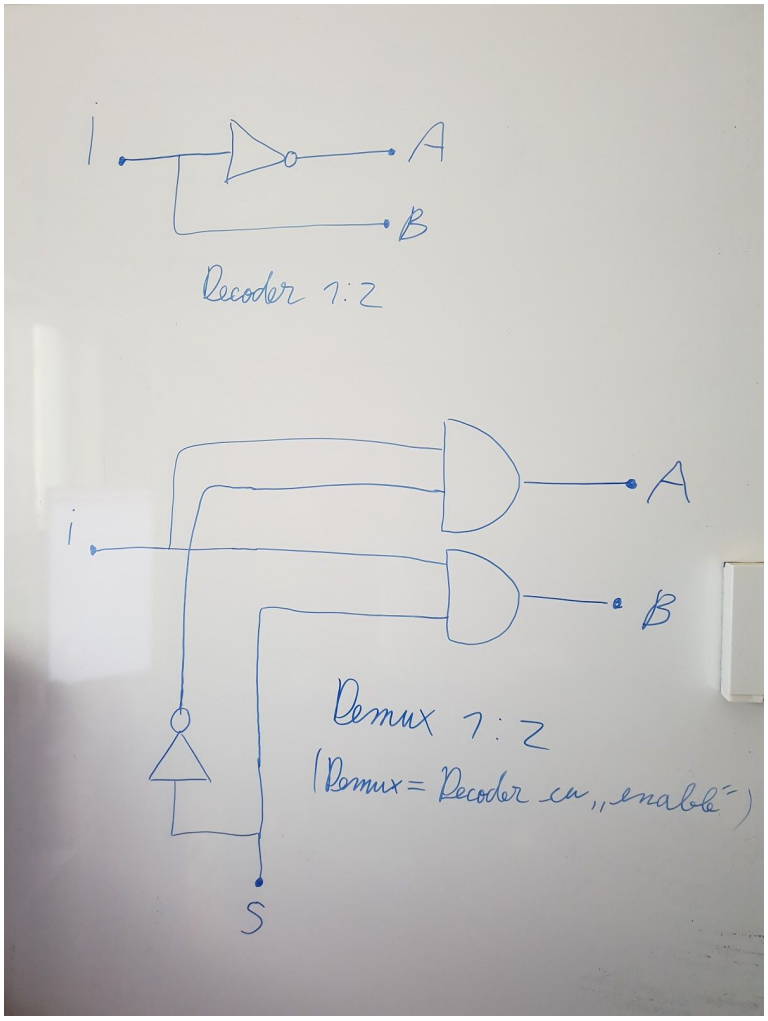
(a) General programmable combinational logic

(b) PAL: programmable AND array, fixed OR array

(c) PLA: programmable AND and OR arrays

Logică combinațională programabilă: structură generală și două clase de circuite (PAL și PLA). În același circuit este nevoie și de un PROM cu un plan AND fix (rol de decodificator) și un plan OR programabil.

2. Proiectati un circuit de tip decodifier 1:2. Aveti la dispozitie numai parti logice de tip NOT, AND si OR cu doua intrati.
https://www.allaboutcircuits.com/textbook/digital/chpt-9/decoder/



3. Un procesor petrece 30% din timp cu adunari flp, 60% cu inmultiri flp si 5% cu impartiri flp. Evaluati imbunatariea performantei pentru:

a) Sumatorul flp este de 3 ori mai rapid
   S = 1 / (0.3/3 + 0.6 + 0.05 + 0.05) = 1/0.8 = 1.25
b) Multiplicatorul flp este de 2 ori mai rapid
   S = 1 / (0.3 + 0.6/2 + 0.05 + 0.05) = 1/0.7 = 1.43
c) Unitatea de impartire flp este de 5 ori mai rapid
   S = 1 / (0.3 + 0.6 + 0.05/5 + 0.05) = 1/0.96 = 1.04

4. Este vizibila, din perspectiva software-ului, diferenta dintre o masina CISC si una RISC? Explicati.

5. Descrieti cei 5 pasi din executia unei instructiuni (fetch, decode, UAL, MEM, WB).

6. Care este principalul dezavantaj al sumatorului select?

7. Implementati un CLA pe 4 biti. Scrieti ecuatiile pentru P si G si desenati schema acestuia.

8. Care dintre termenii din cadrul Legii de fier se minimizeaza la CISC? Dar la RISC?
CISC: Numarul de instructiuni per program
RISC: Numarul de clocks per instruction

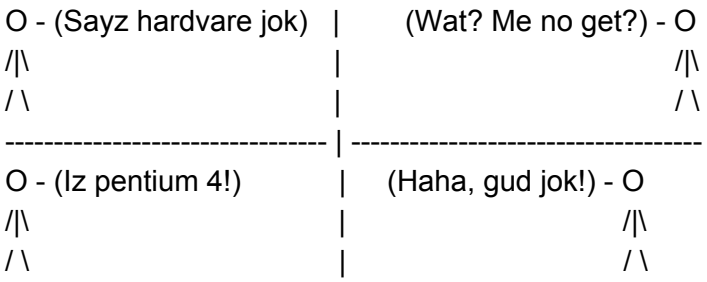
9. De ce sunt preferati tranzistorii MOSFET in locul celor bipolari? Dati 3 argumente.

1 - Consumul de putere este mai mic decat la tranzistoarele bipolare
2 - Un MOSFET e mai tolerant la caldura decat un tranzistor bipolar
3 - MOSFETs sunt mai usor de fabricat (??)
4 - MOSFETs sunt mai usor de scalat
5 - MOSFET - impedanta de intrare mare

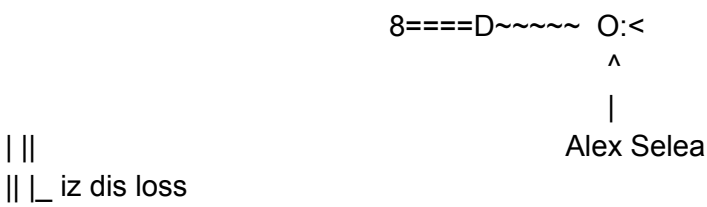
10. Cu tehnologia disponibila in zilele noastre, ce implementare ati alege pentru ISA-ul vostru: CISC sau RISC? Motivati.
Daca se doreste o implementare mai usoara, eficienta din punctul de vedere al memoriei folosite si fara limitari de hardware, putem folosi CISC.
Daca dorim o implementare mai rapida, fara constrangeri de memorie, putem folosi RISC.

+ Alt bullshit, ce iti mai vine prin cap

11. Desenati un comic pentru CN2.



\- Comic by Graet Artiste, 2018



Examen CN1 12.06.2017

1. Un procesor cu arhitectura Harvard:
a) Are memorii de date si instructiuni separate
b) Executa orice operatie intr-un singur ciclu de ceas - nu e bn ca e ciclu masina, nu ciclu ceas
c) Are o banda de asamblare cu cinci etape - idk
d) Toate de mai sus

2. Adevarat sau fals: O intrerupere ne-mascabila se executa intotdeauna de catre procesor.
ADEVARAT

3. Pentru a putea rula la 3GHz, procesorul unui calculator trebuie sa fie alimentat la o tensiune de 2V, iar curentul de mers in gol al acestuia este 100mA. Stiind ca procesorul are o capacitate parazita echivalenta de 10nF, care este puterea necesara pentru ca acesta sa funcioneza?
a) 60.1W b) 60W **c) 60.2W** d) 120.2W e) 120W f) 120.1W

P = 1⁄2 * C * (VDD^2) * f + IDD * VDD
P = ½ * 10 * 10^(-9) * (2^2) * 3 * 10^9 + 100 * 10^(-3) * 2
P = ½ * 10 * 4 * 3 + 0.2
P = 60.2W

4. Numarul 0xCAFEBABE, reprezentat in cod IEEE 754 single precision este:
a) Prea mare pentru a putea fi reprezentat (CAFEBABE are 32 de biti, deci poate fi reprezentat in format IEEE 754)
b) Negativ (C este 1100, deci bitul de semn o sa fie 1, deci numar negativ)
c) Are exponentul negativ (Exponentul este 1001 0101, care este 149 > 127, pozitiv)
d) Nicio optiune

CAFEBABE = 1100 1010 1111 1110 1011 1010 1011 1110
-> primul bit de semn
-> urmatorii 8 biti exponentul
-> restul de biti sunt mantissa

5. Presupuneti ca un procesor in b.a. Executa urmatoareae operatie: addi r1, r1, 0xbeef, in care etapa este actualizat r1?
a) Instruction Fetch
b) Decode
c) Execute
d) Memory
e) Write Back

6. Un procesor petrece 80% din timp cu operatii UAL. Daca puteti imbunatati performanta UAL-ului de 4 ori, dar, ca rezultat veti incetini executia celorlalte instructiuni cu 50%, ce soeedup obtineti?

a) 5.3
b) 3.5
c) 2 **S = 1 / (0.8 / 4 + 1.5 * 0.2) = 1 / (0.2 + 0.3) = 1 / 0.5 = 2**
d) 3.33

7. Hazardele structurale:

a) Sunt intalnite doar la risc
b) Produc bule in b.a.
c) Sunt rezolvate prin multiplicarea resurselor partajate

8. Care este avantajul primar al sumatorului CLA fata de sumatorul ripple-carry?

Sumatorul CLA foloseste o structura auxiliara care poate sa determine unde vor aparea biti de carry, nemaifiind nevoie astfel sa se astepte rezultatul fiecarui sumator de dinainte, lucru care va duce la eficienta mai buna atunci cand este vorba de un numar mare de sumatoare (N > 16).

9. Care sunt pasii pe care un procesor trebuie sa ii execute pentru tratarea unei intreruperi?

Întreruperi asincrone: invocarea rutinei de tratare a întreruperii



- Un dispozitiv I/O cere atenție prin semnalizarea pe o linie dedicată de interrupt request (IRQ)
- Când un procesor decide să proceseze întreruperea
  - Oprește execuția programului curent la instrucțiunea I_i, completând execuția tuturor instrucțiunilor până la I_{i-1} *(întrerupere precisă)*
  - Salvează PC-ul instrucțiunii I_i într-un registru special (EPC)
  - Maschează toate întreruperile și transferă controlul unei rutine speciale pentru tratarea întreruperii (pentru un OS ca Linux/Windows aceasta e situată de obicei în kernel)

Rutina de tratare a întreruperii



- Salvează EPC înainte să permită execuția altor întreruperi (întreruperi imbricate) ⇒
  - Avem nevoie de o instrucțiune pentru a muta EPC în GPR-uri
  - Avem nevoie de un mecanism pentru a masca toate întreruperile până când EPC este salvat
- Trebuie să citească un Registru de Stare care indică ce periferic sau subsistem a cauzat întreruperea
- Folosește o instrucțiune specială de salt indirect RFE (*return-from-exception*) care
  - Demaschează întreruperile
  - Întoarce procesorul înapoi în modul utilizator
  - Reface statusul inițial hardware și starea unității de control

Întreruperi sincrone



- O întrerupere sincronă (excepție) este cauzată de o anumită instrucțiune
- În general, instrucțiunea nu poate fi completată și trebuie să fie restartată după tratarea excepției pe care a generat-o
  - Necesită refacerea efectului unei sau mai multor instrucțiuni executate parțial
- În cazul unui system call trap, se consideră că instrucțiunea și-a incheiat execuția
  - O instrucțiune de jump specială ce face o trecere a procesorului în modul kernel

10. Dati exemplu de un hazard de date de tip load/store si un hazard de control.

- Hazard load/store
  -> RAW
  ```
  Mov r1, r2
  Add r3, r4, r1
  ```
  -> WAR
  ```
  Add r3, r1, r2
  Add r2, r4, r5
  ```

- Hazard de control
  ```
  Cmp eax, ebx ; eax = 100, ebx = 1
  Jg label1
  Jl label2
  ...
  Label1: ...
  Label2:...
  ```

1. Un coleg va propune sa adaugati instructiunea add3 la ISA-ul procesorului de la curs: Add3 r1, r2, r3, r4 -> R[r1] = R[r2] + R[r3] + R[r4]. Se schimba ISA in acest caz? Are aceasta schimbare vreun impact asupra arhitecturii procesorului?

2. Dati cel putin un avantaj si un dezavantaj daca, in loc de instructiuni de lungime fixa(de ex. 32 de biti), am folosi un ISA cu instructiuni de lungime variabila?

3. Dati trei caracteristici ale ISA pentru un procesor de tip RISC.
Lucreaza doar cu registrii
Numar fix de parametrii
Se executa instructiunile mai rapid ca la cisc???

4. Daca aplicam optimizarea B, imbunatatim viteza de executie cu 20% din programul A. Care este speedup-ul maxim teoretic oferit de B?

5. Completati
   Completez
   Eu nu vreau sa completez. Fight me >:(

6. Victor decide sa isi cumpere urmatoarea tableta bazandu-se pe metrica MIPS^3/(Watt^2 * Greutate)
Coaie victor efectiv cauta sa isi ia bataie nu tableta

a) Care sunt caracteristicile pe care Victor le cauta la tableta sa?
- Vrea sa fie puternica, i guess

b) Care sunt caracteristicile pe care Victor doreste sa le evite?

- Vrea sa nu fie grea si sa nu se duca repede bateria

c) Aranjati aceste caracteristici de la cea mai importanta la cea mai putin importanta.

- Pai cu cat e puterea mai mare cu atat e mai importanta. Deci aia cu la a treia e cea mai importanta, dupa aia la a doua, dupa aia simpla

7. Un procesor petrece 60% din timp executand operatii cu UAL-ul si 20% din timp in operatii cu memoria. Daca puteti optimiza de 2 ori viteza de executie a operatiilor cu UAL si de 4 ori viteza celor cu memorie, cvare o sa fie speedup-ul obtinut?

S = 1 / (0.6 / 2 + 0.2 / 4 + 0.2) = 1 / (0.3 + 0.05 + 0.2) = 1/ 0.55 = 1.8

8. Procedeul de bypassing(sau forwarding) elimina hazardele structurale sau cele de date?
Elimina hazardele de date

9. Conform Legii lui Moore, ce fel de rata de crestere in raport cu timpul o are numarul de tranzistoare dintr-un circuit integrat?
Crescatoare (se dubleaza)

10. Urmatorul cod se executa pe un procesor in b.a. cu 5 etape. Care sunt registrele care sunt citite la inceputul celui de-al 5-lea ciclu de ceas si care sunt cele scrise la sfarsitul aceluiasi ciclu de ceas? Luati in calcul numai registrele din tabela de registre generale(ex. R1, R2, R3..etc.)
ADD R1, R2, R3
ADD R4, R5, R6
ADD R7, R8, R9
ADD R10, R11, R12
ADD R13, R14, R15

Vreti sa invatati chitara si pentru asta va trebuie o chitara acordata. Pentru ca nu v-ati antrenat inca urechea incat sa faceti asta singuri, va propuneti sa faceti un tuner automat bazat pe FPGA care va da un feedback legat de cat de "aproape" e o coarda de o nota muzicala corecta.

Aveti la dispozitie: - un FPGA cu 6 LED-uri (de ce 6, explicatii imediat) - 12 valori intregi, care reprezinta frecventele echidistante ale notelor corecte (in ordinea frecventei: A, A#, B, C, C#, D, D#, E, F, F#, G, G#); distanta dintre doua note adiacente este si ea cunoscuta - un modul gata implementat (prin microfon etc) care da frecventa pe care o emite coarda; el arata astfel: module microphone (output frequency[15:0]); - un modul gata implementat module interval closest

(input values[11:0][15:01 input x_val[15:0], output closest_low[15:0], closest_high[15:0], closest[15: 0]); care va da 2 valori din "vectorul" valuescele mai apropiate de valoarea x_val (in scenariul adevarat, x val va fi mereu intre aceste doua valori) si valoarea cea mai apropiata (evident, va fi una dintre closest low si closest high)

Modulul pe care it yeti implementa va arata astfel: module tuner 10; si va contine ca si constante frecventele notelor corecte (va trebui doar sa le declarati ca atare, valorile for efective nu sunt importante in aceasta problema).
Veti afisa pe cele 6 LED-uri cat de aproape este frequency (data de modulul-microfon) de una din constantele pe care le aveti (cea mai apropiata). Unul dintre LED-uri se va aprinde sau nu, cu semnificatia "frecventa corzii este mai mare/mai mica decat frecventa corecta". Celelalte 5 LED-uri se vor aprinde in functie de diferenta de frecventa intre coarda si nota corecta. Frecventa corzii va fi tot timpul intre A si G#.
Exemplu: sa zicem ca vreti sa acordati o coarda pentru nota B, dar deocamdata coarda e mai aproape de A# decat de B. Atunci LED-utile (x = aprins) vor arata, sa zicem, xxxx00 (primul aprins, ca frecventa e mai mare decat A#, cea mai apropiata, iar celelalte 3 indicand ca diferenta e destul de mare). Mai strangeti coarda (LED-urile se vor aprinde pe treptat in configuratiile xxxxx0, xxxxxx, apoi Oxxxxx, Oxxxx0, Oxxx00) si verificati tuner-ul, iar acesta va afisa Oxx000, indicand ca nota e mai jos decat urmatoarea (B), cu o diferenta mai mica. Strangand coarda putin cate putin, cele 2 LED-uri aprinse se vor stinge treptat (0x0000, apoi 000000), indicand ca ati ajuns fix pe nota corecta.
Indicatii: - folositi-va de modulele pe care be aveti la dispozitie, instantiati-le corespunzator - va puteti crea un mic modul auxiliar care va da bitii corespunzatori celor 5 LED-uri care se aprind proportional; de exemplu, primul din ele va fi setat daca diferenta dintre coarda si cea mai apropiata nota e mai mare de 1/5 din distanta_note_adiacente/2 (supra 2 din cauza alegerii notei mai apropiate, diferenta maxima pana la o nota corecta va fi tot timpul mai mica decat distanta_note_adiacente/2) - alegeti-va de la inceput pe ce logica lucrati (secventiala/combinationala) si pastrati-o - comentati si explicati-va metoda

CISC	RISC
Emphasis on hardware	Emphasis on software
Includes multi-clock complex instructions	Single-clock, reduced instruction only
Memory-to-memory: "LOAD" and "STORE" incorporated in instructions	Register to register: "LOAD" and "STORE" are independent instructions
Small code sizes, high cycles per second	Low cycles per second, large code sizes
Transistors used for storing complex instructions	Spends more transistors on memory registers

Felicitari ca ai ajuns pana aici :). Esti pe drumul cel drum in a deveni Hokage. Share la 3 prieteni pwp

Calculatoare numerice I (Seria CA)

Început la	vineri, 19 iunie 2020, 08:09
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:48

Timp luat 38 min 46 secs
Notează **8,00** din maxim 10,00 (**80**%) posibil

1 întrebare

Corect
Marcat 1,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

Un procesor petrece 80% din timp cu operații UAL. De câte ori mai rapidă trebuie să fie unitatea aritmetică logică pentru ca procesorul să ruleze de patru ori mai repede?

Alegeți o opțiune:
- a. **16**
- b. **6**
- c. **8**
- d. **1.5**

Feedback

Răspunsul corect este: **16**

2 întrebare

Corect
Marcat 1,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

Numărul 2.25, poate fi reprezentat în cod IEEE 754 single precision ca:

Alegeți o opțiune:
- a. **0x40600000**
- b. **0x40200000**
- c. **Nicio opțiune**
- d. **0x40800000**
- e. **0x40100000**

Feedback

Răspunsul corect este: **0x40100000**

3 întrebare

Corect
Marcat 1,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

De ce este important sa stim daca o masina este big-endian sau little-endian?

Alegeți o opțiune:

- a. **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare**
- b. **Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers**
- c. **Pentru ca anumite programe scriu date in memorie intr-o anumita ordine**
- d. **Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare**

Feedback

Răspunsul corect este: **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare**

4 întrebare

Corect
Marcat 1,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	HLT	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **9**
- b. **Nu se poate determina**
- c. **B**
- d. **8**
- e. **A**

Feedback

Răspunsul corect este: **9**

5 întrebare

Incorect
Marcat 0,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

Adevarat sau fals: Un UAL este o unitate funcțională secvențială.

Selectați o opțiune:
- Adevărat
- Fals

Feedback

Răspunsul corect este "Fals".

6 întrebare

Corect
Marcat 1,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

Dacă reproiectăm arhitectura procesorului de la curs să lucreze pe 64 de biți, care este efectul asupra ISA?

Alegeți o opțiune:
- a. **Codificarea instrucțiunilor se va modifica**
- b. **Se modifică toate instrucțiunile ce fac acces la memorie**
- c. **Câmpul *opcode* se dublează în lungime**
- d. **Nici un efect**
- e. **Instrucțiunile se vor executa mai rapid**

Feedback

Răspunsul corect este: **Codificarea instrucțiunilor se va modifica**

7 întrebare

Corect
Marcat 1,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

Presupuneți că un procesor fără b.a. execută următoarea operație: *ldi r16, 0xdeadc0de*. În care etapă este actualizat *r16*?

Alegeți o opțiune:
- a. **Write Back**
- b. **Execute**
- c. **Memory**
- d. **Decode**
- e. **Instruction Fetch**

Feedback

Răspunsul corect este: **Write Back**

8 întrebare

Incorect
Marcat 0,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

Dacă îmbunătățim timpul de acces la memorie, atunci în ce sens putem opera asupra frecvenței procesorului?

Alegeți o opțiune:
- a. **Rămâne neschimbată**
- b. **Se apropie de 3GHz**
- c. **Nu se poate determina**
- d. **Scădere**
- e. **Creștere**

Feedback

Răspunsul corect este: **Creștere**

9 întrebare

Corect
Marcat 1,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

Hazardele de date:

Alegeți o opțiune:
- a. **Pot fi solutionate prin bypassing**
- b. **Au latenta minima**
- c. **Sunt cauzate de UAL**
- d. **Apar doar la arhitecturile RISC**

Feedback

Răspunsul corect este: **Pot fi solutionate prin bypassing**

10 întrebare

Corect
Marcat 1,00 din 1,00
Fără flagÎntrebare cu flag

Textul întrebării

Pentru a putea rula la 0.5GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 2V, iar curentul de mers în gol al acestuia este 10mA. Știind că procesorul are o capacitate parazită echivalentă de 300nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:
- a. **300W**
- b. **300.03W**
- c. **50W**
- d. **100.1W**
- e. **300.02W**

Feedback

Răspunsul corect este: **300.02W**

Finalizare verificare
◄ Inregistrare curs 11&12
Salt la... Salt la...
Examen - Partea II ►
Omite Navigare în test

Navigare în test

Întrebare 1 Această pagină Întrebare 2 Această pagină Întrebare 3 Această pagină Întrebare 4 Această pagină
Întrebare 5 Această pagină Întrebare 6 Această pagină Întrebare 7 Această pagină Întrebare 8 Această pagină
Întrebare 9 Această pagină Întrebare 10 Această pagină
Afișați câte o pagină pe rândFinalizare verificare

Prieteni
Mesaje selectate: 1 ×

Caută
Prieteni 0
Caută

Setări

- Prieteni
- Solicitări 0

No contacts
No contact requests

Contact request sent

Personal space

Save draft messages, links, notes etc. to access later.

Început la	vineri, 19 iunie 2020, 09:01
Status	Terminat
Completat la	vineri, 19 iunie 2020, 09:35
Timp luat	33 min 58 secs
Notează	**8,00** din maxim 10,00 (**80**%) posibil

1 întrebare

Parțial corect

Marcat 1,00 din 3,00

Fie următorul cod în assembly (x86). Indentificați hazardele de date. Pentru fiecare menționați intrucțiunile implicate, arătând cauzele fiecărui hazard.

Treceți hazardele în ordinea crescătoare a instrucțiunilor.

Exemplu: Dacă există:

- un hazard de tipul RAW între liniile x și y (x < y) din cauza registrului eax
- un hazard de tipul RAW între liniile x și z (x < z, y < z) din cauza registrului eax
- un hazard de tipul RAW între liniile t și z (x < t, t < z) din cauza registrului eax

atunci veți scrie:

Ix - Iy: RAW (eax)

Ix - Iz: RAW (eax)

It - Iz: RAW (eax)

```
0: sub edx, eax       ; edx = edx - eax
1: mov eax, 0xABADBABE    ; eax = 0xABADBABE
2: sub ebx, edx       ; ebx = ebx - edx
3: mov eax, ebx       ; eax = ebx
4: mov ecx, [0xABADBABE]    ; ecx = MEM[0xABADBABE]
```

Procesor **single-core**:

I 0 ✔ - I 2 ✔ : RAW ✔ (edx ✔)

I 2 ✔ - I 3 ✔ : RAW ✔ (ebx ✔)

I 3 ✘ - I 4 ✘ : RAW ✘ (eax ✘)

In plus, la **multi-core**:

I 0 ✔ - I 2 ✘ : WAR ✔ (edx ✘)

I 1 ✘ - I 3 ✔ : WAR ✔ (edx ✘)

I 2 ✘ - I 3 ✔ : WAR ✘ (ebx ✘)

Vom puncta hazardele identificate corect chiar daca nu au fost puse in ordinea crescatoare a numarului instructiunii.

Comentariu:

2 întrebare

Corect

Marcat 7,00 din 7,00

Avem un procesor cu frecvența de **2GHz**, care execută acest cod.

Știm numărul de cicli de ceas pentru fiecare tip de instrucțiune:

- operațiile cu date și memorie (**mov**): **2 cicli**;
- operațiile aritmetice/logice (**cmp, add, inc, dec, xor, and, test, shl**): **1 ciclu**;
- operațiile de salt necondiționat (**jmp**): **5 cicli**;
- operațiile de salt condiționat (**jg, jl, je/jz, jbe**): **20 cicli** (dacă nu a fost prezis corect) sau **5 cicli** (dacă a fost prezis corect);
- afisarea: **1 ciclu**.

Fie procesorul B care implementează un branch predictor **global**, iar metoda de branch prediction folosită este **2-bit counter** (strongly taken/weak taken/weak not taken/strongly not taken). Starea inițială este **strongly not taken**.

Se consideră inputul **c = 5**.

1. Ce valoare afișează programul?

0 ✖

2. Enumerați prin ce stări trece branch predictorul, semnalând dacă branch-ul a fost prezis corect sau greșit.

	Iteratia 1	Iteratia 2	Iteratia 3	Iteratia 4	Iteratia 5
`xor eax, eax`	eax: 0 ✔ 1 ✔				
`xor ebx, ebx`	ebx: 0 ✔ 1 ✔				
`inc ebx`	ebx: 1 ✔ 1 ✔				
`mov ecx, $c`	ecx: 5 ✔ 2 ✔				
`count_loop:`					
`test ecx, ecx`	1 ✔	1 ✔	1 ✔	1 ✔	1 ✔
`je done`	5 ✔ Corect ✔ SNT ✔	5 ✔ Corect ✔ SNT ✔	5 ✔ Corect ✔ SNT ✔	5 ✔ Corect ✔ SNT ✔	5 ✔ Corect ✔ SNT ✔
`test ecx, ebx`	1 ✔	1 ✔	1 ✔	1 ✔	1 ✔

	Iteratia 1	Iteratia 2	Iteratia 3	Iteratia 4	Iteratia 5
	20 ✗	20 ✓	20 ✗	20 ✓	20 ✗
jz no_add	Gresit ✗	Gresit ✓	Gresit ✗	Gresit ✓	Gresit ✗
	WNT ✗	WNT ✓	WNT ✗	WNT ✓	WNT ✗
add eax, ecx	eax: 0 ✗	eax: 0 ✓	eax: 0 ✗	eax: 0 ✓	eax: 0 ✗
	Nu se executa ✗	Nu se executa ✓	Nu se executa ✗	Nu se executa ✓	Nu se executa ✗

no_add:

dec ecx	ecx: 4 ✓	ecx: 3 ✓	ecx: 2 ✓	ecx: 1 ✓	ecx: 0 ✓
	1 ✓	1 ✓	1 ✓	1 ✓	1 ✓
jmp count_loop	5 ✓	5 ✓	5 ✓	5 ✓	5 ✓

done:

;se afiseaza conținutul lui eax (trebuie trecuti cicli)	Nu se executa ✓	Nu se executa ✓	Nu se executa ✓	Nu se executa ✓	Nu se executa ✓

3. Câte branch-uri au fost prezise corect?

5 ✗

4. Câte branch-uri au fost prezise incorect?

6 ✗

5. Care este timpul de rulare al programului (in ns)?

96 ✗

Din cauza multiplelor probleme cu acest subiect (Branch Predictor), va fi acordat punctajul intreg.

Comentariu:

◄ Examen - Partea I

Salt la...

Model Colocviu ►

Rezolvare CN

Problemă: branch prediction → se implementează JUMP
→ nu se întâmplă JUMP

```
mov ax, $a   | 2
mov bx, $b   | 2
mov cx, $c   | 2
loop: cmp cx, 0   | 1
      je out      | 10
      add ax, bx  | 1
      mov dx, ax  | 2
      mov ax, bx  | 2
      mov bx, dx  | 2
      add bx, 1   | 1
      dec cx      | 1
      jmp loop    | 10
out: se afișează ax, bx
```

- f = 1 GHz, 1 bit pred
- are doar două stări
- abordare pesimistă - STĂRI
- counter pt. fiecare branch
- mov - 2 cicli
- cmp, add, dec - 1 ciclu
- jmp - 10 cicli
- je - 40 cicli (nu a fost prezis ok)
  10 cicli

a = 1
b = 20
c = 5

Cerințe: a) Ce val. se afișează?
b) Stările pt. branch.
c) Timpul de rulare.
d) Îmbunătățirile ideală de rulare.

	cicli				
mov					
mov					
mov					↑
cmp	1	1	1	1	40
je	10	10	10	10	
add	1	1	1	1	
mov	2	2	2	2	
mov	2	2	2	2	
mov	2	2	2	2	
mov	2	2	2	1	
add	1	1	1	1	
dec	1	1	1	10	
jmp	10	10	10	—	—
	35	30	30	30	80

ax = 110
bx = 177
- nr de je corecte și dorite

c) Timp de rulare = 120 + 35 + 41 =
= 120 + 77
= 197 cicli

$T = \frac{1}{f} = \frac{1}{10^9} = 1ns$

$1ns \cdot 197 = 197 ns$

d) - nu putem să avem mai puține de 1 ciclu
je - 90 ns → 45 ns (imb.)
$T_2 = 197 - 45 = 152 ns$

$speed\text{-}up = \frac{T_1}{T_2} = \frac{197}{152} = 1,3$

Scanned by CamScanner

instr.	ax	bx	MT	T	Nr. cicli	cx
			•			
je	1	20	•		10	5
	21	20				5
	20	21				4
je	20	22	•		10	4
	42	22				
je	22	43	•		10	3
je	43	66	•		10	2
	109	66				
	66	109				
		110				
je	66	110	•		10	1
	176					
	110	176				
	110	177				
je	110	177		•	40	0

Scanned by CamScanner

(2.)

```
mov ax, $a      1
mov bx, $b      1
```

	1	1	1	1	1	1	1	1
while: cmp ax, bx	1	1	1	1	1	1	1	1
jg greater	2	20	2	2	2	20	20	20
jl less	20	2	2	20				
je final	20							
jmp final								
greater: sub ax, bx	1	1	1	1				
jmp while	2	2	2	2				
less: sub bx, ax	1	1	1					
jmp while	2	2	2					

final :; se afișează ax, bx

INPUT:

ax = 36

bx = 32

mov, cmp, sub = 1 cicli

j → 2 cicli - afară

→ 20 cicli - găsit

jmp = 2 cicli

f = 1 GHz

Predictor global

Scanned by CamScanner

instr	ax	bx	SMT	VINT	WT	ST	Nr. cicli
Ex:			•	•	•	•	
jg	26	32	•				2
jl	26	32		•			20
jg	26	6			•		20
jg	20	6				•	2
jg	14	6				•	2
jg	8	6				•	2
jg	2	6			•		20
jl	2	6				•	2
jg	2	4			•		20
jl	2	4				•	2
jg	2	2			•		20
jl	2	2		•			20
je	2	2			•		20

Scanned by CamScanner

Predictor local

```
mov r60 1
mov r61 1
while:  cmp 1 1 1 1 1 1 1 1
        jg  2 20 20 2 2 20 20 2
        jl  20 20 2 20
        je  20
        jmp
greater: sub 1 1 1 1
         jmp 2 2 2 2
less:   sub 1 1 1
        jmp 2 2 2
```

Scanned by CamScanner

instr	ax	bx	jg	jl	je	Mr. aidi
jg	26	32	SMT	SMT	SMT	
jg	26	32	SMT	-	-	2
jl	26	32	SMT	WMT	SMT	20
jg	26	6	WMT	WMT	SMT	20
jg	20	6	WT	WMT	SMT	20
jg	14	6	ST	WMT	SMT	2
jg	8	6	ST	WMT	SMT	2
jg	2	6	WT	WMT	SMT	20
jl	2	6	WT	WT	SMT	20
jg	2	4	WMT	WT	SMT	20
jl	2	4	WMT	ST	SMT	2
jg	2	2	SMT	ST	SMT	2
jl	2	2	SMT	WT	SMT	20
je	2	2	SMT	WT	WMT	20

Scanned by CamScanner

SUBIECTE - EXAMEN (IUNIE 2008)

I Limbajul Verilog.

1. Conventii Lexicale.

2. Structura unui Program.

3. Tipuri de Date

    3.1 Tipuri de Date Fizice
    3.2 Tipuri de Date Abstracte

4. Operatori
    4.1 Operatori Arithmetici. Binari.
    4.2 Operatori Arithmetici. Unari.
    4.3 Operatori Relationali.
    4.4 Operatori Logici .
    4.5. Operatori la Nivel de Bit.
    4.6. Operatori Unari de Reducere.
    4.7 Alti Operatori.
    4.8 Operator de Precedenta.

5 Constructii de Control.
    5.1 Selectia: Instructiunile - if si case.
    5.2 Repetarea: Instructiunile - for, while si repeat.

6 Alte instructiuni.
    6.1 Instructiunea - parameter .
    6.2 Atribuirea Continua .
    6.3 Atribuiri Procedurale Blocante si Nonblocante.

7. Task-uri si Functii .

8 Controlul Sincronizarii (Timing).
    8.1 Controlul Intarzierii (#) .
    8.2 Evenimente.
    8.3 Instructiunea wait.
    8.4 Instructiunile fork and join.

9. Exemple de descrieri Verilog a unor module hardware.
    9.1 Registru.
    9.2 Bistabil.
    9.3 Atribuire blocanta.
    9.4 Atribuire nonblocanta.
    9.5 Sumator complet

9.6 Exemple de la laborator

II Elemente introductive privind informatia.

1. Introducere – masurarea informatiei.

2. Probleme privind masurarea informatiei.

3. Codificarea informatiei.
   3.1.Codificarea cu lungime fixa.
   3.2 Codificare cu lungime variabila.

4.Detectarea si corectarea erorilor.
   4.1. Detectarea erorilor.
   4.2. Corectarea erorilor.

5. Probleme I.

III Intarzierea in circuitele combinationale, sincronizarea elementelor de memorare a informatiei, calculul perioadei ceasului.

1.Intarzierile in circuitele combinationale
   1.1.Tehnologia CMOS - tehnologia de baza în calculul întârzierilor.
   1.2.Modelul general al intarzierii in circuitele combinationale.
   .1.3.Conventii pentru sincronizarea sistemelor numerice.

2. Elemente de memorare cu intrare de ceas.
   2.1.Latch/"zavor" transparent:
   2.2.Registru/Bistabil de tip D, comandat pe front:
   2.3.Realizarea unui Latch.
   2.4.Variante de realizare a Latch-urilor CMOS statice.
   2.5.Parametrii sincronizarii/timing-ului latch-ului:
   2.6. Parametrii sincronizarii bistabilelor:
   2.7.Calculul perioadei ceasului in conditiile comenzii pe front:
   2.8.Cauzele alunecarii ceasului. Modalitati de combatere a acestui fenomen.
   2.9.Comparatie intre skew si jitter:
   2.10.Retea de distributie a ceasului.

3. Probleme II

4.Componentele Unitatii de Executie:
   4.1.Elementele combinationale: blocurile constructive de baza.
   4.2.Elementele de memorare: blocurile constructive de baza: registrul, registrele generale, memoria ideala
   4.3. Metodologia de sincronizare (clocking)

IV Arii/Retele/Tablouri de Porti Programabile. (Field Programmable Gate Arrays - FPGAs)

1. Ce reprezinta FPGA?

2. De ce se utilizeaza FPGA?

3. Fluxul proiectarii cu FPGA. Descrierea etapelor.

4. Care sunt componentele structural ale FPGA-urilor.

5. Cum se pot realiza blocurile logice?

6. Blocul logic simplificat utilizat în familia FPGA Xilinx Seria XC 4000.

7. Conexiunea "cross-point".

8. În ce fel de structuri se pot plasa FPGA-urile? Descrieti succint fiecare structură.

V. Modalitati de reprezentare a calculatoarelor.

1. Reprezentarea functional/comportamentala.

2. Reprezentarea sub forma unei ierarhii de niveluri imbricate.

3. Arhitectura a unui calculator

4. Arhitectura Microprocesorului Intel 8080

5. Reprezentarea structurala a unui calculator

VI. Bazele Aritmetice ale Calculatoarelor Numerice.

1 Informatia in calculatoarele numerice
- 1.1. Tipuri de date utilizate la nivelul hardware-lui calculatoarelor numerice.
- 1.2. Conversia numerelor dintr-o baza in alta.
- 1.3. Reprezentarea informatiei numerice in calculatoare.
- 1.4. Coduri de reprezentare a numerelor intregi, cu semn, in calculatoare.
  - Codul direct,
  - Codul invers.
  - Codul complementar.
  - Codul binar-zecimal.
  - Codul in exces.

1.5. Terminologia folosita in legatura cu erorile de calcul.
1.6. Reprezentarea numerelor in Virgula Mobila Standardul IEEE 754, Formatul Scurt de Baza.
1.7. Reprezentarea numerelor in Virgula Mobila Standardul IEEE 754, Formatul Lung de Baza.
1.8. Erori in reprezentarea numerelor in virgula mobila.
- numarul numerelor reprezentabile;
- numarul care are valoarea/marimea cea mai mare;
- numarul diferit de zero, care are valoare/marimea cea mai mica;
- dimensiunea celei mai mari distante intre doua numere succesive;
- dimensiunea celei mai mici distante intre doua numere succesive;
- erorile relative.

1.9. Coduri alfanumerice:ASCII, EBCDIC, UNICODE.
1.10. Coduri detectoare de erori.
1.11. Coduri corectoare de erori.

2. Operatiile Aritmetice in Calculatoarele numerice.
2.1. Procesorul Aritmetic.
2.2. Operatii Aritmetice in virgula fixa: Adunarea si Scaderea.
2.3. Implementarea sumatoarelor:
- Sumator cu transport succesiv.
- Sumator cu intarziere minima.
- Sumator cu transport anticipat.
- Sumator cu salvare a transportului.

2.4. Operatii Aritmetice in virgula fixa: Inmultirea.
- Inmultirea in cod direct: solutia paralela, solutia serial-paralela.
- Inmultirea in cod complementar: solutii directe.
- Inmultirea in cod direct: metoda lui Booth. Descriere in Verilog.

2.5. Operatii Aritmetice in virgula mobila: Schema bloc a Unitatii Aritmetice in Virgula Mobila.

VII. Microprocesoare integrate structurate pe "transe de biti"

1. Unitati de executie integrate structurate pe transe de biti.
1.1 Unitatea de executie AMD 2901. Schema bloc.
1.2 Multiplexarea datelor de intrare pentru circuitul AMD 2901.
1.3 Conectarea in cascada a circuitelor AMD 2901.
1.4 Formatul microinstructiunii pentru comanda circuitului AMD 2901.

2. Unitati de comanda integrate structurate pe transe de biti.
2.1 Secventiatorul de microprogram AMD 2909. Schema bloc.
2.2 Schema bloc a unei unitati de comanda microprogramata, care contine circuitul AMD 2909.
2.3 Concatenarea circuitelor AMD 2909.
2.4 Formatul microinstructiunii pentru comanda unitatii AMD 2909.

Problemă examen CN

31.08.2016

Fie un procesorul ce rulează la frecvența de 1Ghz. Metoda de branch-prediction folosită este 2-bit counter (strongly taken/weakly taken/weakly not taken/strongly not taken). Predictor-ul pornește cu o abordare pesimistă, având starea inițială strongly not taken. Predictor-ul este unul local, astfel încât există un counter/state-machine pentru fiecare branch.

Fie următorul cod în assembly

```
      mov ax, $a
      mov bx, $b
      mov cx, $c
      mov dx, $d

loop: cmp cx, 0
      je out
      add ax, bx
      inc bx
      add dx, bx
      add dx, ax
      mov ax, bx
      mov dx, ax
      dec bx
      dec cx
      jmp loop

out:  ; Se afiseaza conținutul din registrele ax, bx, cx si dx
```

Se știu numărul de cicli de ceas pentru fiecare tip de instrucțiune

- Operațiile cu date și memorie (**mov**) durează 2 cicli
- Operațiile aritmetice/logice (**cmp**, **add**, **inc**, **dec**) durează 1 ciclu
- Operațiile de salt necondiționat (**jmp**) durează 10 cicli
- Operațiile de salt condiționat (**je**) durează 40 cicli (dacă nu a fost prezis corect) sau 10 cicli (dacă a fost prezis corect)

Se știu **$a = 1**, **$b = 20**, **$c = 3**, **$d = 0**. Se cer următoarele

1. **(1p)** Ce valori se afișează?
2. **(4p)** Evidențiați prin ce stări trec branch-predictor-ele indicând și care branch-uri au fost prezise corect, respectiv incorect.
3. **(2p)** Calculați timpul de rulare al programului.
4. **(3p)** Dacă ați putea îmbunătăți viteza de rulare a unei singure categorii de instrucțiuni dintre cele enumerate mai sus astfel încât timpul de execuție al acelei categorii să scadă la jumătate (nu poate scădea sub 1 ciclu), ce categorie ați alege pentru a obține cel mai bun speed-up în cazul programului vostru? Cât este acest speed-up?

Rezolvare problemă CN

31.08.2016

TODO

Problemă examen CN

01.09.2016

Tocmai v-ați angajat la o companie multinațională (pe un salariu considerabil) și prima voastră sarcină este să asigurați protecția celor mai noi tehnologii dezvoltate de companie, care sunt tinute într-un depozit secre. Pentru aceasta va trebui să contruiți un sistem care nu deschide usa depozitului decât dacă este introdus un cod corect. În pauza de prânz ați făcut deja majoritatea muncii, tot ce mai rămâne de făcut este logica de control.

Dispozitivul care controlează ușa este bazat pe FPGA și are 10 intrări de la 10 butoane (unul pentru fiecare cifră: **button0**, **button1**, ..., **button9**), o intrare de la un buton de **reset** și o ieșire **open_door** (care deschide ușa). Scrieți un modul Verilog care să controleze acest dispozitiv și să aibă următoarea funcționalitate:

- Modulul așteaptă combinația corectă pentru a deschide usa (o succesiune de apăsări, nu mai multe în același timp)
- Starea inițială a modulului este: nu a fost apăsat niciun buton până acum și ușa este închisă
- Butonul **reset** aduce modulul în starea inițială oricând este apăsat
- Dacă un buton (**button0**, **button1**, …, **button9**, **reset**) este apăsat atunci intrarea corespunzătoare lui este 1, altfel este 0
- Dacă ieșirea (**open_door**) este 0 atunci ușa este închisă, altfel este deschisă
- Dacă a fost introdus codul corect atunci ușa va ramane deschisă până la resetarea modulului
- Dacă nu a fost introdus codul corect, după apăsarea oricărei taste greșite ușa va rămâne închisă până la resetarea modulului
- Codul pentru deschiderea ușii este “123456” (adică trebuie apăsate, in ordinea asta, **button1**, **button2**, …, **button6**)

Exemplu de funcționare: se introduce “01547”, ușa nu se deschide; se introduce “12345”, ușa nu se deschide; se introduce “0123456”, ușa nu se deschide; se introduce “123456”, ușa se deschide; se introduce “123456789”, ușa se deschide.

Se cer

1. **(3p)** Graful stărilor, dacă automatul este de tip Mealy.
2. **(3p)** Graful stărilor, dacă automatul este de tip Moore.
3. **(4p)** Modulul Verilog, dacă automatul este de tip Moore.

Rezolvare problemă CN

01.09.2016

TODO

9 iunie

ex 3

1 si 2

			cicli						**stare3**	
ax	**bx**	**instructiune**	**cpu1**	**cpu2**	**stare1**	**stare2**	**cpu3**	**jg _greater**	**jl _less**	**je _final**
26		mov ax 26	1	1	NT	SNT	1	SNT	SNT	SNT
	32	mov bx 32	1	1			1			
		cmp ax bx	1	1			1			
		jg _greater	2	2			2			
		jl _less	20	20	T	WNT	20		WNT	
	8	sub bx,ax	1	1			1			
		jmp	2	2			2			
		cmp ax bx	1	1			1			
26	8	jg _greater	2	20		WT	20	WNT		
18		sub ax,bx	1	1			1			
		jmp	2	2			2			
18	8	cmp ax bx	1	1			1			
		jg _greater	2	2		ST	20	WT		
10		sub ax,bx	1	1			1			
		jmp	2	2			2			
10	8	cmp ax bx	1	1			1			
		jg _greater	2	2			2	ST		
2		sub ax,bx	1	1			1			
		jmp	2	2			2			
2	8	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	20	WT		
		jl _less	20	2	T	ST	20		WT	
	6	sub bx,ax	1	1			1			
		jmp	2	2			2			
2	6	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	20	WNT		
		jl _less	20	2	T	ST	2		ST	
	4	sub bx,ax	1	1			1			
		jmp	2	2			2			
2	4	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	2	SNT		
		jl _less	20	2	T	ST	2		ST	
	2	sub bx,ax	1	1			1			
		jmp	2	2			2			
2	2	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	2	SNT		
		jl _less	2	20		WNT	20		WT	
		je _final	20	20	T	WT	20			WNT
2	2	print ax	1	1			1			
FINAL			#NAME? ns	#NAME? ns	T	WT	#NAME? ns	SNT	WT	WNT

3) speedup este 0

Examen CN1 - Partea II - 31.08.2017

Nume: …….. Grupa: ……..

Fie următorul cod în assembly

```
      mov ax, $a
      mov bx, $b
      mov cx, $c

loop: cmp cx, 0
      je out
      add ax, bx
      mov dx, ax
      mov ax, bx
      mov bx, dx
      add bx, 1
      dec cx
      jmp loop

out:  ; Se afiseaza conținutul din registrele ax si bx
```

Procesorul ce execută acest cod rulează la frecvența de **1Ghz**. Metoda de *branch-prediction* folosită de fiecare este 1-bit counter (taken/not taken). Predictorul pornește cu o abordare pesimistă, counter-ul/state-machine-ul având starea inițială *not taken*. Predictorul este unul local, astfel încât există un counter/state-machine pentru fiecare branch.

Se știu numărul de cicli de ceas pentru fiecare tip de instrucțiune

- Operațiile cu date și memorie (**mov**) durează 2 cicli.
- Operațiile aritmetice/logice (**cmp**, **add**, **dec**) durează 1 ciclu.
- Operațiile de salt necondiționat (**jmp**) durează 10 cicli.
- Operațiile de salt condiționat (**je**) durează 40 cicli (dacă nu a fost prezis corect) sau 10 cicli (dacă a fost prezis corect).

Se știu **$a = 1**, **$b = 20**, **$c = 5**. Se cer următoarele

1. **(1p)** Ce valori se afișează.
2. **(4p)** Să se evidențieze prin ce stări trec branch-predictor-ele semnalându-se și care branch-uri au fost prezise corect, respectiv incorect.
3. **(2p)** Să se calculeze timpul de rulare al programului.
4. **(3p)** Dacă ați putea îmbunătăți viteza de rulare a unei singure categorii de instrucțiuni dintre cele enumerate mai sus astfel încât timpul de execuție al acelei categorii să scadă la jumătate (nu poate scădea sub 1 ciclu), ce categorie ați alege pentru a obține cel mai bun speed-up în cazul programului vostru? Cât este acest speed-up?

Rezolvare

1. Valori
    a. ax = 110
    b. bx = 177
2. Predictor je
    a. NT (corect)
    b. NT (corect)
    c. NT (corect)
    d. NT (corect)
    e. NT (corect)
    f. NT (greșit)
3. Timp de rulare: 197 cicli, 197 ns
4. Speed-up
    a. Categoria 4 (salt condiționat)
    b. Nou timp de rulare 152 cicli
    c. Speed-up 197 / 152 = 1.3

Examen CN1 - Partea II - 01.09.2017

Nume: …….. Grupa: ……..

Tocmai v-ați angajat la o companie multinațională (pe un salariu considerabil) și prima voastră sarcină este să asigurați protecția celor mai noi tehnologii dezvoltate de companie, care sunt tinute într-un depozit secret. Pentru aceasta va trebui să contruiți un sistem care nu deschide usa depozitului decât dacă este introdus un cod corect. În pauza de prânz ați făcut deja majoritatea muncii, tot ce mai rămâne de făcut este logica de control.

Dispozitivul care controlează ușa este bazat pe FPGA și are 10 intrări de la 10 butoane (unul pentru fiecare cifră: **button0**, **button1**, ..., **button9**), o intrare de la un buton de **reset** și o ieșire **open_door** (care deschide ușa). Scrieți un modul Verilog care să controleze acest dispozitiv și să aibă următoarea funcționalitate:

- Modulul așteaptă combinația corectă pentru a deschide ușa (o succesiune de apăsări, nu mai multe în același timp)
- Starea inițială a modulului este: nu a fost apăsat niciun buton până acum și ușa este închisă
- Butonul **reset** aduce modulul în starea inițială oricând este apăsat
- Dacă un buton (**button0**, **button1**, …, **button9**, **reset**) este apăsat atunci intrarea corespunzătoare lui este 1, altfel este 0
- Dacă ieșirea (**open_door**) este 0 atunci ușa este închisă, altfel este deschisă
- Dacă a fost introdus codul corect atunci ușa va rămâne deschisă până la resetarea modulului
- Dacă nu a fost introdus codul corect, după apăsarea oricărei taste greșite ușa va rămâne închisă până la resetarea modulului
- Codul pentru deschiderea ușii este “123456” (adică trebuie apăsate, în ordinea asta, **button1**, **button2**, …, **button6**)

Exemplu de funcționare: se introduce “01547”, ușa nu se deschide; se introduce “12345”, ușa nu se deschide; se introduce “0123456”, ușa nu se deschide; se introduce “123456”, ușa se deschide; se introduce “123456789”, ușa se deschide.

Se cer

1. **(3p)** Graful stărilor, dacă automatul este de tip Mealy.
2. **(3p)** Graful stărilor, dacă automatul este de tip Moore.
3. **(4p)** Modulul Verilog, dacă automatul este de tip Moore.

Rezolvare

TODO: Rezolvare

Subiecte CN

Sesiunea iunie 2010

La **323 CA** a dat la teorie:
1. Modelul general al intarzierilor in circuitele combinationale
2. Eficienta codificariii
3. Alg lui Booth
4. Diagrama bloc cu str procesorului DLX
5. Organigrama cu intreruperile la DLX

La **321CA si 322CA** subiectele au fost:
1. Codificarea informatiei: cod de lungime fixa, respectiv variabila.
2. Calculul perioadei ceasului pe frontul crescator si de explicat termenii ce apareau
3. Reprezentare in virgula mobila - format de baza (standard IEEE 754)
4. Diagrama unit. aritm. pentru virgula mobila
5. Formatul instructiunilor procesorului DLX si moduri de adresare

Problema semana f mult cu cea de pe site. (e pe acolo pe la materiale pt curs si se numeste Problema.pdf)

Început la	vineri, 19 iunie 2020, 08:05
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:35
Timp luat	29 min 49 secs
Notează	**8,00** din maxim 10,00 (**80**%) posibil

1 întrebare

Corect

Marcat 1,00 din 1,00

Adevarat sau fals: O intrerupere ne-mascabila se executa intotdeauna de catre procesor.

Selectați o opțiune:

- Adevărat ✔
- Fals

Răspunsul corect este "Adevărat".

2 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor petrece 50% din timp cu operatii UAL si 40% in operatii cu memoria. Daca puteti imbunatati timpul de acces la memorie de 4 ori, dar, ca rezultat veti incetini executia celorlalte instructiuni cu 50%, ce speedup obtineti?

Alegeți o opțiune:

- a. **3.33**
- b. **nu poate fi determinat**
- c. **1** ✔
- d. **5.3**
- e. **2**

Răspunsul corect este: **1**

3 întrebare

Incorect

Marcat 0,00 din 1,00

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	0x01	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **8**
- b. **B**
- c. **9** ✖
- d. **A**
- e. **Nu se poate determina**

Răspunsul corect este: **Nu se poate determina**

4 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că un procesor fără b.a. execută următoarea operație: *addi r14, 0xda*. În care etapă este actualizat *r14*?

Alegeți o opțiune:

- a. **Decode**
- b. **Execute**
- c. **Write Back** ✔
- d. **Instruction Fetch**
- e. **Memory**

Răspunsul corect este: **Write Back**

5 întrebare

Incorect

Marcat 0,00 din 1,00

Întreruperile mascabile:

Alegeți o opțiune:

- a. **Produc bule de exact o instrucțiune lungime în b.a.** ✖
- b. **Sunt întotdeauna sincrone cu ceasul sistemului**
- c. **Poartă și numele de excepții**
- d. **Pot fi implementate doar de procesoarele RISC**
- e. **Pot să nu fie tratate de procesor**

Răspunsul corect este: **Pot să nu fie tratate de procesor**

6 întrebare

Corect

Marcat 1,00 din 1,00

Numărul -1.75, poate fi reprezentat în cod IEEE 754 single precision ca:

Alegeți o opțiune:

- a. **0xcfe00000**
- b. **Nicio opțiune**
- c. **0xdfe00000**
- d. **0xafe00000**
- e. **0xbfe00000** ✔

Răspunsul corect este: **0xbfe00000**

7 întrebare

Corect

Marcat 1,00 din 1,00

Dacă trebuie să proiectați un procesor performant (frecvență mare de ceas, superscalar) folosind tehnologia actuală, ce fel de ISA ați alege pentru acesta?

Alegeți o opțiune:

- a. **CISC**
- b. **Mașină virtuală Java**
- c. **RISC** ✔
- d. **MISC**
- e. **VLIW**

Răspunsul corect este: **RISC**

8 întrebare

Corect

Marcat 1,00 din 1,00

De ce este important sa stim daca o masina este big-endian sau little-endian?

Alegeți o opțiune:

- a. **Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare**
- b. **Pentru ca anumite programe scriu date in memorie intr-o anumita ordine**
- c. **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare** ✔
- d. **Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers**

Răspunsul corect este: **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare**

9 întrebare

Corect

Marcat 1,00 din 1,00

Hazardele de control:

Alegeți o opțiune:

- a. **Sunt determinate în etapa de Decode sau Execute**
- b. **Sunt produse doar de operații tip JMP**
- c. **Influențează adresa următoare din Instruction Register** ✔
- d. **Necesită o unitate dedicată de control**
- e. **Apar doar la arhitecturile RISC**

Răspunsul corect este: **Influențează adresa următoare din Instruction Register**

10 întrebare

Corect

Marcat 1,00 din 1,00

Pentru a putea rula la 0.5GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 2V, iar curentul de mers în gol al acestuia este 10mA. Știind că procesorul are o capacitate parazită echivalentă de 300nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:

- a. **300.02W** ✔
- b. **100.1W**
- c. **300.03W**
- d. **50W**
- e. **300W**

Răspunsul corect este: **300.02W**

◄ Inregistrare curs 11&12 Salt la... Examen - Partea II ►

Început la	vineri, 19 iunie 2020, 08:08
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:46
Timp luat	38 min
Notează	**9,00** din maxim 10,00 (**90**%) posibil

1 întrebare

Corect

Marcat 1,00 din 1,00

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	HLT	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **9** ✔
- b. **Nu se poate determina**
- c. **8**
- d. **A**
- e. **B**

Răspunsul corect este: **9**

2 întrebare

Corect

Marcat 1,00 din 1,00

Adevarat sau fals: Procesorul cu banda de asamblare executa toate instructiunile intr-un singur ciclu de ceas.

Selectați o opțiune:

- Adevărat
- Fals ✔

Răspunsul corect este "Fals".

3 întrebare

Incorect

Marcat 0,00 din 1,00

Dacă îmbunătățim timpul de acces la memorie, atunci în ce sens putem opera asupra frecvenței procesorului?

Alegeți o opțiune:

- a. **Rămâne neschimbată**
- b. **Nu se poate determina** ✖
- c. **Se apropie de 3GHz**
- d. **Creștere**
- e. **Scădere**

Răspunsul corect este: **Creștere**

4 întrebare

Corect

Marcat 1,00 din 1,00

De ce este important sa stim daca o masina este big-endian sau little-endian?

Alegeți o opțiune:

- a. **Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare**
- b. **Pentru ca anumite programe scriu date in memorie intr-o anumita ordine**
- c. **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare** ✔
- d. **Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers**

Răspunsul corect este: **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare**

5 întrebare

Corect

Marcat 1,00 din 1,00

Numarul 0xC000FEFE, stocat ca *signed int (32 de biți)* este:

Alegeți o opțiune:

- a. **Nicio opțiune**
- b. **Pozitiv**
- c. **Prea mare pentru a putea fi reprezentat**
- d. **Negativ** ✔

Răspunsul corect este: **Negativ**

6 întrebare

Corect

Marcat 1,00 din 1,00

Care este, în medie, efectul pe care dublarea numărului de registre generale îl are asupra ratei apariției hazardelor de date?

Alegeți o opțiune:

- a. **Va crește**
- b. **Ramâne constantă**
- c. **Va scade** ✔
- d. **Imposibil de determinat**
- e. **Le elimină complet**

Răspunsul corect este: **Va scade**

7 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că un procesor în b.a. execută următoarea operație: *addi r1, 0xbeef*. În ce etapă este actualizat *r1*?

Alegeți o opțiune:

- a. **Execute**
- b. **Decode**
- c. **Memory**
- d. **Write Back** ✔
- e. **Instruction Fetch**

Răspunsul corect este: **Write Back**

8 întrebare

Corect

Marcat 1,00 din 1,00

Dacă trebuie să proiectați un procesor performant (frecvență mare de ceas, superscalar) folosind tehnologia actuală, ce fel de ISA ați alege pentru acesta?

Alegeți o opțiune:

- a. **Mașină virtuală Java**
- b. **RISC** ✔
- c. **MISC**
- d. **CISC**
- e. **VLIW**

Răspunsul corect este: **RISC**

9 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor petrece 40% din timp cu operații UAL, 30% în operații cu memoria și 30% din timp cu alte operații. Dacă puteți îmbunătăți performanța UAL-ului de 5 ori, accesul la memorie de 10 ori dar, ca rezultat veți încetini execuția celorlalte instrucțiuni cu 30%, ce speedup obțineți?

Alegeți o opțiune:

- a. **2** ✔
- b. **1.5**
- c. **3.33**
- d. **5**
- e. **10**

Răspunsul corect este: **2**

10 întrebare

Corect

Marcat 1,00 din 1,00

Pentru a putea rula la 1GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 1V, iar curentul de mers în gol al acestuia este 50mA. Știind că procesorul are o capacitate parazită echivalentă de 100nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:

- a. **100.1W**
- b. **50.05W** ✔
- c. **50W**
- d. **100.05W**
- e. **100W**

Răspunsul corect este: **50.05W**

◄ Inregistrare curs 11&12 Salt la... Examen - Partea II ►

Început la	vineri, 19 iunie 2020, 08:08
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:43
Timp luat	35 min 24 secs
Notează	**7,00** din maxim 10,00 (**70**%) posibil

1 întrebare

Incorect

Marcat 0,00 din 1,00

Dacă îmbunătățim timpul de acces la memorie, atunci în ce sens putem opera asupra frecvenței procesorului?

Alegeți o opțiune:

- a. **Se apropie de 3GHz**
- b. **Scădere**
- c. **Creștere**
- d. **Nu se poate determina** ✖
- e. **Rămâne neschimbată**

Răspunsul corect este: **Creștere**

2 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor petrece 60% din timp cu operații UAL. De câte ori mai rapidă trebuie să fie unitatea aritmetică logică pentru ca procesorul să ruleze de două ori mai repede?

Alegeți o opțiune:

- a. **5**
- b. **3**
- c. **2**
- d. **1.5**
- e. **6** ✔

Răspunsul corect este: **6**

3 întrebare

Corect

Marcat 1,00 din 1,00

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	HLT	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **9** ✔
- b. **Nu se poate determina**
- c. **B**
- d. **A**
- e. **8**

Răspunsul corect este: **9**

4 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor cu arhitectura Harvard:

Alegeți o opțiune:

- a. **Toate de mai sus**
- b. **Are memorii de date si instructiuni separate** ✔
- c. **Executa orice operatie intr-un singur ciclu de ceas**
- d. **Are o banda de asamblare cu cinci etape**

Răspunsul corect este: **Are memorii de date si instructiuni separate**

5 întrebare

Corect

Marcat 1,00 din 1,00

Hazardele de date:

Alegeți o opțiune:

- a. **Au latenta minima**
- b. **Sunt cauzate de UAL**
- c. **Apar doar la arhitecturile RISC**
- d. **Pot fi solutionate prin bypassing** ✔

Răspunsul corect este: **Pot fi solutionate prin bypassing**

6 întrebare

Corect

Marcat 1,00 din 1,00

Pentru a putea rula la 0.5GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 2V, iar curentul de mers în gol al acestuia este 10mA. Știind că procesorul are o capacitate parazită echivalentă de 200nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:

- a. **200.02W** ✔
- b. **200W**
- c. **100.1W**
- d. **100.01W**
- e. **50W**

Răspunsul corect este: **200.02W**

7 întrebare

Incorect

Marcat 0,00 din 1,00

Dacă CPU-ul este implementat în logică rapidă, în timp ce accesul la memorie este de cel puțin 10x mai lent, ce fel de ISA alegi pentru a contracara pe cât de mult posibil limitările hardware?

Alegeți o opțiune:

- a. **Mașină virtuală Java**
- b. **VLIW**
- c. **CISC**
- d. **RISC** ✖
- e. **MISC**

Răspunsul corect este: **CISC**

8 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că un procesor fără b.a. execută următoarea operație: *ldi r16, 0xdeadc0de*. În care etapă este actualizat *r16*?

Alegeți o opțiune:

- a. **Instruction Fetch**
- b. **Decode**
- c. **Memory**
- d. **Execute**
- e. **Write Back** ✔

Răspunsul corect este: **Write Back**

9 întrebare

Corect

Marcat 1,00 din 1,00

Numărul 2.25, poate fi reprezentat în cod IEEE 754 single precision ca:

Alegeți o opțiune:

- a. **0x40100000** ✔
- b. **0x40200000**
- c. **0x40600000**
- d. **Nicio opțiune**
- e. **0x40800000**

Răspunsul corect este: **0x40100000**

10 întrebare

Incorect

Marcat 0,00 din 1,00

Adevarat sau fals: Un procesor CISC este în medie mai lent decât un procesor RISC.

Selectați o opțiune:

- Adevărat ✖
- Fals

Răspunsul corect este "Fals".

◄ Inregistrare curs 11&12

Salt la...

Examen - Partea II ►

Început la	vineri, 19 iunie 2020, 08:05
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:35
Timp luat	29 min 49 secs
Notează	**8,00** din maxim 10,00 (**80**%) posibil

1 întrebare

Corect

Marcat 1,00 din 1,00

Adevarat sau fals: O intrerupere ne-mascabila se executa intotdeauna de catre procesor.

Selectați o opțiune:

- Adevărat ✔
- Fals

Răspunsul corect este "Adevărat".

2 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor petrece 50% din timp cu operatii UAL si 40% in operatii cu memoria. Daca puteti imbunatati timpul de acces la memorie de 4 ori, dar, ca rezultat veti incetini executia celorlalte instructiuni cu 50%, ce speedup obtineti?

Alegeți o opțiune:

- a. **3.33**
- b. **nu poate fi determinat**
- c. **1** ✔
- d. **5.3**
- e. **2**

Răspunsul corect este: **1**

3 întrebare

Incorect

Marcat 0,00 din 1,00

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	0x01	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **8**
- b. **B**
- c. **9** ✖
- d. **A**
- e. **Nu se poate determina**

Răspunsul corect este: **Nu se poate determina**

4 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că un procesor fără b.a. execută următoarea operație: *addi r14, 0xda*. În care etapă este actualizat *r14*?

Alegeți o opțiune:

- a. **Decode**
- b. **Execute**
- c. **Write Back** ✔
- d. **Instruction Fetch**
- e. **Memory**

Răspunsul corect este: **Write Back**

5 întrebare

Incorect

Marcat 0,00 din 1,00

Întreruperile mascabile:

Alegeți o opțiune:

- a. **Produc bule de exact o instrucțiune lungime în b.a.** ✖
- b. **Sunt întotdeauna sincrone cu ceasul sistemului**
- c. **Poartă și numele de excepții**
- d. **Pot fi implementate doar de procesoarele RISC**
- e. **Pot să nu fie tratate de procesor**

Răspunsul corect este: **Pot să nu fie tratate de procesor**

6 întrebare

Corect

Marcat 1,00 din 1,00

Numărul -1.75, poate fi reprezentat în cod IEEE 754 single precision ca:

Alegeți o opțiune:

- a. **0xcfe00000**
- b. **Nicio opțiune**
- c. **0xdfe00000**
- d. **0xafe00000**
- e. **0xbfe00000** ✔

Răspunsul corect este: **0xbfe00000**

7 întrebare

Corect

Marcat 1,00 din 1,00

Dacă trebuie să proiectați un procesor performant (frecvență mare de ceas, superscalar) folosind tehnologia actuală, ce fel de ISA ați alege pentru acesta?

Alegeți o opțiune:

- a. **CISC**
- b. **Mașină virtuală Java**
- c. **RISC** ✔
- d. **MISC**
- e. **VLIW**

Răspunsul corect este: **RISC**

8 întrebare

Corect

Marcat 1,00 din 1,00

De ce este important sa stim daca o masina este big-endian sau little-endian?

Alegeți o opțiune:

- a. **Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare**
- b. **Pentru ca anumite programe scriu date in memorie intr-o anumita ordine**
- c. **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare** ✔
- d. **Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers**

Răspunsul corect este: **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare**

9 întrebare

Corect

Marcat 1,00 din 1,00

Hazardele de control:

Alegeți o opțiune:

- a. **Sunt determinate în etapa de Decode sau Execute**
- b. **Sunt produse doar de operații tip JMP**
- c. **Influențează adresa următoare din Instruction Register** ✔
- d. **Necesită o unitate dedicată de control**
- e. **Apar doar la arhitecturile RISC**

Răspunsul corect este: **Influențează adresa următoare din Instruction Register**

10 întrebare

Corect

Marcat 1,00 din 1,00

Pentru a putea rula la 0.5GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 2V, iar curentul de mers în gol al acestuia este 10mA. Știind că procesorul are o capacitate parazită echivalentă de 300nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:

- a. **300.02W** ✔
- b. **100.1W**
- c. **300.03W**
- d. **50W**
- e. **300W**

Răspunsul corect este: **300.02W**

◄ Inregistrare curs 11&12 Salt la... Examen - Partea II ►

Început la	vineri, 19 iunie 2020, 09:01
Status	Terminat
Completat la	vineri, 19 iunie 2020, 09:41
Timp luat	40 min
Notează	**10,00** din maxim 10,00 (**100**%) posibil

1 întrebare

Corect

Marcat 3,00 din 3,00

Fie următorul cod în assembly (x86). Indentificați hazardele de date. Pentru fiecare menționați intrucțiunile implicate, arătând cauzele fiecărui hazard.

Treceți hazardele în ordinea crescătoare a instrucțiunilor.

Exemplu: Dacă există:

- un hazard de tipul RAW între liniile x și y (x < y) din cauza registrului eax
- un hazard de tipul RAW între liniile x și z (x < z, y < z) din cauza registrului eax
- un hazard de tipul RAW între liniile t și z (x < t, t < z) din cauza registrului eax

atunci veți scrie:

Ix - Iy: RAW (eax)

Ix - Iz: RAW (eax)

It - Iz: RAW (eax)

```
0: mov eax, [0x00DEC0DE]    ; eax = MEM[0x00DEC0DE]
1: add ecx, eax             ; ecx = ecx + eax
2: sub edx, ebx             ; edx = edx - ebx
3: mov [0x0000F00D], edx    ; MEM[0x0000F00D] = edx
4: add eax, edx             ; eax = eax + edx
```

Procesor **single-core**:

I 0 ✔ - I 1 ✔ : RAW ✔ (eax ✔)

I 2 ✔ - I 3 ✔ : RAW ✔ (edx ✔)

I 2 ✔ - I 4 ✔ : RAW ✔ (edx ✔)

In plus, la **multi-core**:

I 0 ✔ - I 4 ✔ : WAW ✔ (eax ✔)

I 1 ✔ - I 4 ✔ : WAR ✔ (eax ✔)

I Nu mai exista ✔ - I Nu mai exista ✔ : Nu mai exista ✔ (Nu mai exista ✔)

Vom puncta hazardele identificate corect chiar daca nu au fost puse in ordinea crescatoare a numarului instructiunii.

2 întrebare

Corect

Marcat 7,00 din 7,00

Avem un procesor cu frecvența de **2GHz**, care execută acest cod.

Știm numărul de cicli de ceas pentru fiecare tip de instrucțiune:

- operațiile cu date și memorie (**mov**): **2 cicli**;
- operațiile aritmetice/logice (**cmp, add, inc, dec, xor, and, test, shl**): **1 ciclu**;
- operațiile de salt necondiționat (**jmp**): **5 cicli**;
- operațiile de salt condiționat (**jg, jl, je/jz, jbe**): **20 cicli** (dacă nu a fost prezis corect) sau **5 cicli** (dacă a fost prezis corect);
- afisarea: **1 ciclu**.

Fie procesorul B care implementează un branch predictor **global**, iar metoda de branch prediction folosită este **2-bit counter** (strongly taken/weak taken/weak not taken/strongly not taken). Starea inițială este **strongly not taken**.

Se consideră inputul **c = 5**.

1. Ce valoare afișează programul?

14 ✖

2. Enumerați prin ce stări trece branch predictorul, semnalând dacă branch-ul a fost prezis corect sau greșit.

	Iteratia 1	Iteratia 2	Iteratia 3	Iteratia 4	Iteratia 5
`xor eax, eax`	eax: 0 ✔ 1 ✔				
`xor ebx, ebx`	ebx: 0 ✔ 1 ✔				
`inc ebx`	ebx: 1 ✔ 1 ✔				
`mov ecx, $c`	ecx: 5 ✔ 2 ✔				
`count_loop:`					
`test ecx, ecx`	1 ✔	1 ✔	1 ✔	1 ✔	1 ✔
`je done`	5 ✔ Corect ✔ SNT ✔	5 ✔ Corect ✔ SNT ✔	5 ✔ Corect ✔ SNT ✔	5 ✔ Corect ✔ SNT ✔	5 ✔ Corect ✔ SNT ✔
`test ecx, ebx`	1 ✔	1 ✔	1 ✔	1 ✔	1 ✔

	Iteratia 1	Iteratia 2	Iteratia 3	Iteratia 4	Iteratia 5
jz no_add	5 ✔ Corect ✔ SNT ✔	5 ✖ Corect ✖ SNT ✖	5 ✔ Corect ✔ SNT ✔	5 ✖ Corect ✖ SNT ✖	20 ✖ Gresit ✖ WNT ✖
add eax, ecx	eax: 5 ✔ 1 ✔	eax: 9 ✖ 1 ✖	eax: 12 ✖ 1 ✔	eax: 14 ✖ 1 ✖	eax: 0 ✖ Nu se executa ✖
no_add:					
dec ecx	ecx: 4 ✔ 1 ✔	ecx: 3 ✔ 1 ✔	ecx: 2 ✔ 1 ✔	ecx: 1 ✔ 1 ✔	ecx: 0 ✔ 1 ✔
jmp count_loop	5 ✔	5 ✔	5 ✔	5 ✔	5 ✔
done:					
;se afiseaza conținutul lui eax (trebuie trecuti cicli)	Nu se executa ✔	Nu se executa ✔	Nu se executa ✔	Nu se executa ✔	Nu se executa ✔

3. Câte branch-uri au fost prezise corect?

9 ✖

4. Câte branch-uri au fost prezise incorect?

2 ✖

5. Care este timpul de rulare al programului (in ns)?

68 ✖

Din cauza multiplelor probleme cu acest subiect (Branch Predictor), va fi acordat punctajul intreg.

Comentariu:

◄ Examen - Partea I

Salt la...

Model Colocviu ►

Laboratorul 2 – Implementarea unui automat cu stari finite

Obiective

Acest laborator este o introducere in proiectarea logica a automatelor cu stari finite (Finite State Machines - FSM). In cadrul laboratorului vor fi prezentate diversele tipuri de FSM si moduri de proiectare in vederea implementarii. Scopul acestuia este de a va familiariza cu unele concepte de proiectare pe care le veti utiliza in viitor.
Deasemenea, in cadrul acestui laborator se vor introduce notiuni privind modularizarea unui proiect si reutilizarea codului.
Pentru a obtine punctajul acordat acestui laborator, trebuie sa prezentati laborantului functionarea corecta in hardware a proiectului dumneavoastra.

Considerente de proiectare

Exista mai multe metode de a proiecta automate cu stari finite. Totusi, urmand anumite metode de proiectare se asigura faptul ca unealta de sinteza identifica si optimizeaza algoritmul ce se doreste a fi implementat, imbunatatind simularea, intarzierile si depanarea circuitului.
Automatele cu stari finite se pot imparti in implementari: Mealy vs. Moore, One-hot vs. Binary sau Safe vs. Fast. Principalele compromisuri intre aceste implementari sunt explicate in sectiunile urmatoare. Recomandarea generala pentru alegerea unui tip de automat cu stari finite depinde de arhitectura dispozitivului tinta si de specificul automatului (dimeniune si comportament). Totusi, in automatele de tip Moore, One-hot se implementeaza mai bine in FPGA-uri, iar cele Mealy, Binary se implementeaza mai bine in CPLD-uri.

Mealy vs. Moore

Sunt doua tipuri bine cunoscute de implementari pentru automate cu stari finite: Mealy si Moore. Diferenta intre ele este ca automatele Mealy determina valorile iesirilor bazanduse si pe starea curenta si pe intrari, in timp ce automatele Moore determina valorile iesirilor numai pe baza starii curente. In general, automatele de tip Moore se implementeaza mai bine in FPGA-uri deoarece cel mai des se alege codarea One-hot si deci este foarte usor (chiar trivial) sa se obtina valorile iesirilor. Daca se alege codarea binara (Binary) este posibil sa se ajunga la un automat mai compact si cateodata mai rapid utilizand automate de tip Mealy. Totusi, acest lucru nu este intotdeauna valabil si nu este usor de determinat fara a cunoaste specificul automatului cu stari finite.

Codarea One-hot vs. Binary

Exista mai multe metode de a codifica starile unui automat. Totusi, doua sunt utilizate mai des in proiectarea cu FPGA-uri sau CPLD-uri. Acestea sunt One-hot si Binary. Pentru cele mai multe arhitecturi FPGA, codarea One-hot este mai buna datorita numarului mare de bistabili (se mapeaza mai bine in interiorul LUT-urilor). Cand dispozitivul tinta este un CPLD, codarea binara (Binary) poate fi de multe ori mai eficienta datorita structurii logice a CPLD-ului si a resurselor mai putin numeroase. In orice caz, majoritatea uneltelor de sinteza contin un algoritm de extragere a FSM-urilor ce poate identifica structura si poate alege cea mai buna metoda de codare pentru marimea, tipul si arhitectura tinta. Desi aceasta facilitate exista (la Xilinx este State Diagram), in cele mai multe cazuri este mai eficient sa aleaga de catre proiectant modul de codare pentru a permite un control mai ridicat si o depanare mai usoara. Este indicat sa consultati documentatia uneltei de sinteza pentru detalii privind capacitatea de extragere a automateleor.

Safe vs. Fast

Cand se proiecteaza un automat cu stari finite, in general sunt doua tinte care se contrazic si trebuiesc intelese: siguranta (Safe) si viteza (Fast). O implementare sigura se refera la cazul in care automatul ar primi un set de valori de intrare necunoscut sau ajunge intr-o stare necunoscuta si poate sa revina intr-o stare cunoscuta la urmatoarea perioada de ceas. Pe de alta parte, daca se renunta la aceasta cerinta, de cele mai multe ori automatul poate fi implementat utilizand mai putine resurse si mai rapid.

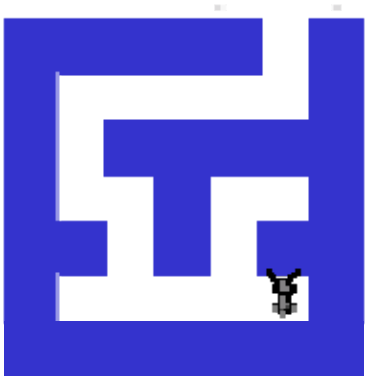
Descrierea problemei

Se cere sa se proiecteze, folosind circuite FPGA, automatul secvential reprezentand unitatea de comanda a unei “insecte electronice” (RoboAnt®), care se deplaseaza intr-un labirint. De asemenea, este realizat un simulator software/hardware, cu interfata grafica, pentru urmarirea comportarii insectei in labirint.
Specificatii de proiectare:

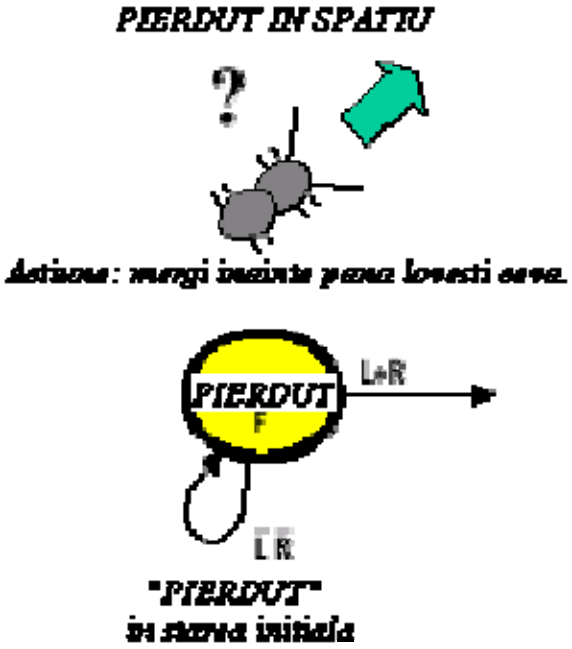
- Senzori: doua antene L (stanga) si R (dreapta), care vor furniza semnalul 1 atunci cand vin in contact cu un obstacol

- Organe de executie: Pas inainte F, Rotire cu un unghi stanga TL si dreapta TR.

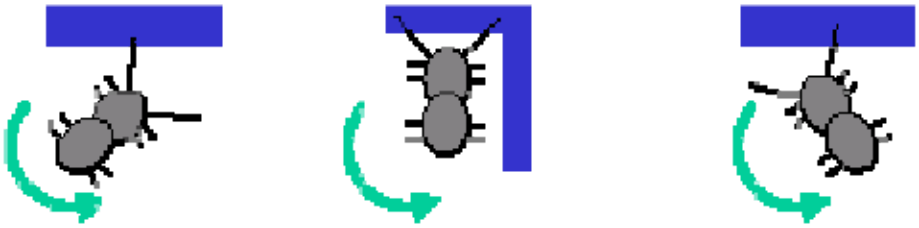
- Scopul urmarit: inzestrarea insectei cu „inteligenta”, astfel incat ea sa poata iesi din labirint.



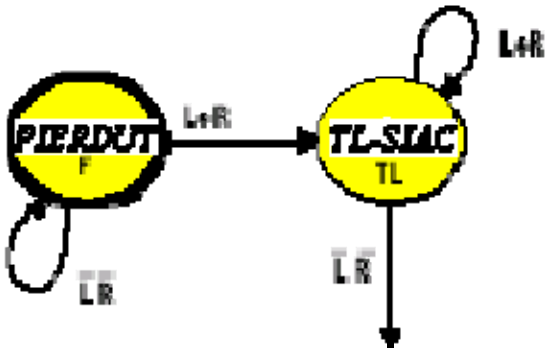
Strategia: “Antena din dreapta pe peretele labirintului”.
Situatii in care se poate afla insecta:



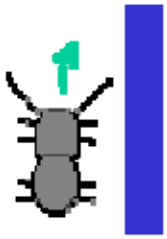
ATINGERE OBSTACOL



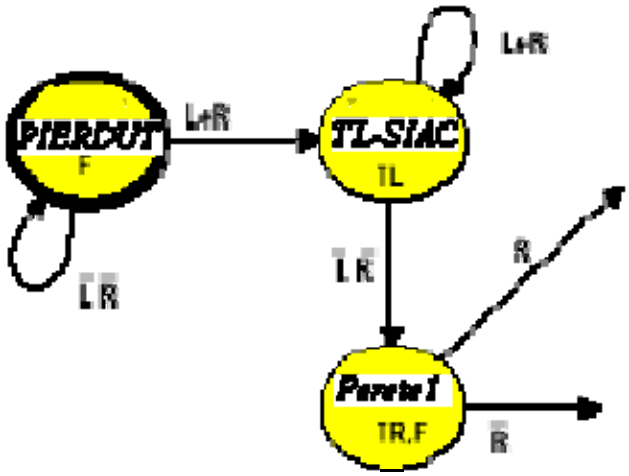
Actiune: TL (Stop Invers Atelor de Coarnie -SIAC), pana cand obstacolul nu mai este atins



PUTIN LA DREAPTA...



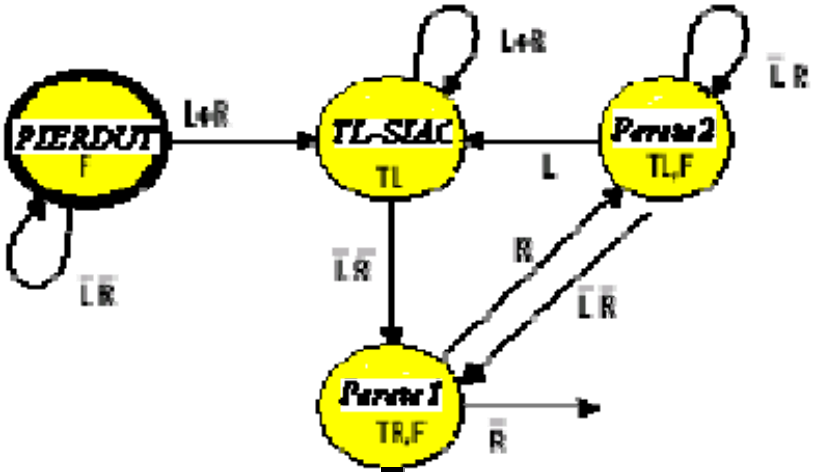
Actiune: TR si F, stabileste daca a fost atins un obstacol

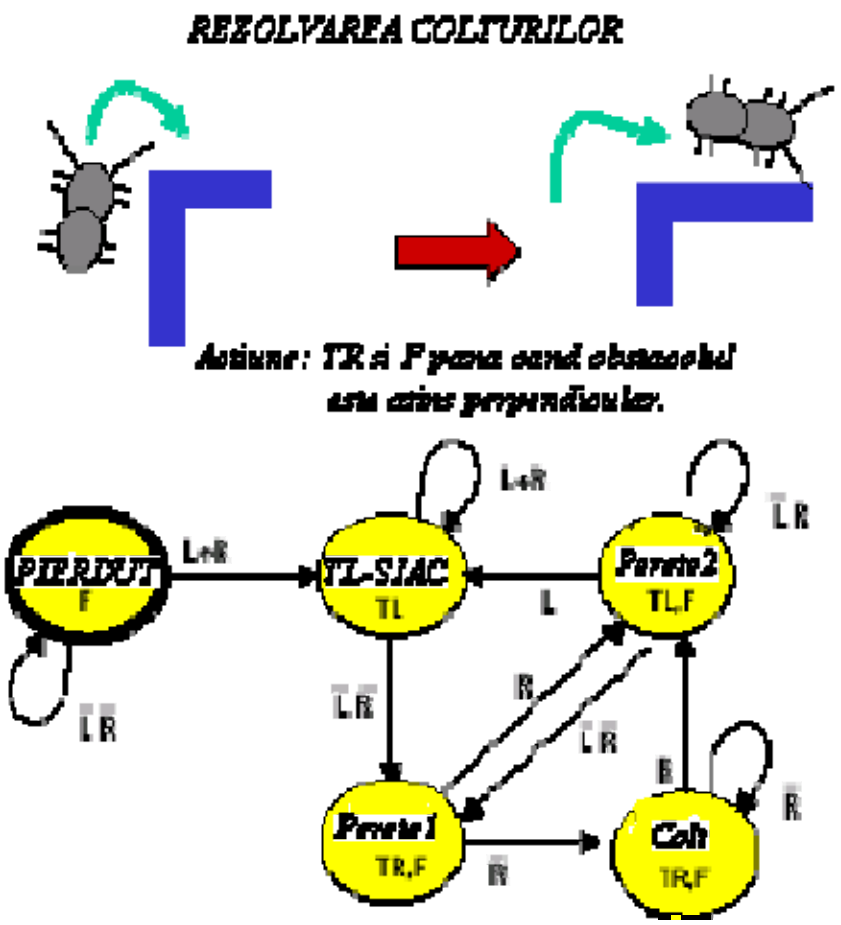


APOI PUTIN LA STANGA..



Actiune: TL si F,stabileste daca a fost atins un obstacol





Reducerea starilor echivalente:

Observatie: Si = Sj daca

1. Starile au iesiri identice;
2. Fiecare intrare -> stari echivalente.

Strategia de Reducere: Gaseste perechi de stari echivalente si efectueaza FUZIUNEA lor.

Observam ca strategia “Antena din dreapta pe peretele labirintului” nu conduce insecta spre iesire in cazul unui labirint cu insule (odata ce insecta ajunge in contact cu o insula, se va invarti la nesfarsit in jurul ei). Pentru a suplini acest dezavantaj se introduce un nou senzor S (Smell) si doua actiuni M si E (Mark si Erase). Scopul acestora este de a putea marca drumul pe care am mai fost si astfel de a evita cazul in care insecta se va invarti in jurul insulei. Cand insecta detecteaza (prin smell) ca a ajuns intr-o zona in care a facut anterior mark, ea nu va mai lua aceeasi „decizie” pentru a nu repeta parcusul anterior.

Pentru a putea testa intr-un mediu „real” unitatea de control (botezata in continuare mind), se pune la dispozitie un proiect ce contine o interfata VGA si un labirint memorat (alaturi de conexiunile necesare) la care se va adauga implementarea modulului cu urmatoarea interfata:

```
module mind(clk, clr, L, R, S, TL, TR, F, M, E);
   input clk;
   input clr;
   input L;
   input R;
   input S;
   output TL;
   output TR;
   output F;
   output M;
   output E;
   // Implementare
endmodule
```

In cazul in care nu se doreste implementarea astfel incat sa se poata trata si labirinturile ce contin insule, comenzile M si E vor fi permanent 0 iar intrarea S va fi ignorata in luarea deciziilor.

In continuare se prezinta diverse structuri pentru implementarea in Verilog a tipurilor de automate cu stari finite mentionate in paragrafele anterioare.

1. Mealy – One-Hot – Fast – 4 states

```
parameter <state1> = 4'b0001;
parameter <state2> = 4'b0010;
parameter <state3> = 4'b0100;
parameter <state4> = 4'b1000;

reg [3:0] state = <state1>;

always@(posedge <clock>)
  if (<reset>) begin
```

```
  state <= <state1>;
end
else
  case (state)
    <state1> : begin
      if (<condition>)
        state <= <next_state>;
      else if (<condition>)
        state <= <next_state>;
      else
        state <= <next_state>;
    end
    <state2> : begin
      if (<condition>)
        state <= <next_state>;
      else if (<condition>)
        state <= <next_state>;
      else
        state <= <next_state>;
    end
    <state3> : begin
      if (<condition>)
        state <= <next_state>;
      else if (<condition>)
        state <= <next_state>;
      else
        state <= <next_state>;
    end
    <state4> : begin
      if (<condition>)
```

```
            state <= <next_state>;
          else if (<condition>)
            state <= <next_state>;
          else
            state <= <next_state>;
        end
      endcase

assign <output1> = <logic_equation_based_on_states_and_inputs>;
assign <output2> = <logic_equation_based_on_states_and_inputs>;
```

2. Mealy – One-Hot – Safe – 4 states

```
parameter <state1> = 4'b0001;
parameter <state2> = 4'b0010;
parameter <state3> = 4'b0100;
parameter <state4> = 4'b1000;

reg [3:0] state = <state1>;

always@(posedge <clock>)
  if (<reset>) begin
    state <= <state1>;
  end
  else
    case (state)
      <state1> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
```

```
  else
    state <= <next_state>;
end
<state2> : begin
  if (<condition>)
    state <= <next_state>;
  else if (<condition>)
    state <= <next_state>;
  else
    state <= <next_state>;
end
<state3> : begin
  if (<condition>)
    state <= <next_state>;
  else if (<condition>)
    state <= <next_state>;
  else
    state <= <next_state>;
end
<state4> : begin
  if (<condition>)
    state <= <next_state>;
  else if (<condition>)
    state <= <next_state>;
  else
    state <= <next_state>;
end
default : begin  // Fault Recovery
  state <= <state1>;
end
```

```
    endcase

assign <output1> = <logic_equation_based_on_states_and_inputs>;
assign <output2> = <logic_equation_based_on_states_and_inputs>;
```

3. Mealy – Binary – Fast – 4 states

```
 parameter <state1> = 2'b00;
parameter <state2> = 2'b01;
parameter <state3> = 2'b10;
parameter <state4> = 2'b11;

reg [1:0] state = <state1>;

always@(posedge <clock>)
  if (<reset>) begin
    state <= <state1>;
  end
  else
    case (state)
      <state1> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
        else
          state <= <next_state>;
      end
      <state2> : begin
        if (<condition>)
          state <= <next_state>;
```

```
        else if (<condition>)
          state <= <next_state>;
        else
          state <= <next_state>;
      end
      <state3> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
        else
          state <= <next_state>;
      end
      <state4> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
        else
          state <= <next_state>;
      end
    endcase

  assign <output1> = <logic_equation_based_on_states_and_inputs>;
  assign <output2> = <logic_equation_based_on_states_and_inputs>;
```

4. Mealy – Binary – Safe – 4 states

```
  parameter <state1> = 2'b00;
  parameter <state2> = 2'b01;
  parameter <state3> = 2'b10;
```

```
parameter <state4> = 2'b11;

reg [1:0] state = <state1>;

always@(posedge <clock>)
  if (<reset>) begin
    state <= <state1>;
  end
  else
    case (state)
      <state1> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
        else
          state <= <next_state>;
      end
      <state2> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
        else
          state <= <next_state>;
      end
      <state3> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
```

```
            state <= <next_state>;
          else
            state <= <next_state>;
        end
        <state4> : begin
          if (<condition>)
            state <= <next_state>;
          else if (<condition>)
            state <= <next_state>;
          else
            state <= <next_state>;
        end
        default : begin  // Fault Recovery
          state <= <state1>;
        end
      endcase

    assign <output1> = <logic_equation_based_on_states_and_inputs>;
    assign <output2> = <logic_equation_based_on_states_and_inputs>;
```

5. Moore – One-Hot – Fast – 4 states

```
    parameter <state1> = 4'b0001;
    parameter <state2> = 4'b0010;
    parameter <state3> = 4'b0100;
    parameter <state4> = 4'b1000;

    reg [3:0] state = <state1>;

    always@(posedge <clock>)
      if (<reset>) begin
```

```
  state <= <state1>;
  <outputs> <= <initial_values>;
end
else
  case (state)
    <state1> : begin
      if (<condition>)
        state <= <next_state>;
      else if (<condition>)
        state <= <next_state>;
      else
        state <= <next_state>;
      <outputs> <= <values>;
    end
    <state2> : begin
      if (<condition>)
        state <= <next_state>;
      else if (<condition>)
        state <= <next_state>;
      else
        state <= <next_state>;
      <outputs> <= <values>;
    end
    <state3> : begin
      if (<condition>)
        state <= <next_state>;
      else if (<condition>)
        state <= <next_state>;
      else
        state <= <next_state>;
```

```
      <outputs> <= <values>;
    end
    <state4> : begin
      if (<condition>)
        state <= <next_state>;
      else if (<condition>)
        state <= <next_state>;
      else
        state <= <next_state>;
      <outputs> <= <values>;
    end
  endcase
```

6. Moore – One-Hot – Safe – 4 states

```
parameter <state1> = 4'b0001;
parameter <state2> = 4'b0010;
parameter <state3> = 4'b0100;
parameter <state4> = 4'b1000;

reg [3:0] state = <state1>;

always@(posedge <clock>)
  if (<reset>) begin
    state <= <state1>;
    <outputs> <= <initial_values>;
  end
  else
    case (state)
      <state1> : begin
        if (<condition>)
```

```
      state <= <next_state>;
    else if (<condition>)
      state <= <next_state>;
    else
      state <= <next_state>;
    <outputs> <= <values>;
end
<state2> : begin
    if (<condition>)
      state <= <next_state>;
    else if (<condition>)
      state <= <next_state>;
    else
      state <= <next_state>;
    <outputs> <= <values>;
end
<state3> : begin
    if (<condition>)
      state <= <next_state>;
    else if (<condition>)
      state <= <next_state>;
    else
      state <= <next_state>;
    <outputs> <= <values>;
end
<state4> : begin
    if (<condition>)
      state <= <next_state>;
    else if (<condition>)
      state <= <next_state>;
```

```
        else
          state <= <next_state>;
        <outputs> <= <values>;
      end
      default: begin  // Fault Recovery
        state <= <state1>;
        <outputs> <= <values>;
          end
    endcase
```

7. Moore – Binary – Fast – 4 states

```
 parameter <state1> = 2'b00;
parameter <state2> = 2'b01;
parameter <state3> = 2'b10;
parameter <state4> = 2'b11;

reg [1:0] state = <state1>;

always@(posedge <clock>)
  if (<reset>) begin
    state <= <state1>;
    <outputs> <= <initial_values>;
  end
  else
    case (state)
      <state1> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
```

```
  else
    state <= <next_state>;
  <outputs> <= <values>;
end
<state2> : begin
  if (<condition>)
    state <= <next_state>;
  else if (<condition>)
    state <= <next_state>;
  else
    state <= <next_state>;
  <outputs> <= <values>;
end
<state3> : begin
  if (<condition>)
    state <= <next_state>;
  else if (<condition>)
    state <= <next_state>;
  else
    state <= <next_state>;
  <outputs> <= <values>;
end
<state4> : begin
  if (<condition>)
    state <= <next_state>;
  else if (<condition>)
    state <= <next_state>;
  else
    state <= <next_state>;
  <outputs> <= <values>;
```

```
        end
      endcase
```

8. Moore – Binary – Safe – 4 states

```
parameter <state1> = 2'b00;
parameter <state2> = 2'b01;
parameter <state3> = 2'b10;
parameter <state4> = 2'b11;

reg [1:0] state = <state1>;

always@(posedge <clock>)
  if (<reset>) begin
    state <= <state1>;
    <outputs> <= <initial_values>;
  end
  else
    case (state)
      <state1> : begin
        if (<condition>)
          state <= <next_state>;
        else if (<condition>)
          state <= <next_state>;
        else
          state <= <next_state>;
        <outputs> <= <values>;
      end
      <state2> : begin
        if (<condition>)
          state <= <next_state>;
```

```
    else if (<condition>)
      state <= <next_state>;
    else
      state <= <next_state>;
    <outputs> <= <values>;
  end
  <state3> : begin
    if (<condition>)
      state <= <next_state>;
    else if (<condition>)
      state <= <next_state>;
    else
      state <= <next_state>;
    <outputs> <= <values>;
  end
  <state4> : begin
    if (<condition>)
      state <= <next_state>;
    else if (<condition>)
      state <= <next_state>;
    else
      state <= <next_state>;
    <outputs> <= <values>;
  end
  default : begin  // Fault Recovery
    state <= <state1>;
    <ouputs> <= <values>;
  end
endcase
```

Laboratorul 3 – Implementarea interfetelor cu mediul exterior

Obiective

Acest laborator isi propune sa prezinte modul de realizare a unor interfete cu mediul exterior astfel incat sa se poata trimite date si sa se vizualizeze rezultatele utilizand diverse implementari.

Proiectarea

Leduri

Cel mai simplu mod de a vizualiza un rezultat este comandarea unui led (sau a unui grup de leduri). Ledurile sunt conectate la masa si prin intermediul unei rezistente, ce are rolul de a limita curentul, la unul din pinii FPGA-ului. Astfel prin setarea iesirii pe valoarea „1” ledul se aprinde (cand valoarea iesirii este „0” ledul este stins). Pinii corespunzatori celor 8 leduri disponibile pe placa de test sunt:

LED1	P11
LED2	P10
LED3	P9
LED4	P8
LED5	P7
LED6	P6
LED7	P5
LED8	P4

Switchuri si butoane

Orice implementare are nevoie si de date de intrare. Cel mai simplu mod de a introduce date este prin intermediul switchurilor si a butoanelor.

Switchurile au doua pozitii. In pozitia „activa” ele conecteaza pinul corespunzator la Vcc prin intermediul unei rezistente. Astfel intrarea primeste valoarea „1”. In pozitia inactiva, switchul este conectat la masa (Gnd) si astfel inrarea primeste valoarea „0”.

Butoanele realizeaza contact doar la actionare revenind ulterior in starea initiala. Pinul de intrare al FPGA-ului pote fi conectat fie la masa (Gnd), fie la Vcc prin intermediul unei rezistente. Astfel se obtine valoarea permanenta „0”, respectiv „1”. Butonul are rolul de a conecta temprar pinul de intrare la valoarea opusa.

Pinii corespunzatori celor 8 switchuri si al butonului disponibile pe placa de test sunt:

SW1	P23
SW2	P22
SW3	P21
SW4	P20
SW5	P18
SW6	P17
SW7	P16
SW8	P15
BTN	P188

Ceasul

Pentru implementarea circuitelor logice sincrone este nevoie de un semnal de ceas cu o frecventa cunoscuta si stabila. Acest semnal se obtine de obicei de la un oscilator extern ce este conectat la FPGA prin intermediul unui pin de intrare special.

In cazul placii de test, oscilatorul are frecventa de 50MHz si este conectat la FPGA cu ajutorul pinului:

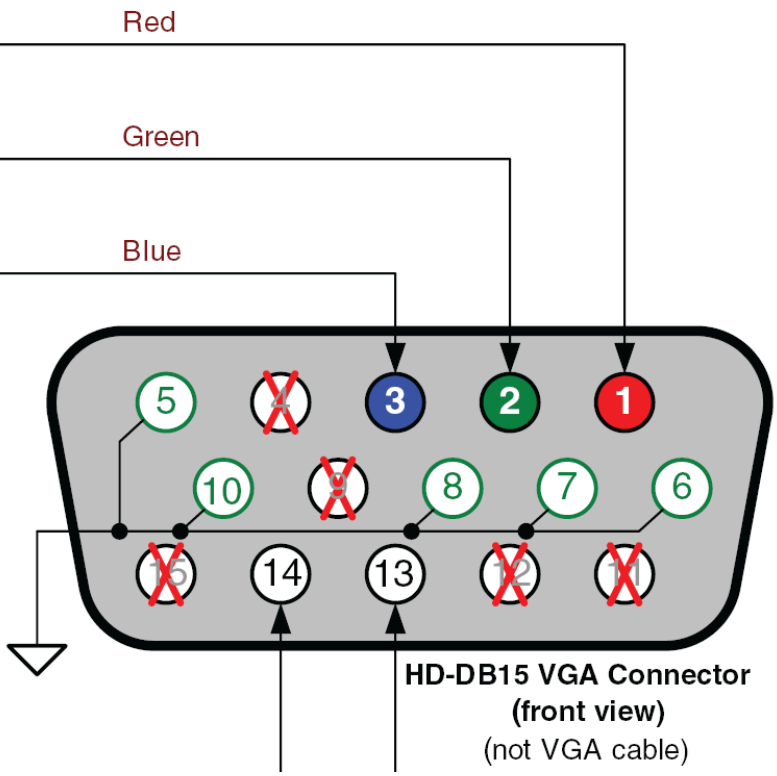
CLK	P182

Daca este nevoie de alte frecvente de lucru, se poate utiliza un alt oscilator prin intermediul unui soclu liber sau se poate deriva un semnal de ceas din cel initial cu ajutorul unor divizoare sau a componetelor de management al ceasului din cadrul FPGA-ului (DCM).

Modulul VGA

Deoarece posibilitatile de afisare pe leduri sunt limitate, iar interpretarea rezultatelor poate deveni greoaie, o buna alternativa este reprezentarea lor pe un monitor. Pentru a realiza acest lucru trebuie implementat un modul care genereaza semanlele necesare comandarii monitorului conform standardului prin intermediul conectorului HD-DB15.

Acest conector utilizeaza 5 semanale utile: Hsync, Vsync, Red, Green si Blue. Primele doua sunt semnale de sincronizare si au valori logice („0” sau „1”). Semnalele corespunzatoare celor trei culori au valori cuprinse intre 0 si 0.7 V si ele sunt obtinute cu ajutorul unor retele de rezistente de diverse valori. Pentru a obtine un numar mai mare de culori este nevoie de mai multe iesiri pentru fiecare culoare. In cazul placii de test, pentru fiecare culoare se utilizeaza 3 biti si astfel se pot obtine 512 valori distincte (3*3=9 si 2^9=512).



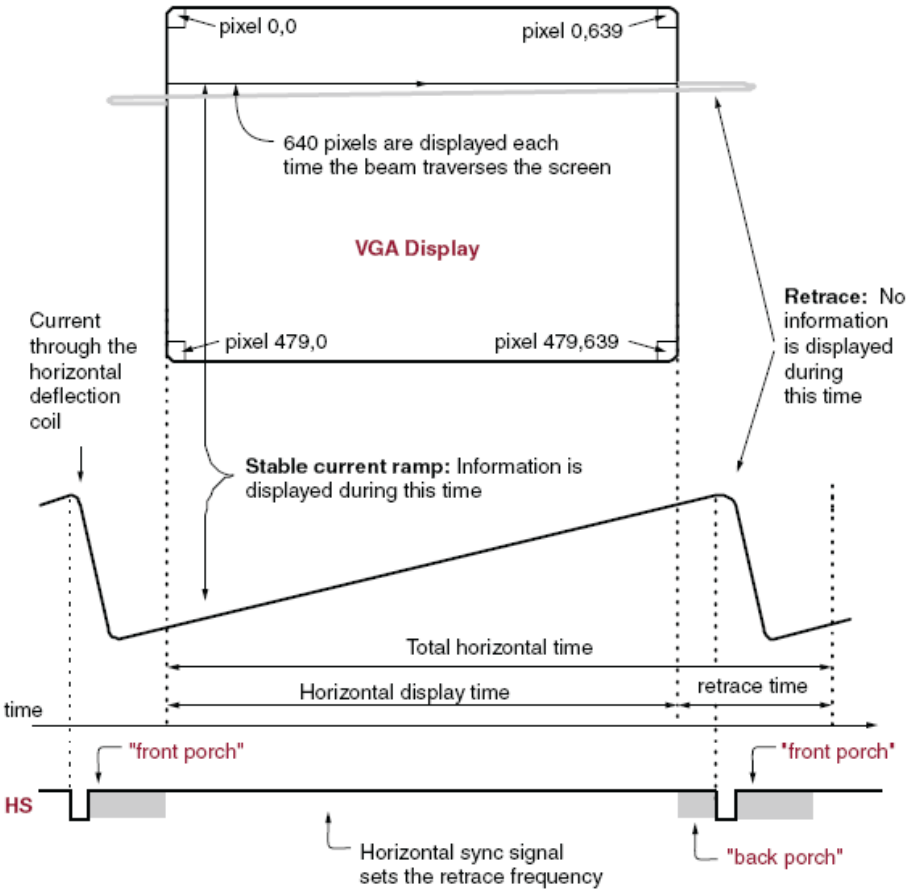
Pinii 13 si 14 sunt conectati la Hsync si respectiv Vsync

Semnalele de sincronizare sunt independente de numarul de culori ce se poate reprezenta.

Monitoarele CRT utilizeaza de modularea in amplitudine a fascicolelor de electroni emisi realizanduse astfel pozitionarea pe orizontala si verticala. Monitoarele LCD utilizeaza o matrice de comutatoare ce aplica o tensiune mica creistalelor lichide schimbandu-le astfel transparenta. Desi cele doua tipuri de monitoare utilizeaza metode diferite, semnalele de sincronizare sunt aceleasi. Afisarea incepe cu pixelul din stanga sus si se termina cu cel din dreapta jos. Ea se face pixel cu pixel (cate un pixel la fiecare perioada de ceas) pana cand se termina linia curenta si se trece la linia urmatoare (se afiseaza intai pe orizontala in cadrul unei linii).

Specificatiile sunt realizate de organizatia VESA (www.vesa.org), existand mai multe rezolutii standard astfel incat sa se poata utiliza diverse monitoare si conditiile in care rezultatele sa fie asemanatoare.

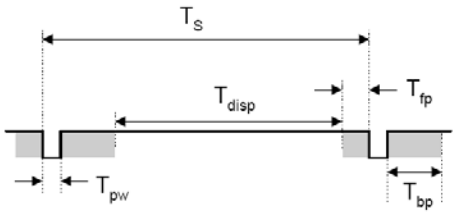
In continuare se va prezenta un exemplu pentru rezolutia de 640x480 pixeli cu o rata de 60Hz si valorile parametrilor pentru rezolutia folosita in cadul experimentelor de laboator (800x600 pixeli la 72Hz).



Pentru fiecare pixel se stabileste culoarea prin setarea valorilor adecvate pentru cei 9 biti (3 Red, 3 Green si 3 Blue).

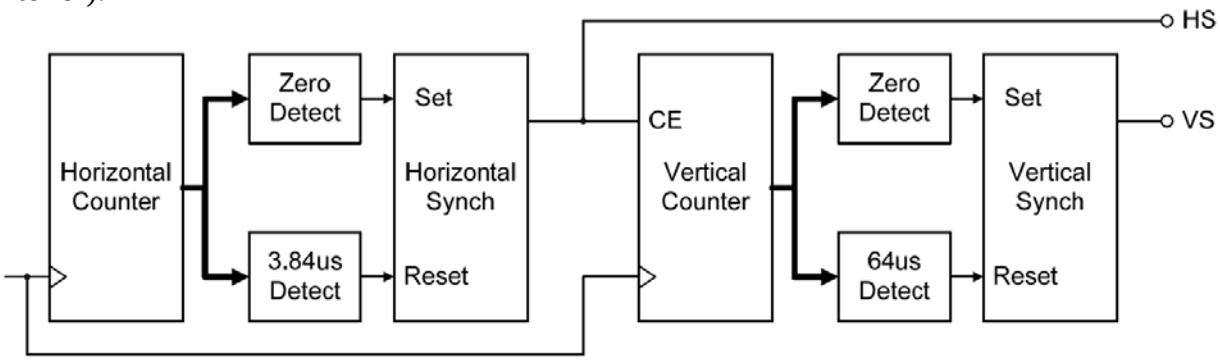
Rezolutia si rata de improspatare a imaginii sunt date de frecventa de ceas cu care este trimis semnalul. Astfel pentru un ceas cu o frecventa de 25MHz se obtine o rezolutie de 640x480 pixeli si 60Hz, iar pentru un ceas cu o frecventa de 50MHz se obtine rezoluita de 800x600 pixeli si 72Hz.

Symbol	Parameter	Vertical Sync			Horizontal Sync	
		Time	Clocks	Lines	Time	Clocks
T_S	Sync pulse time	16.7ms	416,800	521	32 us	800
T_{disp}	Display time	15.36ms	384,000	480	25.6 us	640
T_{pw}	VS pulse width	64 us	1,600	2	3.84 us	96
T_{fp}	VS front porch	320 us	8,000	10	640 ns	16
T_{bp}	VS back porch	928 us	23,200	29	1.92 us	48



640x480@60Hz	25MHz	640	48	96	16	480	29	2	10
640x480@60Hz	25,175MHz	640	48	96	16	480	33	2	10
800x600@72Hz	50MHz	800	64	120	56	600	23	6	37

Pentru generearea semnalelor de sincronizare (Hsync si Vsync) in functie de semnalul de ceas se poate folosi urmatoarea schema pricipiala (cu adaptarea parametrilor conform cu tabelul anterior):



Pinii corespunzatori celor 9 biti de culoare si ale celor 2 semnale de sincronizare sunt:

RED0	P201
RED1	P200
RED2	P199
GREEN0	P198
GREEN1	P194
GREEN2	P193
BLUE0	P192
BLUE1	P191
BLUE2	P189
HSYNC	P203
VSYNC	P202

Interfata PS/2 (mouse si tastatura)

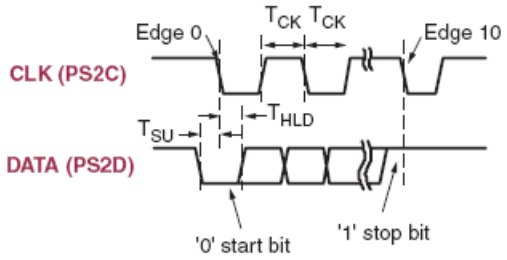
Atat mouseul cat si tastatura folosesc o magistrala cu doua fire pentru a comunica (PS/2). Un fir este folosit pentru sincronizare (transmiterea semnalului de ceas), iar al doilea fir este folosit pentru transmiterea datelor. Ambele dispozitive folosesc pentru comunicatie un protocol serial sincron ce transmite pachete de cate 11 biti (1 bit de start, 8 biti de date, un bit de paritate si un bit de stop). Pachetele de date pot circula in ambele sensuri (de exemplu se poate comanda aprinderea ledurilor de la tastatura sau schimbarea vitezei de comunicatie cu mouseul).

Diagrama semnalelor este prezentata in continuare. Semnalul de ceas si cel de date sunt active numai pe parcursul transferului (altfel ele sunt in starea de Idle si au valoarea logica „1”).

Placa de test are doua porturi fizice conectate la FPGA prin intermediul pinilor:

PS2_CLK1	P205
PS2_DATA1	P204
PS2_CLK2	P3
PS2_DATA2	P206

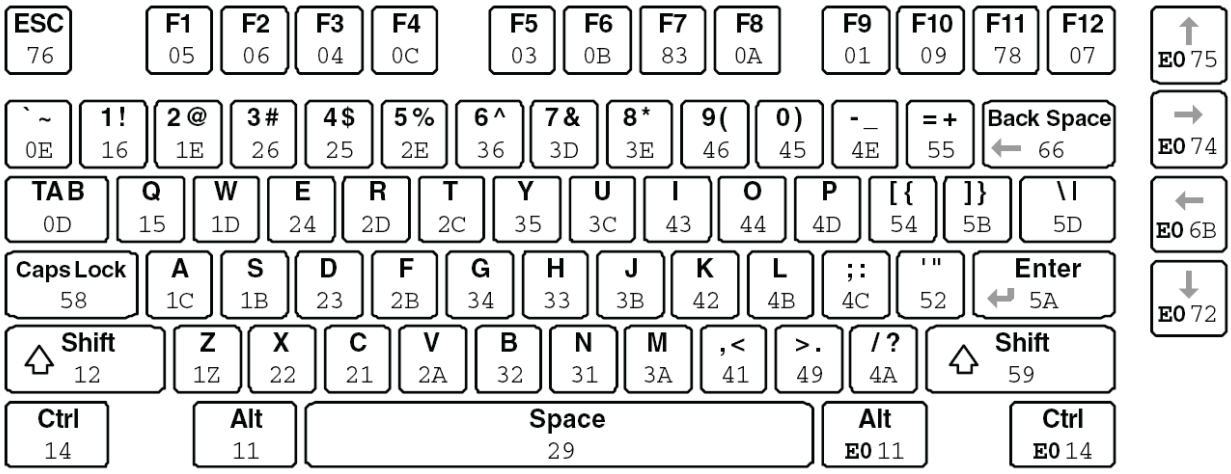
Symbol	Parameter	Min	Max
T_{CK}	Clock High or Low Time	30 µs	50 µs
T_{SU}	Data-to-clock Setup Time	5 µs	25 µs
T_{HLD}	Clock-to-data Hold Time	5 µs	25 µs



Tastatura

O tastatura PS/2 comunica prin trimiterea de coduri corespunzatoare tastelor. Codul unic este trimis de fiecare data cand este apasata tasta. Daca tasta este apasata continuu, codul se trimite periodic la interval de aproximativ 100ms. Cand tasta este eliberata se trimite un cod „F0” (eliberare tasta) urmat de codul tastei. Tastatura trimite acelasi cod chiar daca tasta permite generarea mai multor caractere. Combinatiile cu Shift (sau Caps Lock) se realizeaza la receptie.

Unele taste (numite extinse) trimit inaintea codului „E0” iar inaintea codului pentru eliberare tasta „E0 F0”.



Comenzile trimise tastaturii sunt:

<table>
<tr><th>Command</th><th>Description</th></tr>
<tr><td>ED</td><td>Turn on/off Num Lock, Caps Lock, and Scroll Lock LEDs. The keyboard acknowledges receipt of an “ED” command by replying with an “FA”, after which the host sends another byte to set LED status. The bit positions for the keyboard LEDs are shown below. Write a ‘1’ to the specific bit to illuminate the associated keyboard LED.
<table><tr><th>7</th><th>6</th><th>5</th><th>4</th><th>3</th><th>2</th><th>1</th><th>0</th></tr><tr><td colspan="5">Ignored</td><td>Caps Lock</td><td>Num Lock</td><td>Scroll Lock</td></tr></table></td></tr>
<tr><td>EE</td><td>Echo. Upon receiving an echo command, the keyboard replies with the same scan code “EE”.</td></tr>
<tr><td>F3</td><td>Set scan code repeat rate. The keyboard acknowledges receipt of an “F3” by returning an “FA”, after which the host sends a second byte to set the repeat rate.</td></tr>
<tr><td>FE</td><td>Resend. Upon receiving a resend command, the keyboard resends the last scan code.</td></tr>
<tr><td>FF</td><td>Reset. Resets the keyboard.</td></tr>
</table>

Tastatura transmite gazdei doar atunci cand ambele semnale (ceas si date) sunt „1” (in starea Idle). Deoarece gazda verifica daca tastatura transmite inainte de a incepe comunicatia, linia de ceas poate fi utilizata ca si semnal „clear to send”. Daca gazda incepe transmisia controland linia de ceas, tastatura nu mai trebuie sa transmita pana cand transmisia a fost terminata.

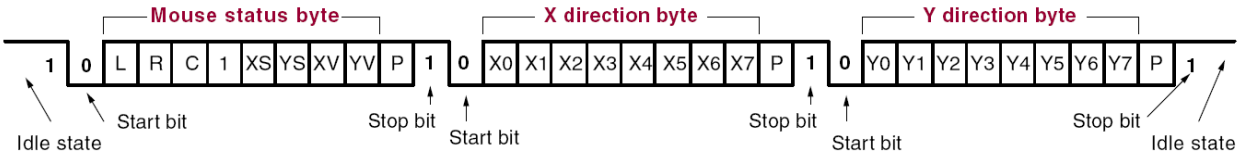
Tastatura trimite cuvinte de 11 biti impachetate astfel: 1 bit de start („0”), 8 biti de date continand codul (LSB...MSB), bitul de paritate (impara) si bitul de stop („1”). Cand tastatura trimite date, ea genereaza 11 perioade de ceas cu o frecventa intre 20 si 30 kHz si datele sunt valide pe frontul negativ al ceasului.

Mouse

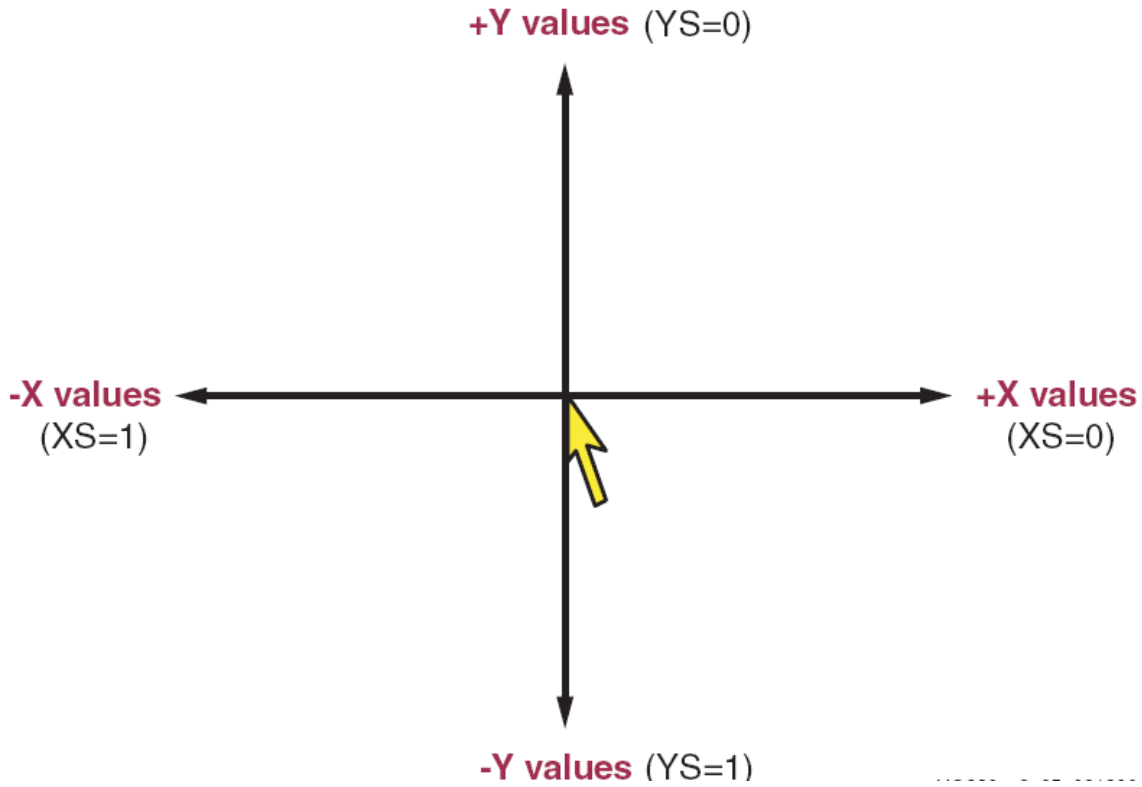
Un mouse compatibil PS/2 suporta doua moduri de comunicatie: interogare (gazda interogheaza mouseul daca s-a inregistrat un eveniment) si streaming (mouseul raporteaza de fiecare data cand s-a produs un eveniment). Implicit este modul streaming.

Pentru a seta modul streaming, FPGA-ul trebuie sa trimita comanda „Set Stream Mode” (0xEA). Ca urmare a receptionarii acestei comenzi, mouseul genereaza apoi semnalul de ceas si cel de date de fiecare data cand se inregistreaza un eveniment (miscare sau apasarea unui buton). Cand nu se transmite nimic cele doua semnale au valoarea „1” si sunt in starea Idle.

De fiecare data cand se transmite un eveniment, mouseul transmite trei pachete de 11 biti cu formatul: 1 bit de start („0”), 8 biti de date (LSB...MSB), un bit de paritate (impara) si un bit de stop („1”). Fiecare transmisie are 33 de biti, avand bitii 0, 11 si 22 „0” (biti de start), bitii 10, 21 si 32 biti de stop („1”). Datele sunt valide pe frontul negativ al ceasului, iar frecventa acestuia este intre 20 si 30 de kHz.



Un mouse PS/2 foloseste coordonate relative (miscarea este codata in functie de pozitia curenta). Daca mouseul se misca spre dreapta se va inregistra o miscare cu o valoare pozitiva pe axa Ox (XS=„0” si masura miscarii este data de valoarea din X7-X0). Daca mouseul se va misca in jos se va inregistra o miscare negativa pe axa Oy (YS=„1” si masura miscarii este data de valoarea din Y7-Y0). Bitii XS si YS sunt transmisi in pachetul de status (primul pachet) si codifica semnul miscarii (valoarea negativa este codata „1”).



Amploarea miscarii pe X sau Y este data de valoarea corespunzatoare. Cu cat aceasta valoare este mai mare cu atat mouseul (cursorul) se va misca mai repede. Bitii XV si YV semnaleaza depasirea valorii maxime ce poate fi reprezentata pe 8 biti (255). Daca mouseul se misca continuu, cei 33 de biti sunt retransmisi la un interval de aproximativ 50ms.

Bitii L, R si C din pachetul de status (primul pachet) corespund butoanelor Left, Right si Center. Valoarea „1” indica faptul ca butonul este apasat.

Implementarea

Pentru a utiliza o parte din interfetele prezentate se propune realizarea unui terminal text ce va permite navigarea cu ajutorul unui cursor ce lasa o urma pe drumul parcurs.

Deoarece rezolutia ce poate fi obtinuta utilizand placa de test (oscilatorul de 50 MHz) este de 800x600 pixeli, iar un caracter ocupa 8x8 pixeli, se propune utilizarea unei memorii in care sa se stocheze drumul parcurs (100 de coloane si 75 de linii). Aceasta memorie va fi o memorie de tip biport pentru a permite accesul pe doua seturi de magistrale. Un port va fi utilizat pentru scriere in memorie iar cel de al doilea pentru citirea si afisarea continutului pe ecran.

Adresa pentru citirea in vederea afisarii se poate calcula pornind de la coordonatele pixelului curent si ignorarea a 3 biti (cei mai putini semnificativi) care realizeaza pozitionarea in cadrul caracterului.

Deplasarea cursorului prin intermediul unei tastaturi se va face cu ajutorul unui modul ce realizeaza comunicatia pe baza protocolului PS/2 si a unui cursor care indica pozitia curenta in memorie (pozitie la care se fac operatiile curente). Marcarea drumului se va face prin scrierea in memorie a unei valori (ce este imterpretata apoi ca si urma).

Limitarile sunt impuse de faptul ca dimensiunea memoriei interne a FPGA-ului este prea mica pentru a stoca mai multi biti pentru fiecare din cele 7500 de celule (100x75).

Laboratorul 4 – Operatii aritmetice in virgula fixa

Obiective

Acest laborator isi propune sa prezinte modul de implementare a operatiilor aritmetice in virgula fixa si a unor algoritmi pentru accelerarea executiei acestor operatii.

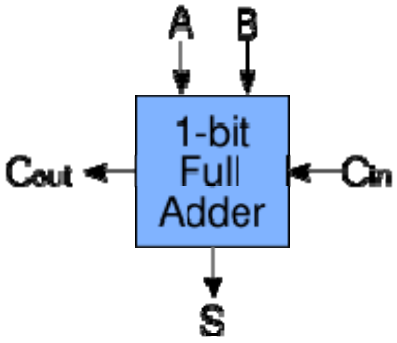
Proiectarea

De obicei, in sistemele numerice, numerele intregi sunt reprezentate binar in complement fata de 2 (numarul de biti variaza in functie cu precizia dorita).

Reprezentarea numerelor in complement fata de 2 prezinta avantajul de a putea decide daca un numar este pozitiv sau negativ analizand bitul cel mai semnificativ. Deasemenea, in cazul in care se doreste schimbarea semnului unui numar este suficient sa se adune o unitate la valoarea negata. Pe baza acestei propietati diferenta dintre doua numere in complement fata de 2 poate fi privita ca suma dintre primul si negatul celui de al doilea (suma cu transport initial „1”).

Sumatorul complet pe un bit

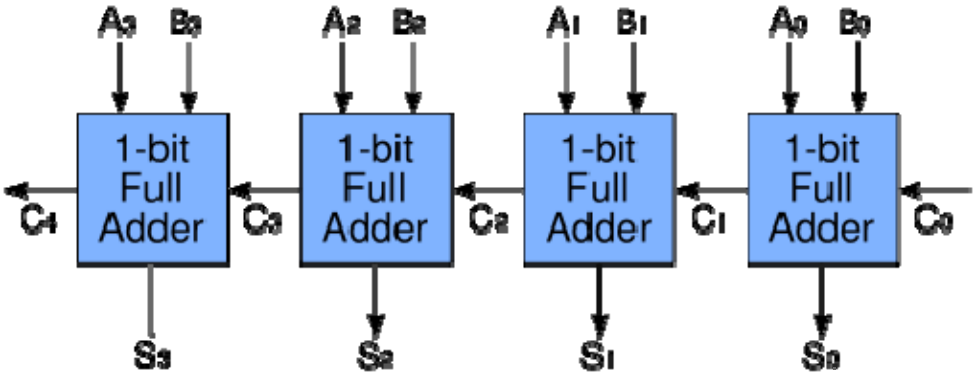
Un sumator complet pentru un singur bit este un circuit combinational cu 3 intrari si 2 iesiri:



Functiile logice ale iesirilor in functie de intrari sunt determinate pe baza tabelului urmator:

A	0	0	0	0	1	1	1	1
B	0	0	1	1	0	0	1	1
Cin	0	1	0	1	0	1	0	1
S	0	1	1	0	1	0	0	1
Cout	0	0	0	1	0	1	1	1

Mai multe astfel de sumatoare elementare pot fi inlantuite pentru a realiza o prelucrare a unor numere pe mai multi biti:



O astfel de arhitectura, desi are o structura extrem de simpla se poate dovedi ineficienta pentru insumarea unor numere mare (reprezentate pe multi biti) deoarece pentru calcularea rangului 2 este nevoie sa se astepte calcularea transportului generat de rangul 1 (si asa mai departe). Intarzierea se poate calcula destul de usor deoarece pentru obtinerea Cout pornind de la Cin sunt necesare doua niveluri de porti logice. Astfel pentru un sumator pe 32 de biti rezultatul este obtinut dupa 31x2+1=63 de niveluri de porti logice (intarzierea sumatorului este de 63 de ori mai mare decat intarzierea datorata unei porti logice elementare).

Sumatorul cu transport anticipat

Pentru a realiza o prelucrare mai rapida se utilizeaza arhitecturi care utilizeaza mai multe resurse in paralel pentru a reduce numarul de niveluri logice (scade numarul de niveluri dar pe fiecare nivel sunt mai multe porti logice).

Sumatorul cu transport anticipat se bazeaza pe generarea a doua semnale intermediare pentru fiecare pereche de biti de intrare. Semnalele sunt P si G (propagare si generare). P este „1” daca nivelul curent propaga transportul de la nivelul anterior, iar G este „1” daca nivelul curent genereaza transport.

Pentru un sumator pe 4 biti avem:

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot C_1$$

$$C_3 = G_2 + P_2 \cdot C_2$$

$$C_4 = G_3 + P_3 \cdot C_3$$

Inlocuind recursiv, in expresiile de mai sus, obtinem:

$$C_1 = G_0 + P_0 \cdot C_0$$

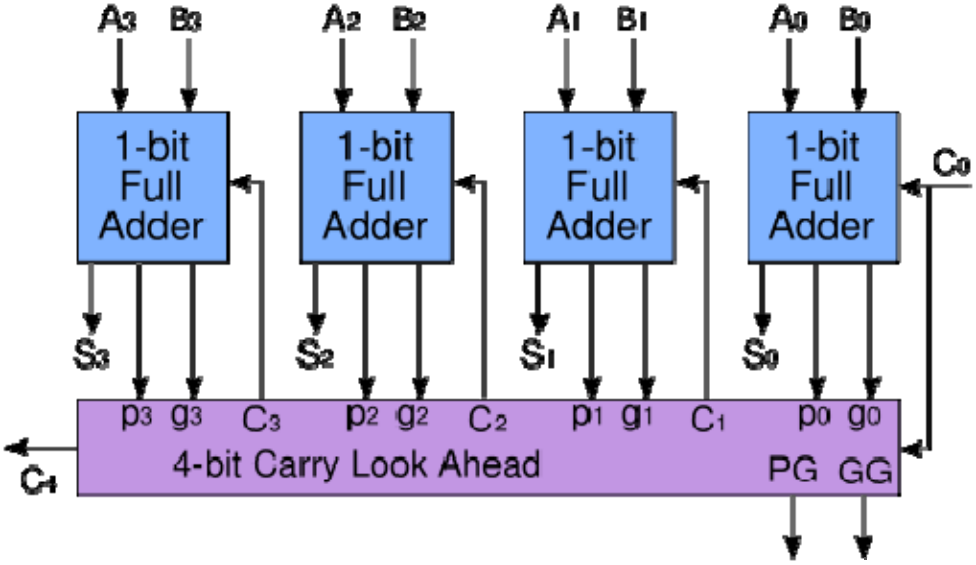
$$C_2 = G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1$$

$$C_3 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + C_0 \cdot P_0 \cdot P_1 \cdot P_2$$

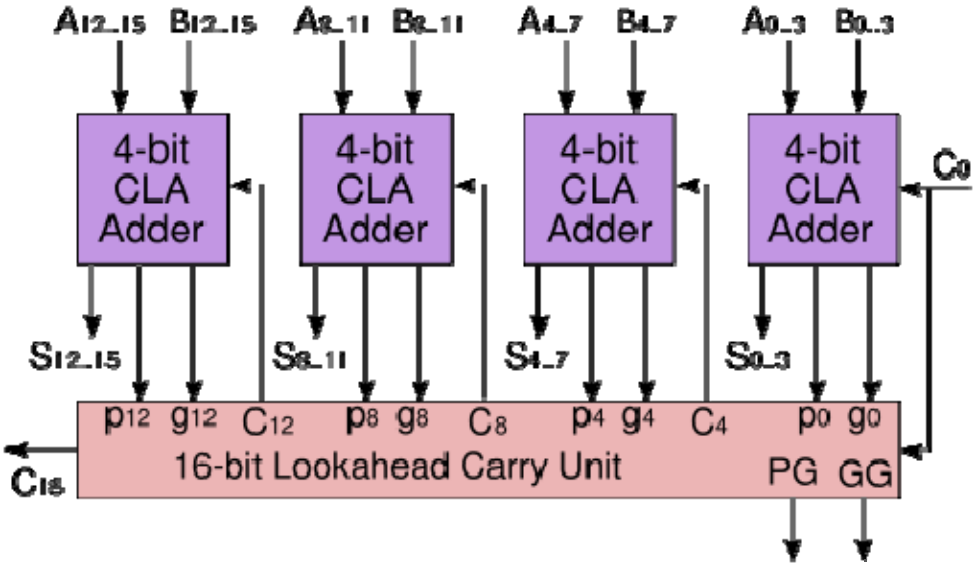
$$C_4 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + C_0 \cdot P_0 \cdot P_1 \cdot P_2 \cdot P_3$$

$$G_i = A_i \cdot B_i \qquad P_i = A_i \oplus B_i$$

Se poate observa ca transportul la orice nivel poate fi determintat exclusiv pe baza P, G si C_0.



Mai multe unitati de acest fel pot fi folosite, utilazand acelasi principiu, pentru a obtine sumatoare de dimensiuni mai mari (pentru numere reprezentate pe mai multi biti):



$$C_4 = G_0 + P_0 \cdot C_0$$

$$C_8 = G_4 + G_0 \cdot P_4 + C_0 \cdot P_0 \cdot P_4$$

$$C_{12} = G_8 + G_4 \cdot P_8 + G_0 \cdot P_4 \cdot P_8 + C_0 \cdot P_0 \cdot P_4 \cdot P_8$$

$$C_{16} = G_{12} + G_8 \cdot P_{12} + G_4 \cdot P_8 \cdot P_{12} + G_0 \cdot P_4 \cdot P_8 \cdot P_{12} + C_0 \cdot P_0 \cdot P_4 \cdot P_8 \cdot P_{12}$$

Calcularea intarzierii unui astfel de sumator este un pic mai dificila la prima vedere:

- Pi si Gi se calculeaza cu 1 nivel logic
- Ci se calculeaza cu 3 niveluri logice
- Calculul transportului de intrare necesita 0, 4 si respectiv 5 si 5 niveluri logice
- Calculul sumelor partiale necesita 4, 7 si respectiv 8 si 8 niveluri logice

Ca urmare a acestor calcule se poate observa ca pentru un sumator pe 16 biti, implementat cu anticiparea transporului, intarzierea este de 8 ori mai mare decat intarzierea datorata unei porti logice (fata de 15x2+1=31 la sumatorul elementar).

Inmultirea

Inmultirea binara este asemanatoare celei zecimale. Ea se realizeaza prin insumarea produselor partiale dintre operandul A si fiecare digit din operandul B. Rezultatul inmultirii are un numar dublu de biti (fata de operanzi). Deoarece fiecare digit din B reprezinta o putere a lui 2, inmultirea este asigurata prin deplasarea la stanga a lui A. Algoritmul prevede urmatorii pasi:

- Se memoreaza cei 2 operanzi
- Pentru ficare digit (bit) din B:
    - Daca e 1 se aduna valoarea lui A (deplasata) la un acumulator (initial 0)
    - Se deplaseaza A la stanga cu o pozitie

Se observa faptul ca pentru a inmulti 2 numere pe N biti este nevoie de N-1 deplasari si (in cel mai defavorabil caz cand toti bitii din B sunt 1) de N adunari.

Algoritmul lui Booth

Pornind de la cazul anterior (cem mai ineficient), se observa ca, pentru un numar care contine o grupare de biti 1 inlantuiti (de exemplu numarul 15 reprezentat pe 4 biti = 1111), inmultirea poate fi privita ca o diferenta de produse partiale cu puteri ale lui 2 (inmultirea cu 15 este de fapt diferenata dintre inmultirea cu 16 si cu 1). Deoarece inmultirea cu o putere a lui 2 este echivalenta cu o deplasare la stanga (cu un numar de biti egali cu puterea), pentru cazul anterior

(al inmultirii unui numar reprezentat pe 4 biti cu 15), in loc de 3 deplasari si 4 adunari este nevoie de o deplasare si 2 adunari (conform celor prezentate anterior, pentru numerele in complement fata de 2, scaderea este echivalenta cu adunarea cu negatul operandului si transport initial 1).

Algoritmul lui Booth foloseste aceasta proprietate si reduce astfel numarul de operatii necesare.

Se analizeaza perechile de biti din B. Daca perechea este „00” sau „11” de trece mai departe (se deplaseaza o copie a lui A si se analizeaza urmatoarea pereche din B). Daca perechea este „01” se adauga prosusul partial corespunzator. Daca perechea este „10” se scade produsul partial corespunzator.

High-Radix

O alta metoda de a creste viteza de prelucrare (folosita mai ales in cazul operatiilor cu numere mari – cum ar fi criptarea RSA) in detrimentul resurselor utilizate, este High-Radix. Aceasta metoda presupune pseudo codificarea numerelor in baze superioare (un numar codat binar poate fi considerat ca fiind codat in hexazecimal daca se grupeaza seturi de 4 biti).

Presupunem inmultirea a doua numere reprezentate pe 32 de biti. Folosind algoritmul clasic va fi nevoie de 31 deplasari la stanga si maxim 32 de adunari.

Daca grupam cei 32 de biti in 8 grupuri de cate 4 (pseudo-reprezentamin hexazecimal), ca trebui sa operam 7 deplasari si maxim 8 adunari. Desigur deplasarile au aceeasi complexitate (numarul de pozitii cu care se face deplasarea este irelevant pentru intarzierea operatiei). Pentru a inmulti 2 grupuri de cate 4 biti avem nevoie insa de o metoda rapida. In acest caz se foloseste un LUT (LookUp Table), care pentru orice combinatie de perechi de 4 biti, intoarce rezultatul inmultirii. Se obtine astfel o intarziere mica (data de logica de selectie) in detrimentul resurselor folosite (LUT-ul poate fi comparat cu o memorie ce are latimea cuvantului de 8 biti si adancimea de 256 de cuvinte).

Impartirea

Impartirea a doua numere reprezentate in binar se face asemanator cu situatia in care ele sunt reprezentate in zecimal. Se incearca sa se scada impartitorul din deimpartit (incepand cu pozitia cea mai semnificativa).

Sa analizam impartirea lui 27 la 5 (rezultatul este in mod evident 5 si rest 2). Reprezentat in binar avem 00011011 impartit la 0101.

- Se incearca sa se scada 0101 din 00011, dar rezultatul este negativ si deci, in acest caz, cel mai semnificativ bit al rezultatului este 0.
- La pasul urmator se incearca sa se scada 0101 din 0110 si se obtine un bit 1 pentru cat si rest 0001

- La pasul urmator se incearca sa se scada 0101 din 0011 si se obtine iar 0
- La pasul urmator se incearca sa se scada 0101 din 0111 si se obtine un bit 1 si rest 0010

Rezultatul final este: Catul 0101 si rest 0010

Implementarea

Utilizand resursele din cadrul laboratorului se urmareste integrarea unei unitati aritmetice (implementate conform algoritmilor prezentati) intr-o interfata ce va permite testarea operatiilor introducand de la tastatura cei doi operanzi si pentru operatia curenta rezultatul va fi afisat pe monitor (impreuna cu cei 2 operanzi pentru a permite verificarea corectitudinii).

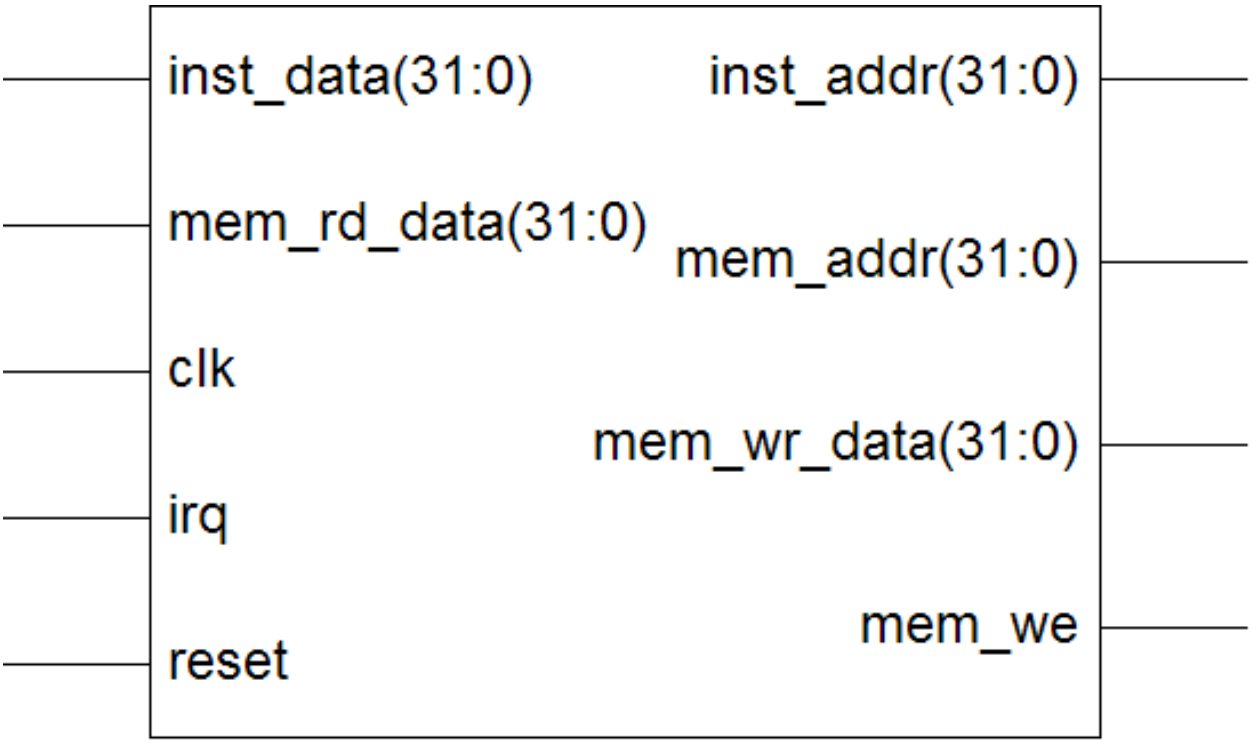
Laboratorul 5 – Implementarea unui System-on-Chip

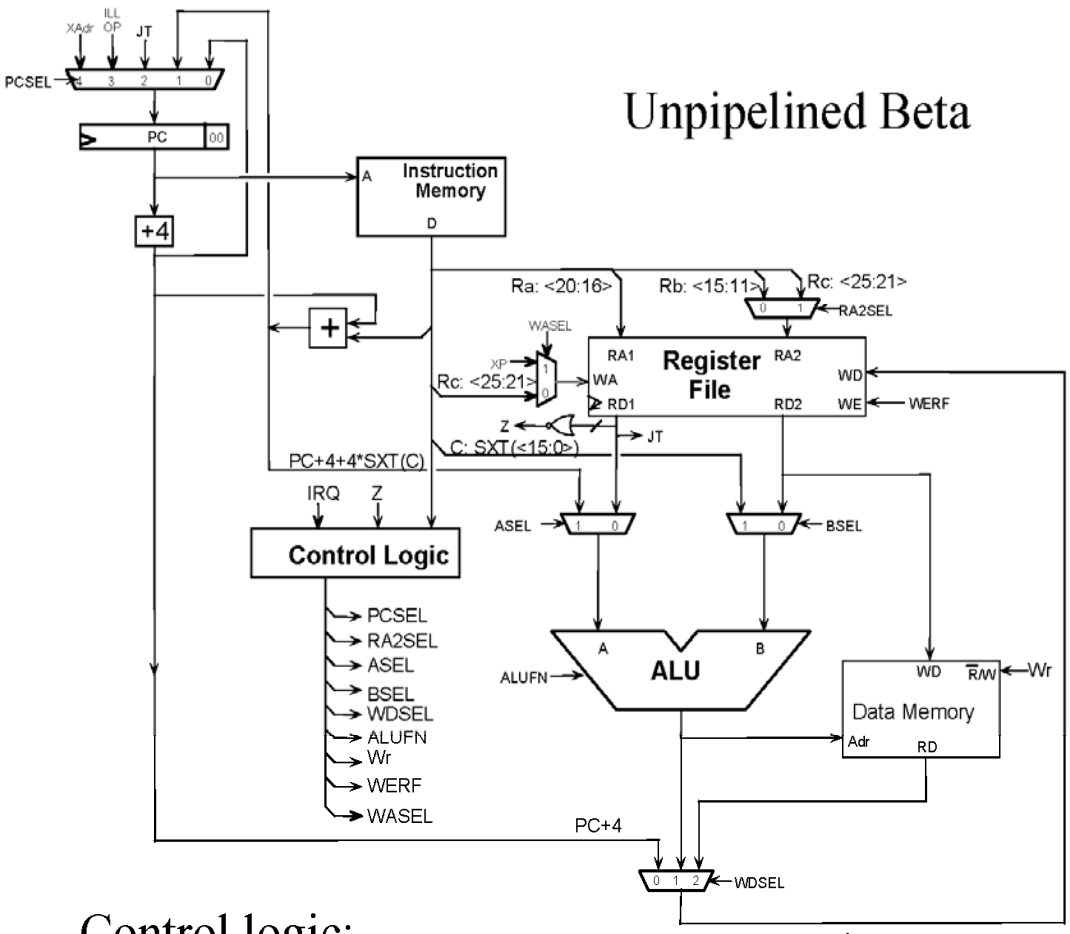
Obiective

Acest laborator isi propune sa prezinte modul de realizare a unui System-on-Chip (SoC). In cadrul laboratorului vor fi prezentate metodele de proiectare a unui sistem folosind elemente constructive deja descrise ce vor fi reutilizate. Scopul acestuia este de a va familiariza cu reutilizarea codului disponibil si cu necesitatile unui sistem de calcul construit cu ajutorul unor componente proiectate independent.
Pentru a obtine punctajul acordat acestui laborator, trebuie sa prezentati laborantului functionarea corecta in hardware a proiectului dumneavoastra.

Proiectarea

Procesorul utilizat se numeste Beta si este un procesor pe 32 de biti cu arhitectura RISC dezvoltat la MIT. Pentru mai multe detalii privind implementarea puteti consulta cursul aferent de la MIT (6.004). Prezentam in continuare interfata acestui procesor si schema bloc:





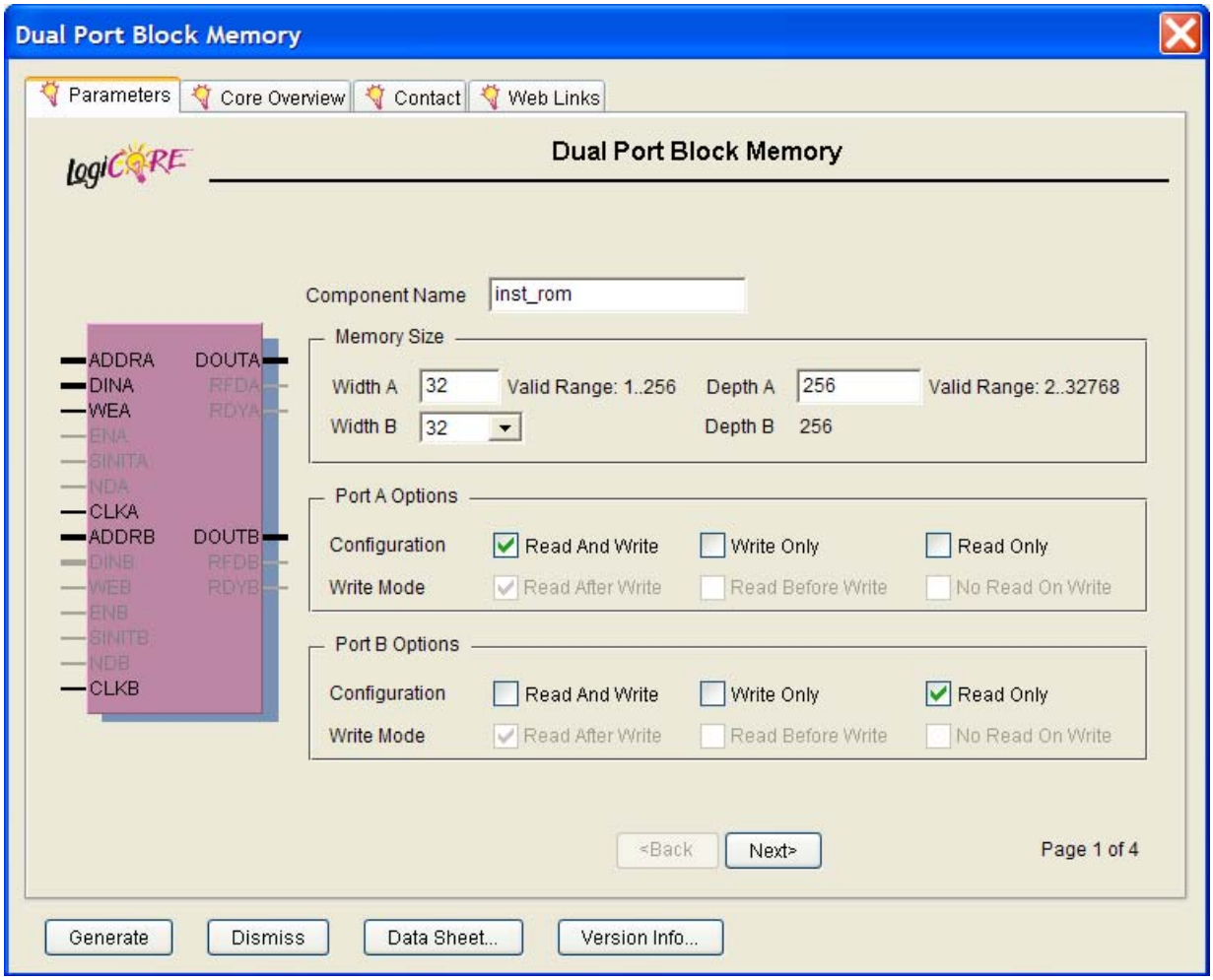
Control logic:

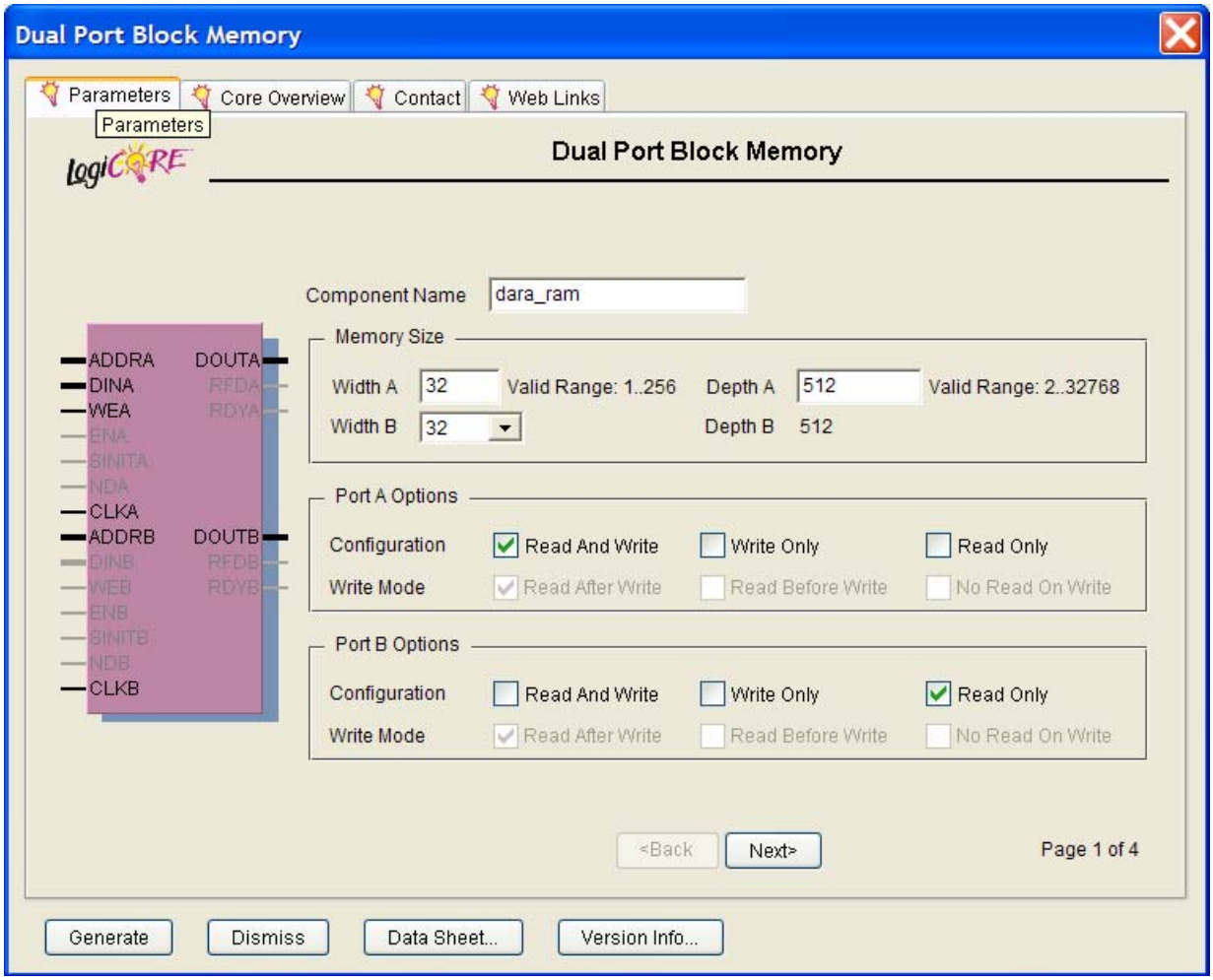
	OP	OPC	LD	ST	JMP	BEQ	BNE	LDR	Illop	IRQ
ALUFN	F(op)	F(op)	"+"	"+"	–	–	–	"A"	–	–
WERF	1	1	1	0	1	1	1	1	1	1
BSEL	0	1	1	1	–	–	–	–	–	–
WDSEL	1	1	2	–	0	0	0	2	0	0
WR	0	0	0	1	0	0	0	0	0	0
RA2SEL	0	–	–	1	–	–	–	–	–	–
PCSEL	0	0	0	0	2	Z ? 1 : 0	Z ? 0 : 1	0	3	4
ASEL	0	0	0	0	–	–	–	1	–	–
WASEL	0	0	0	–	0	0	0	0	1	1

Deoarece ne propunem sa implementam un sistem simplu, nu vom utiliza toate facilitatile oferite de procesorul Beta (nu vom folosi intreruperile si de aceea pe portul irq se va conecta la masa). Pentru a putea analiza efectele executiei instructiunilor de catre procesor, acesta trebuie sa functioneze sincron cu un ceas cu o perioada protrivita. Vom diviza ceasul disponibil cu frecventa de 50MHz (utilizat pentru afisarea VGA) utilizand un contor pe 26 de biti ce se incrementeaza permanent. Bitul cel mai semnificativ al acestui contor va constitui semnalul de ceas pentru procesor. Astfel ceasul este divizat. La procesor se pot conecta doua memorii, una de date si una de instructiuni. Amandoua au latimea cuvantului de 32 de biti. Teoretic, adancimea celor 2 memorii poate fi calculata astfel incat adresa sa fie pe 32 de biti. Deoarece dorim sa afisam in permanenta continutul celor 2 memorii vom face un compromis alegand adancimile de 512 (memoria de date) si 256 (memoria de instructiuni). Aceasta alegere este justificata in paragraful referitor la afisarea memoriilor pe monitor. Din punct de vedere al procesorului memoria de instructiuni este ROM, iar memoria de date este RAM.

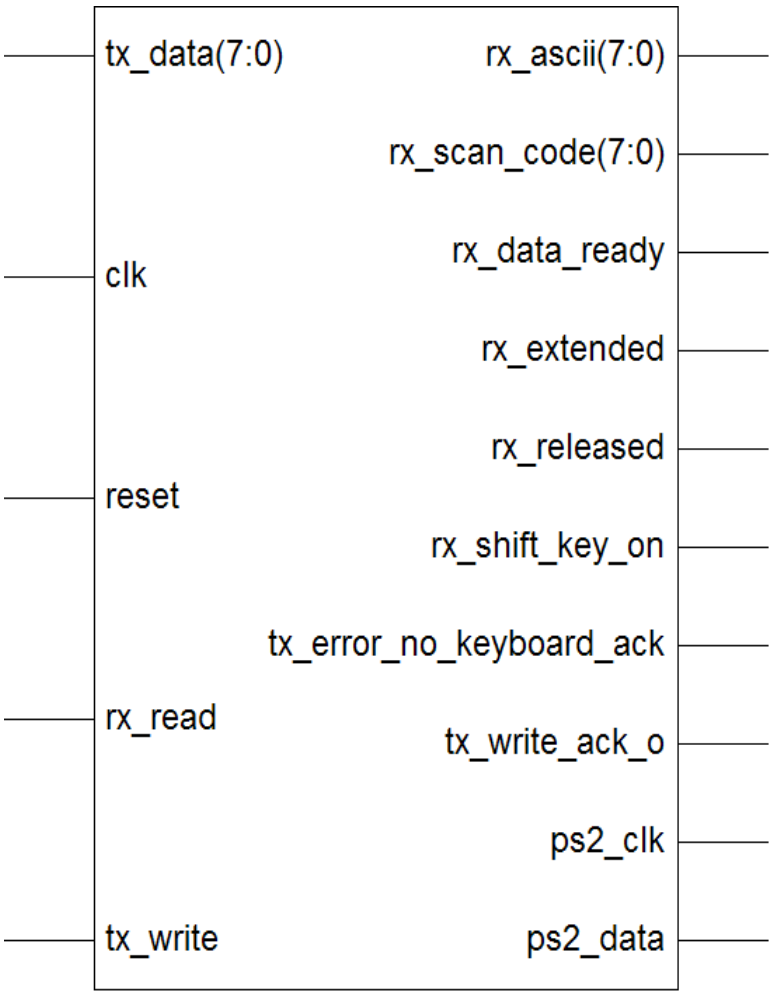
Pentru cele doua memorii vom utiliza IP-uri facilitand astfel utilizarea resurselor specifice disponibile in cadrul FPGA-ului. Deoarece la memorii trebuie sa aiba acces pe de o parte procesorul si pe de alta parte modulul de afisare VGA, vom folosi memorii biport. Desi memoria de instructiuni este din punct de vedere al procesorului o memorie ROM, ea trebuie sa poata fi scrisa inainte de a putea executa programul. Pentru a scrie aceasta memorie vom folosi un mecanism specializat pentru a citi de la tastatura (PS/2) adresa instructiunii si instructiunea ce doreste a fi introdusa in memorie. In concluzie, ambele memorii vor avea portul A de tip RW (in memoria de instructiuni scrie tastatura si citeste procesorul, iar in memoria de date scrie si citeste procesorul), iar portul B va fi de tip RO deoarece modulul de afisare VGA doar citeste continutul memoriilor pentru al afisa.

Atata timp cat se introduc de la tastatura instructiuni in memorie, procesorul nu trebuie sa functioneze si deci magistrala de arese a memoriei de instructiuni (desi partajata) va fi atribuita alternativ in functie de valoarea unui switch extern ce stabileste modul de functionare. Acelasi switch determina si pornirea ceasului procesorului.


Dual Port Block Memory
Parameters
Core Overview
Contact
Web Links
Dual Port Block Memory
Component Name
inst_rom
ADDRA
DINA
WEA
ENA
SINITA
NDA
CLKA
ADDRB
DINB
WEB
ENB
SINITB
NDB
CLKB
DOUTA
RFDA
RDYA
DOUTB
RFDB
RDYB
Memory Size
Width A
32
Valid Range: 1..256
Depth A
256
Valid Range: 2..32768
Width B
32
Depth B
256
Port A Options
Configuration
Read And Write
Write Only
Read Only
Write Mode
Read After Write
Read Before Write
No Read On Write
Port B Options
Configuration
Read And Write
Write Only
Read Only
Write Mode
Read After Write
Read Before Write
No Read On Write
<Back
Next>
Page 1 of 4
Generate
Dismiss
Data Sheet...
Version Info...

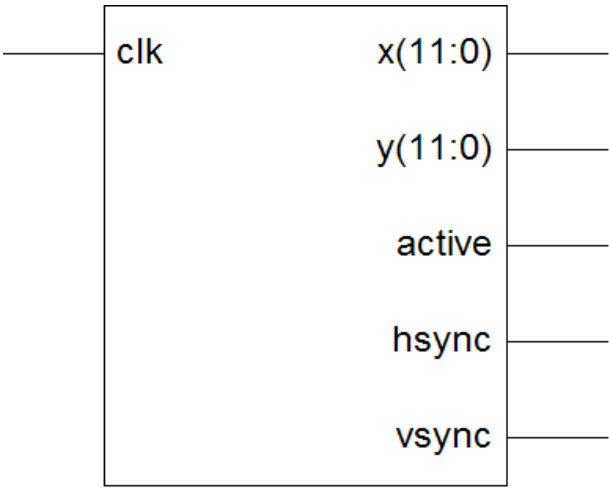

Dual Port Block Memory
Parameters
Core Overview
Contact
Web Links
Parameters
Dual Port Block Memory
Component Name
dara_ram
ADDRA
DINA
WEA
ENA
SINITA
NDA
CLKA
ADDRB
DINB
WEB
ENB
SINITB
NDB
CLKB
DOUTA
RFDA
RDYA
DOUTB
RFDB
RDYB
Memory Size
Width A
32
Valid Range: 1..256
Depth A
512
Valid Range: 2..32768
Width B
32
Depth B
512
Port A Options
Configuration
Read And Write
Write Only
Read Only
Write Mode
Read After Write
Read Before Write
No Read On Write
Port B Options
Configuration
Read And Write
Write Only
Read Only
Write Mode
Read After Write
Read Before Write
No Read On Write
<Back
Next>
Page 1 of 4
Generate
Dismiss
Data Sheet...
Version Info...

Introducerea de la tastatura a instructiunilor in memoria de instructiuni se face prin intermediul unui registru temporar de 40 de biti (primii 8 constituie adresa iar ceilalti 32 instructiunea ce se doreste a fi introdusa). Cand in acest registru se gasesc datele ce se doresc a fi introduse in memorie, cu ajutorul unui buton se transmite comanda de a scrie in memorie (pe busul de adrese se pune adresa, pe busul de intrare de date se pun datele si se activeaza semnalul WE). Astfel la urmatorul front de ceas, datele vor fi scrise in memoria de instructiuni.

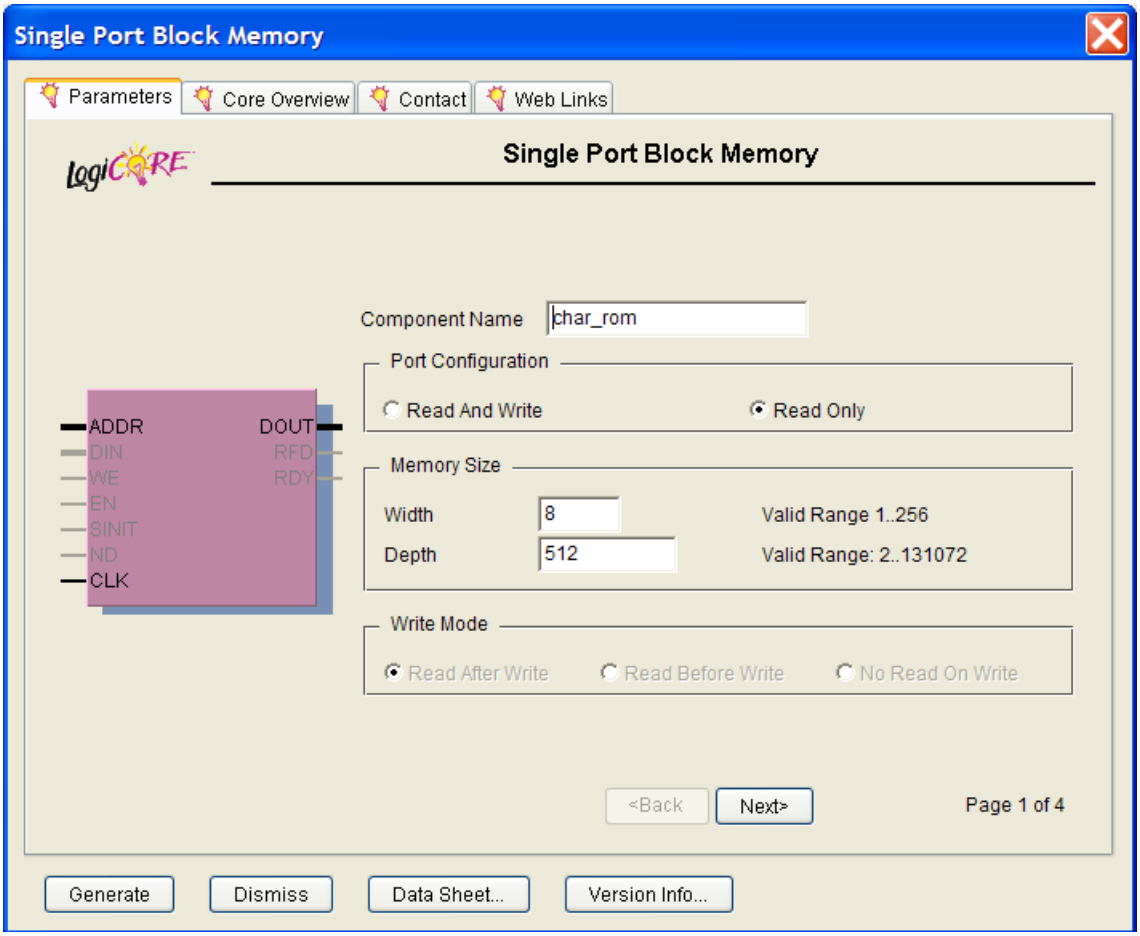


Pentru a citi de la tastatura in registrul temporar se foloseste un controler de tastatura (PS/2) ce poate citi orice tasta. Cand se detecteaza ca s-a citit o tasta aceasta trebuie validata si codul trebuie transformat astfel incat se se accepte numai urmatoarele valori (reprezentabile pe 4 biti): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E si F. Aceste operatii se fac cu ajutorul uinui LookUpTable. Pe masura ce se introduce un caracter, in cadrul registrului temporar se deplaseaza la stanga cu patru pozitii valoarea anterioara si se introduce pe pozitiile cel mai putin semnificative noul cod al caracterului.

Afisarea continutului memoriilor cat si a registrului temporar in care se introduce de la tastatura pe monitor se realizeaza cu ajutorul unui controler VGA care genereaza semanalele de sincronizare necesare si intoarce coordonatele pixelului curent:



Pentru a afisa caractere, avem nevoie de o memorie ROM care sa contina harta caracterelor:



Caracterele sunt reprezentate in matrici de 8x8 pixeli, iar fiecare linie constituie un cuvant in memoria de caractere. Pentru a extrage valoarea pixelului curent, pe baza pozitiei date de X si Y stabilim ce caracter este si ce pixel din matricea caracterului ne intereseaza. Cei mai putin semnificatifi 3 biti din X respectiv Y ne vor spune care pixel din caracterul dorit trebuie afisat. Astfel putem stabili culoarea pixelului curent.

Pentru a putea afisa continutul celor doua memorii am facut compromisul de a alege dimensiunile lor de 256 si respectiv 512 cuvinte. Un cuvant are 32 de biti ce se reprezinta cu ajutorul a 8 caractere (fiecare caracter se poate reprezentat pe 4 biti). In concluzie, un cuvant ocupa 64 de pixeli pe orizontala si 8 pixeli pe verticala. Care caracter din cadrul cuvantului este desemnat de valoarea X[5:3]. Organizam afisarea dupa cum urmeaza: 4 coloane de 64 de biti pe orizontala si cate 64 de cuvinte de 8 biti pe verticala (memoria de instructiuni 4x64=256 cuvinte) si 8 coloane de 64 de biti pe orizontala di cate 64 de cuvinte a 8 biti pe verticala (memoria de date 8x64=512 cuvinte). Reprezentarea memoriilor ocupa 768 de pixeli pe orizontala si 512 pixeli pe verticala. Deoarece rezolutia de afisare corespunzatoare unui ceas de 50MHz este de 800x600 pixeli, ne ramane o zona in partea de jos a ecranului pe care o putem folosi pentru afisarea registrului temporar.

Coloana in care ne aflam se calculeaza pe baza bitilor superiori din X (X[8:6]), iar linia curenta se calculeaza pe baza bitilor superiori din Y (Y[8:3]). Daca afisam din memoria de instructiuni sau din memoria de date se decide pe baza conditiei (X<256). Daca Y[9] este 1 atunci inseamna ca am trecut de limita pe verticala de 512 pixeli si nu mai trebuie sa afisam din continutul memoriei.

Adresele pentru memorii se calculeaza astfel: pentru memoria de instructiuni {X[7:6],Y[8:3]}, iar pentru memoria de date {tmp,X[7:6],Y[8:3]} unde tmp este 1 daca X[9:8]este 2 si 0 altfel.

Utilizarea sistemului

Pentru a utiliza sistemul astfel creat, se scriu programe in cod masina pentru procesorul Beta utilizand documentatia aferenta cursului de la MIT. Programele se introduc in memoria de instructiuni incepand cu adresa 0 la adrese succesive si se executa dupa pornirea procesorului (si trecere din modul de introducere de date in modul executie).

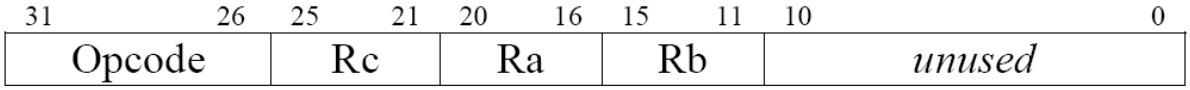
Codificarea instructiunilor

Fiecare instructiune este reprezentata pe 32 de biti. Memoria de date este accesata prin instructiuni de load si store. Instructiunile de ramificare sunt separate de intructiunile de

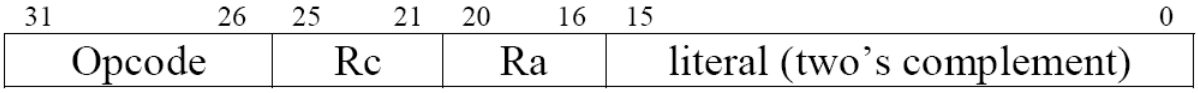
conditionare. Instructiunile de ramificare testeaza valoarea unui registru care poate fi rezultatul unei comparatii anterioare.

Exista doar doua tipuri de instructiuni: Cu Literal si Fara Literal. Instructiunile fara literal sunt instructiunile aritmetice si logice ce au operanzii situati in doua registre si rezultatul este depus in al treilea registru. Instructiunile cu literal sunt toate celelalte instructiuni. Ca si orice alta valoare cu semn in cadrul procesorului Beta, literalul este reprezentat in complement fata de 2.

Fara Literal



Cu Literal



In continuare se prezinta tabelul instructiunilor si codurile operatiilor aferente:

Mnemonic	*Opcode*
ADD	0x20
ADDC	0x30
AND	0x28
ANDC	0x38
BEQ	0x1D
BNE	0x1E
CMPEQ	0x24
CMPEQC	0x34

Mnemonic	*Opcode*
CMPLE	0x26
CMPLEC	0x36
CMPLT	0x25
CMPLTC	0x35
DIV	0x23
DIVC	0x33
JMP	0x1B
LD	0x18

Mnemonic	*Opcode*
LDR	0x1F
MUL	0x22
MULC	0x32
OR	0x29
ORC	0x39
SHL	0x2C
SHLC	0x3C
SHR	0x2D

Mnemonic	*Opcode*
SHRC	0x3D
SRA	0x2E
SRAC	0x3E
SUB	0x21
SUBC	0x31
ST	0x19
XOR	0x2A
XORC	0x3A

Operatiile exacutate sunt prezentate in continuare pentru fiecare instructiune in parte (in ordine alfabetica are mnemonicii):

Usage: ADD(Ra,Rb,Rc)



Operation: PC ← PC + 4

Reg[Rc] ← Reg[Ra] + Reg[Rb]

Usage: ADDC(Ra,literal,Rc)

Opcode:	110000	Rc	Ra	literal

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] + SEXT(literal)

Usage: AND(Ra,Rb,Rc)

Opcode:	101000	Rc	Ra	Rb	*unused*

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] & Reg[Rb]

Usage: ANDC(Ra,*literal*,Rc)

Opcode:	111000	Rc	Ra	literal

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] & SEXT(literal)

Usage: BEQ(Ra,label,Rc)
BF(Ra,label,Rc)

Opcode:	011101	Rc	Ra	literal

Operation:
literal = ((OFFSET(label) – OFFSET(current instruction)) / 4)–1
PC ← PC + 4
EA ← PC + 4*SEXT(literal)
TEMP ← Reg[Ra]
Reg[Rc] ← PC
if TEMP = 0 then PC ← EA

Usage: BNE(Ra,label,Rc)
BT(Ra,label,Rc)

Opcode:	011110	Rc	Ra	literal

Operation:
literal = ((OFFSET(label) – OFFSET(current instruction)) ÷ 4)–1
PC ← PC + 4
EA ← PC + 4*SEXT(literal)
TEMP ← Reg[Ra]
Reg[Rc] ← PC
if TEMP ≠ 0 then PC ← EA

Usage: CMPEQ(Ra,Rb,Rc)

Opcode:

100100	Rc	Ra	Rb	*unused*

Operation: PC ← PC + 4

if Reg[Ra] = Reg[Rb] then Reg[Rc] ← 1 else Reg[Rc] ← 0

Usage: CMPEQC(Ra,literal,Rc)

Opcode:

110100	Rc	Ra	literal

Operation: PC ← PC + 4

if Reg[Ra] = SEXT(literal) then Reg[Rc] ← 1 else Reg[Rc] ← 0

Usage: CMPLE(Ra,Rb,Rc)

Opcode:

100110	Rc	Ra	Rb	*unused*

Operation: PC ← PC + 4

if Reg[Ra] ≤ Reg[Rb] then Reg[Rc] ← 1 else Reg[Rc] ← 0

Usage: CMPLEC(Ra,literal,Rc)

Opcode:

110110	Rc	Ra	literal

Operation: PC ← PC + 4

if Reg[Ra] ≤ SEXT(literal) then Reg[Rc] ← 1 else Reg[Rc] ← 0

Usage: CMPLT(Ra,Rb,Rc)

Opcode:

100101	Rc	Ra	Rb	*unused*

Operation: PC ← PC + 4

if Reg[Ra] < Reg[Rb] then Reg[Rc] ← 1 else Reg[Rc] ← 0

Usage: CMPLTC(Ra,literal,Rc)

Opcode:

110101	Rc	Ra	literal

Operation: PC ← PC + 4

if Reg[Ra] < SEXT(literal) then Reg[Rc] ← 1 else Reg[Rc] ← 0

Usage: DIV(Ra,Rb,Rc)

Opcode:

100011	Rc	Ra	Rb	*unused*

Operation: PC ← PC + 4

Reg[Rc] ← Reg[Ra] / Reg[Rb]

Usage: DIVC(Ra,literal,Rc)

Opcode:

110011	Rc	Ra	literal

Operation:

PC ← PC + 4

Reg[Rc] ← Reg[Ra] / SEXT(literal)

Usage: JMP(Ra,Rc)

Opcode:

011011	Rc	Ra	0000000000000000

Operation:

PC ← PC+4

EA ← Reg[Ra] & 0xFFFFFFFC

Reg[Rc] ← PC

PC ← EA

Usage: LD(Ra,literal,Rc)

Opcode:

011000	Rc	Ra	literal

Operation:

PC ← PC+4

EA ← Reg[Ra] + SEXT(literal)

Reg[Rc] ← Mem[EA]

Usage: LDR(label,Rc)

Opcode:

011111	Rc	11111	literal

Operation:

literal = ((OFFSET(label) – OFFSET(current instruction)) / 4)–1

PC ← PC + 4

EA ← PC + 4*SEXT(literal)

Reg[Rc] ← Mem[EA]

Usage: MUL(Ra,Rb,Rc)

Opcode:

100010	Rc	Ra	Rb	*unused*

Operation:

PC ← PC + 4

Reg[Rc] ← Reg[Ra] * Reg[Rb]

Usage: MULC(Ra,literal,Rc)

Opcode:

110010	Rc	Ra	literal

Operation:

PC ← PC + 4

Reg[Rc] ← Reg[Ra] * SEXT(literal)

Usage: OR(Ra,Rb,Rc)

101001	Rc	Ra	Rb	*unused*

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] | Reg[Rb]

Usage: ORC(Ra,*literal*,Rc)

111001	Rc	Ra	literal

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] | SEXT(literal)

Usage: SHL(Ra,Rb,Rc)

101100	Rc	Ra	Rb	*unused*

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] << $\text{Reg[Rb]}_{4:0}$

Usage: SHLC(Ra,*literal*,Rc)

111100	Rc	Ra	literal

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] << $\text{literal}_{4:0}$

Usage: SHR(Ra,Rb,Rc)

101101	Rc	Ra	Rb	*unused*

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] >> $\text{Reg[Rb]}_{4:0}$

Usage: SHRC(Ra,*literal*,Rc)

111101	Rc	Ra	literal

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] >> $\text{literal}_{4:0}$

Usage: SRA(Ra,Rb,Rc)

101110	Rc	Ra	Rb	*unused*

Operation:
PC ← PC + 4
Reg[Rc] ← Reg[Ra] >> $\text{Reg[Rb]}_{4:0}$

Usage: SRAC(Ra,*literal*,Rc)

Opcode:

111110	Rc	Ra	literal

Operation:

PC ← PC + 4

Reg[Rc] ← Reg[Ra] >> $literal_{4:0}$

Usage: ST(Rc,literal,Ra)

Opcode:

011001	Rc	Ra	literal

Operation:

PC ← PC+4

EA ← Reg[Ra] + SEXT(literal)

Mem[EA] ← Reg[Rc]

Usage: SUB(Ra,Rb,Rc)

Opcode:

100001	Rc	Ra	Rb	*unused*

Operation:

PC ← PC + 4

Reg[Rc] ← Reg[Ra] – Reg[Rb]

Usage: SUBC(Ra,literal,Rc)

Opcode:

110001	Rc	Ra	literal

Operation:

PC ← PC + 4

Reg[Rc] ← Reg[Ra] – SEXT(literal)

Usage: XOR(Ra,Rb,Rc)

Opcode:

101010	Rc	Ra	Rb	*unused*

Operation:

PC ← PC + 4

Reg[Rc] ← Reg[Ra] ^ Reg[Rb]

Usage: XORC(Ra,*literal*,Rc)

Opcode:

111010	Rc	Ra	literal

Operation:

PC ← PC + 4

Reg[Rc] ← Reg[Ra] ^ SEXT(literal)

Un exemplu de implementare este prezentat in continuare:

```
module SoC_top(clk, clr, store, sw, ps_data, ps_clk, hsync, vsync, r2, r1, r0, g2, g1, g0, b2, b1, b0, led, cled);
    input clk;
    input clr;
    input store;
    input sw;
    inout ps_data;
    inout ps_clk;
    output hsync;
    output vsync;
    output r2, r1, r0, g2, g1, g0, b2, b1, b0;
    output [7:0] led;
    output cled;

    reg r2, r1, r0, g2, g1, g0, b2, b1, b0;
    wire active;
    wire [11:0] x;
    wire [11:0] y;
            wire [7:0] led;
            wire cled;
            reg [25:0] cnt;

            wire [0:31] display_word;
            wire [31:0] douti;
            wire [7:0]  addri;
            wire [0:3]          display_byte,display;
            wire [2:0]  dtx,dty;
            wire [2:0]          tmp;
```

```
reg bclk,reset,irq=1'b0;

wire [31:0] inst_addr;
wire [31:0] inst_data;
wire [31:0] mem_addr;
wire [31:0] mem_wr_data;
wire [31:0] mem_rd_data;
wire mem_we;
wire [8:0]  video_rd_addr;
wire [31:0] video_rd_data;

wire [8:0]  addrc;
wire [0:7] doutc;
wire bord,mem_display,ps_display,current_instr,current_data,ind;
wire new_scancode,new_data;
wire [4:0] dsb0,dsb1,dsb2,dsb3;

wire [7:0] scancode;
wire [3:0] data_from_ps2;

reg [3:0] ps_instr [9:0];
reg [3:0] ps_pointer;
reg [3:0] char_pointer;
reg saved;
reg wea,instr_clk;
reg [7:0] addra;
reg [31:0] dina;
```

```
reg rx_read;

wire rx_extended,rx_released,rx_shift_key_on,tx_write_ack_o,tx_error_no_keyboard_ack;
wire [7:0] rx_ascii;

VGATimer #(800,64,120,56, 600,23,6,37, 1) timer (clk, hsync, vsync, active, x, y);

char_rom char(addrc,~clk,doutc);

scancod2cod code(scancode,rx_released,data_from_ps2,new_data);
ps2_keyboard_interface
ps2(clk,~clr,ps_clk,ps_data,rx_extended,rx_released,rx_shift_key_on,scancode,rx_ascii,new_scancode,rx_read,8'd0,
1'b0,tx_write_ack_o,tx_error_no_keyboard_ack);

inst_rom inst(addra,addri,instr_clk,~clk,dina,inst_data,douti,wea);
dara_ram
data(mem_addr[10:2],video_rd_addr,bclk,~clk,mem_wr_data,mem_rd_data,video_rd_data,mem_we);

beta beta(bclk,reset,irq,inst_addr,inst_data,mem_addr,mem_rd_data,mem_we,mem_wr_data);

always @(posedge clk)
  rx_read <= (new_data)? saved: ~saved;

always @(negedge clr or posedge bclk)
        if (~clr) reset <= 1;
        else reset <= 0;

always @(negedge clr or posedge clk)
        if (~clr) cnt <= 0;
```

```
            else cnt <= cnt+1;


        always @(negedge clr or posedge clk)
            if (~clr) bclk <= 0;
            else if (sw) bclk <= cnt[25];
            else bclk <= 0;


        always @(posedge clk)
          if (new_data) begin
             if (~saved) begin
              saved <= 1;
              ps_instr[ps_pointer] <= data_from_ps2;
              if (ps_pointer<9) ps_pointer <= ps_pointer+1;
              else ps_pointer <= 0;
             end
            end else saved <= 0;


        always @(posedge clk)
            if (store&~sw) begin
              dina                                                                                    <=
{ps_instr[2],ps_instr[3],ps_instr[4],ps_instr[5],ps_instr[6],ps_instr[7],ps_instr[8],ps_instr[9]};
              addra <= {ps_instr[0],ps_instr[1]};
              wea <= 1;
              instr_clk <= cnt[25];
            end else begin
              addra <= inst_addr[9:2];
              wea <= 0;
              instr_clk <= cnt[25];
```

```
            end

assign addrc = {2'b11,display,dty};

assign addri={x[7:6],y[8:3]};
assign tmp=(x[9:8]==2'b10) ? 1'b1 : 1'b0;
assign video_rd_addr={tmp,x[7:6],y[8:3]};
assign ind = (x<256);
assign display_word=(ind)? douti:video_rd_data;

assign dsb0={x[5:3],2'b00};
assign dsb1={x[5:3],2'b01};
assign dsb2={x[5:3],2'b10};
assign dsb3={x[5:3],2'b11};

assign display_byte={display_word[dsb0],display_word[dsb1],display_word[dsb2],display_word[dsb3]};
assign display= (y[9])? ps_instr[char_pointer]:display_byte;
assign dtx=x[2:0];
assign dty=y[2:0];
assign bord=((x==0)|(x==799)|(x[5:0]==6'b000000)&(y<512)|(y==0)|(y==599)|(y==511));
assign mem_display=(y<=511)&(x<767);
assign ps_display=(y[9])&(ind)&(y[5:3]==3'b101)&((x[8:3]>10)&(x[8:3]<13)|(x[8:3]>15)&(x[8:3]<24));
assign current_instr=(addri==inst_addr[9:2])&(ind);
assign current_data=(video_rd_addr==mem_addr[10:2])&(~ind);

assign cled=cnt[25];//new_data;

always @(posedge clk)
```

```
begin
 case (x[8:3])
                6'd11 : char_pointer <= 4'd0;
                6'd12 : char_pointer <= 4'd1;
                6'd16 : char_pointer <= 4'd2;
                6'd17 : char_pointer <= 4'd3;
                6'd18 : char_pointer <= 4'd4;
                6'd19 : char_pointer <= 4'd5;
                6'd20 : char_pointer <= 4'd6;
                6'd21 : char_pointer <= 4'd7;
                6'd22 : char_pointer <= 4'd8;
                6'd23 : char_pointer <= 4'd9;
                default : char_pointer <= 4'd0;
        endcase
   r2                                                
```

```
                b1                                                                                                  <=
active&&((ps_display&&doutc[dtx])||(mem_display&&doutc[dtx]&&ind&&~current_instr&&~current_data)||bord
);

            b0                                                                                                      <=
active&&((ps_display&&doutc[dtx])||(mem_display&&doutc[dtx]&&(x[6]^^ind)&&~current_instr&&~current_dat
a)||bord);

              end

endmodule
```

Buna dimineata! <3 S-a trezit dulceata!! Baftix la examen! :)))

Va urez problema usoara clan

Teste de curs

- Registrul Program Counter genereaza adresa instructiunii curente din memorie.

R: Adevărat

- Unitatea de comanda este responsabila pentru generarea adresei instructiunii urmatoare din program.

R: Fals

- Ce numar va fi afisat la finalul executiei urmatorului program SAP-1?

0H: LDA 5H

1H: ADD 7H

2H: SUB 6H

3H: OUT

4H: HLT

5H: 0x03

6H: 0x02

7H: 0x05

R: 0x06

La ce foloseste magistrala W in arhitectura calculatorului SAP-1?

R: Magistrala W este utilizată în cazul instrucțiunilor Enable și Load.

De ce este necesara secventierea instructiunilor pentru calculatorul SAP-1 in microinstructiuni?

R: Arhitectura CISC presupune secventierea executiei instructiunilor in mai multi pasi.

- Orice procesor va secventia executia instructiunilor in 6 pasi (T1, T2, .. T6).

R: Fals

- Numarul -1.5 este reprezentat in IEE754 (aka virgula mobila) pe 32 ca:

R: 0xbfc00000

- Comparatia de mai jos este adevarata pentru orice valoare a variabilei f, declarata initial ca float:

f == (float)(double) f

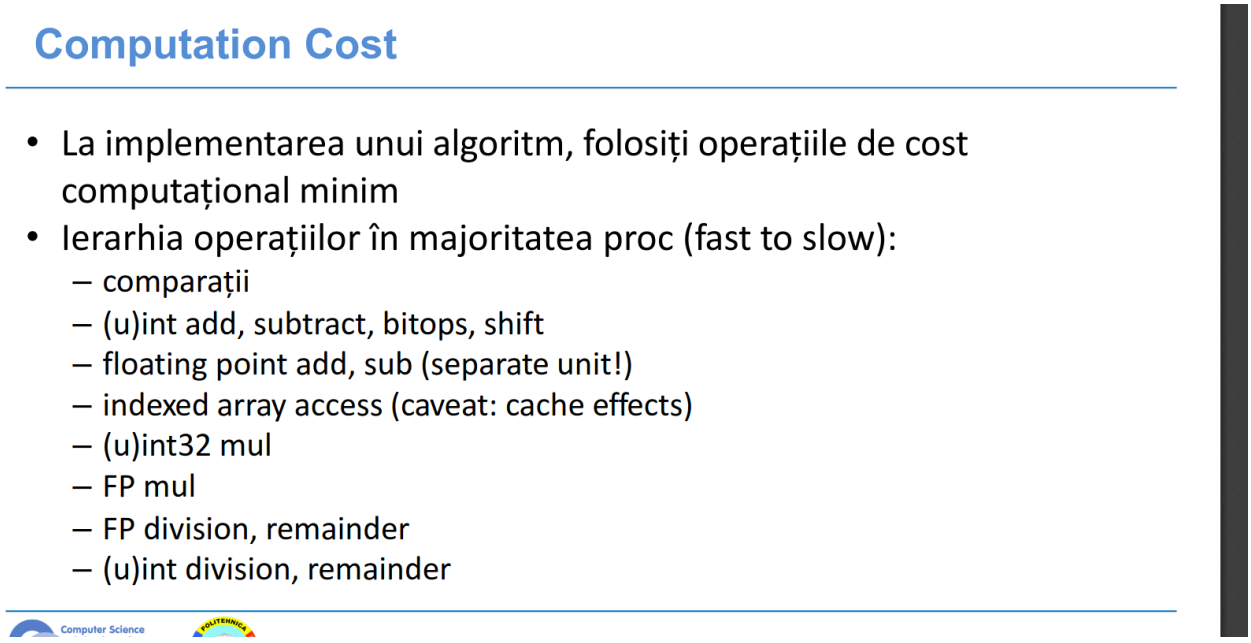
R: Adevărat

- Functia divide1() se executa in medie mai rapid decat divide2() pe o arhitectura x86.

```
uint32_t divide1(uint32_t a, uint32_t b) {
    return a / b;
}
uint32_t divide2(uint32_t a, uint32_t b) {
   double da = (double) a;
   double db = (double) b;
   double q = da/db;
   return (uint32_t) q;
}
```

R: Fals

Motivatie: impartirea pe inturi aparent dureaza mai mult decat cea pe double. Vezi curs daca mori de curiozitate. Vezi si poza de mai jos:



2/3 == 2/3.0

R: Fals

- Pentru orice valori ale lui double d, urmatoarea expresie este adevarata: d == (float) d

R: Fals

- O instructiune cu operand imediat are prioritate in executie fata de restul instructiunilor.

R: Fals

- O arhitectura RISC executa majoritatea instructiunilor intr-un ciclu de ceas.

R: Adevărat

- Unitatea de comanda a unui procesor in arhitectura CISC este mult mai complexa decat unitatea de comanda a unui procesor in arhitectura RISC.

R: Adevărat

- Un procesor petrece 50% din timp in operatii cu memoria. Care este speedup-ul obtinut daca timpul de acces la memorie este imbunatatit de 4 ori?

R: 1 / (0.5 + 0.5/4) = 1/ (0.5 + 0.125) = 1 / 0.625 = 1.6

- Un program contine 30% operatii cu intregi, 20% operatii floating point si 40% operatii cu memoria. Care este speedup-ul obtinut daca masina pe care ruleaza acest program primeste un upgrade al procesorului ce-i permite executia de 3x mai rapida a operatiilor pe intregi si de 2x al operatiilor in flp, dar acest lucru duce cu sine la scaderea de 2x a timpilor de acces la memorie.

R: 1 / (0.3 * ⅓ + 0.2 * ½ + 2 * 0,4 + 0,1) = 1/ (0.1 + 0.1 + 0.8 + 0.1) = 10/11 = 0,91

- Un program contine 20% instructiuni cu intregi ce se executa in 1 ciclu de ceas, 30% instructiuni in virgula mobila ce se executa in 3 cicli de ceas si 10% instructiuni conditionale ce necesita 2 cicli de ceas pentru executie. Restul instructiunilor se executa in 1 ciclu de ceas. Care este CPI-ul mediu?

CPI_mediu = 0.2 * 1 + 0.3 * 3 + 0.1 * 2 + 0.4 * 1.

R: 1,7

- Un program contine doua clase de instructiuni A si B si este rulat pe doua masini de calcul diferite M1 si M2.

Pe M1, clasa A are un CPI de 2 si clasa B un CPI de 3.

Pe M2, clasa A are un CPI de 3 si clasa B un CPI de 1.

Care este procentajul instructiunilor de clasa A in program daca CPI-ul mediu in urma rularii pe masina M2 este de 2 ori mai mic decat CPI-ul mediu in urma rularii programului pe masina M1?

Motivatie:

Fie fA = fractiunea din program care are instructiuni de tip A.

CPI = cycles per instruction.

Timp A: 2fA + 3fB = 2fA + 3(1 - fA) = 3 - fA.

Timp B: 3fA + 1 - fA = 2Fa + 1.

*2 * Timp M2 = Timp M1*

2 * fA + 3 * fB = 2 * fA + 3 * (1 - fA) = 3 – fA (pt M1)

3 * fA + 1 * fB = 3 * fA + 1 – fA = 2 * fA + 1 (pt M2)

(2 * fA + 1) * 2 = 3 – fA

4 * fA + 2 = 3 – fA

fA = 0.2 = 20 %

R: 20

- Procesoarele CISC disipa in general mai putina caldura decat procesoarele RISC.

R: Fals

- Un set de instructiuni RISC va tinde in medie sa minimizeze numarul de instructiuni dintr-un program.

Motivatie: A RISC instruction set normally has a fixed instruction length (often **4 bytes** = 32 bits), whereas a typical CISC instruction set may have instructions of widely varying length (1 to **15 bytes** for x86).

R: Fals

- In etapa de Write Back sunt scrise in memoria RAM rezultatele instructiunii curente.

Motivatie: Write Back scrie in registrii

R: Fals

- Instructiunile Load si Store pentru RISC V presupun calculul adresei prin adunarea unui deplasament la adresa de baza.

R: Adevărat

- Un procesor in b.a. executa instructiuni intr-un singur ciclu de ceas, dar dupa fiecare instructiune va insera doua bule in banda de asamblare. Care este CPI efectiv?

R: 3

- Banda de asamblare mareste performanta prin micsorarea numarului de cicli pe instructiune.

M : banda de asamblare micsoreaza timpi per ciclu.

R: Fals

- Un hazard structural poate fi rezolvat prin bypassing.

R: Fals

- Hazardurile de control se produc doar in cazul instructiunilor de salt conditional.

M: hazardurile de control se produc si la salturi neconditionate

R: Fals

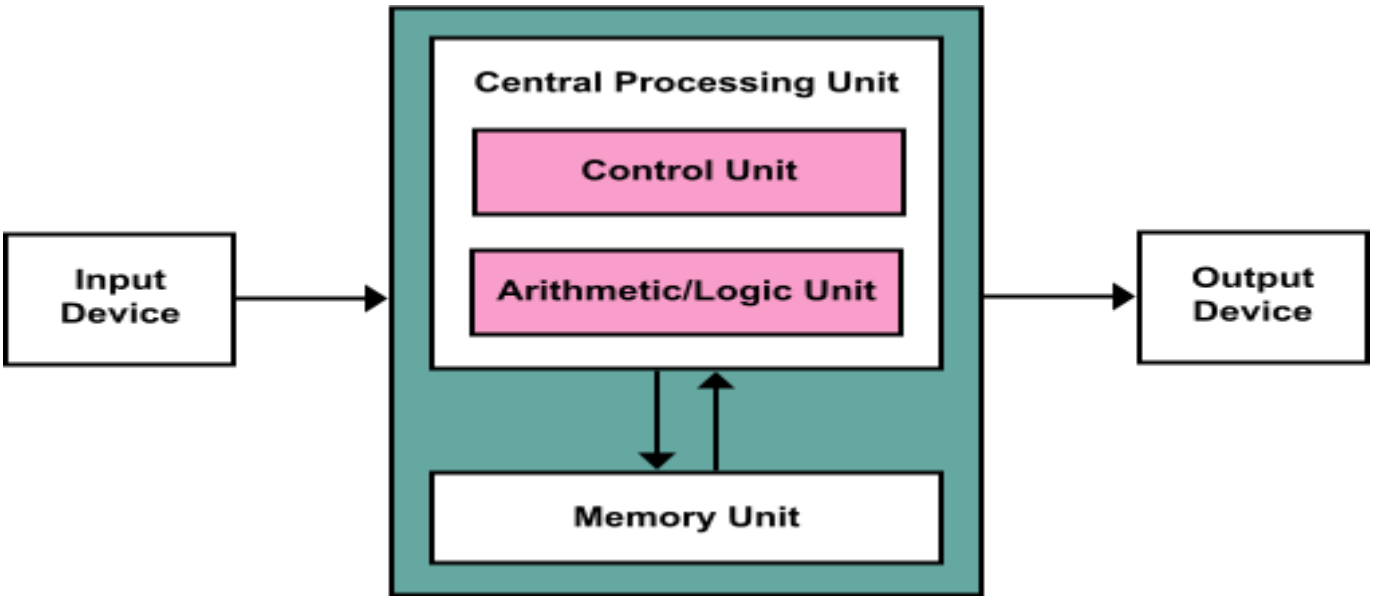
2017 First-Day

- Un procesor cu arhitectura Harvard:
R: Are memorii de date si instructiuni separate

M:

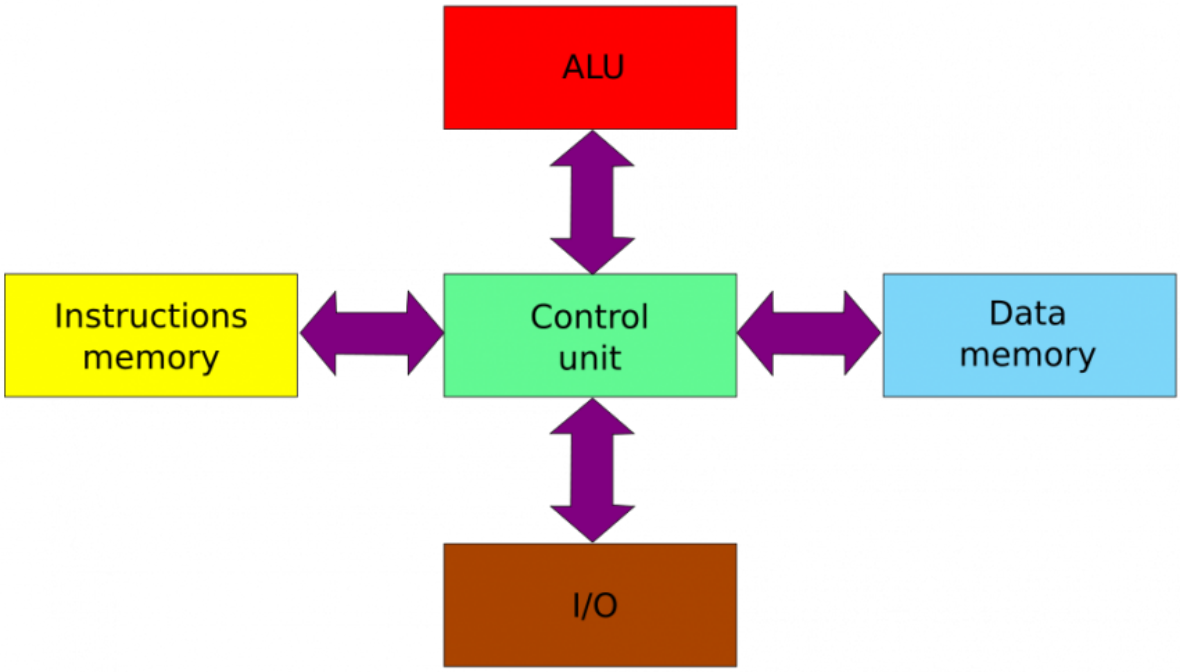
Introdusă în anul 1945 de către matematicianul american John von Neumann, această arhitectură descrie un sistem de calcul ce conține o unitate de procesare, o memorie pentru instrucțiuni și date, și unități de intrare/ieșire. Astfel, un

procesor pentru o astfel de arhitectura va avea o singura magistrală de adrese și o singură magistrală de date, pe care vor circula și date și instrucțiuni. Avantajele acestei arhitecturi sunt simplitatea și faptul că, fiindcă instrucțiunile și datele se află în aceeași memorie, codul se poate auto-modifica.



Arhitectura Harvard pentru sisteme de calcul. (asta e copy-pasta din cursul de CN2).

Spre deosebire de Von Neumann, arhitectura Harvard descrie un sistem de calcul unde memoria de date și memoria de program sunt separate. Asta înseamnă că procesoarele trebuie sa aibă două magistrale de date și de adrese: una pentru instrucțiuni și una pentru date. Avantajul este că aceste magistrale nu trebuie să aibă aceeași dimensiune, deci putem avea un procesor pe 8 biți care adresează mai mult de 256 de octeți de memorie. Un alt avantaj este ca memoria de program poate fi facută non-volatila, deci odată scris un program acesta nu mai trebuie reîncărcat la fiecare pornire.



- O intrerupere ne-mascabila se executa intotdeauna de catre procesor
R: Adevarat

3. Pentru a rula la 3GHz, procesorul unui calculator trebuie sa fie alimentat la o tensiune de 2V, iar curentul de mers in gol al acestua este de 100 mA. Stiind ca procesorul are o capacitate parazita de echivalenta de 10nF, care este puterea necesara ca acesta sa functioneze?
R:

Folosind formula : P = C*Vdd^2* f / 2 + Idd*Vdd

P = 10 * 4 * 10 ^ -9 * 3 * 10 ^ 9 /2 + 100 * 10 ^ -3 * 2 =

= 10 * 4 * 3 /2 + 100 * 10 ^ -3 * 2 =

= 60 + 0.2 = 60.2 W (c)

a. 60,1W

b. 60W

c. 60,2W

d. 120,2W

e. 120 W

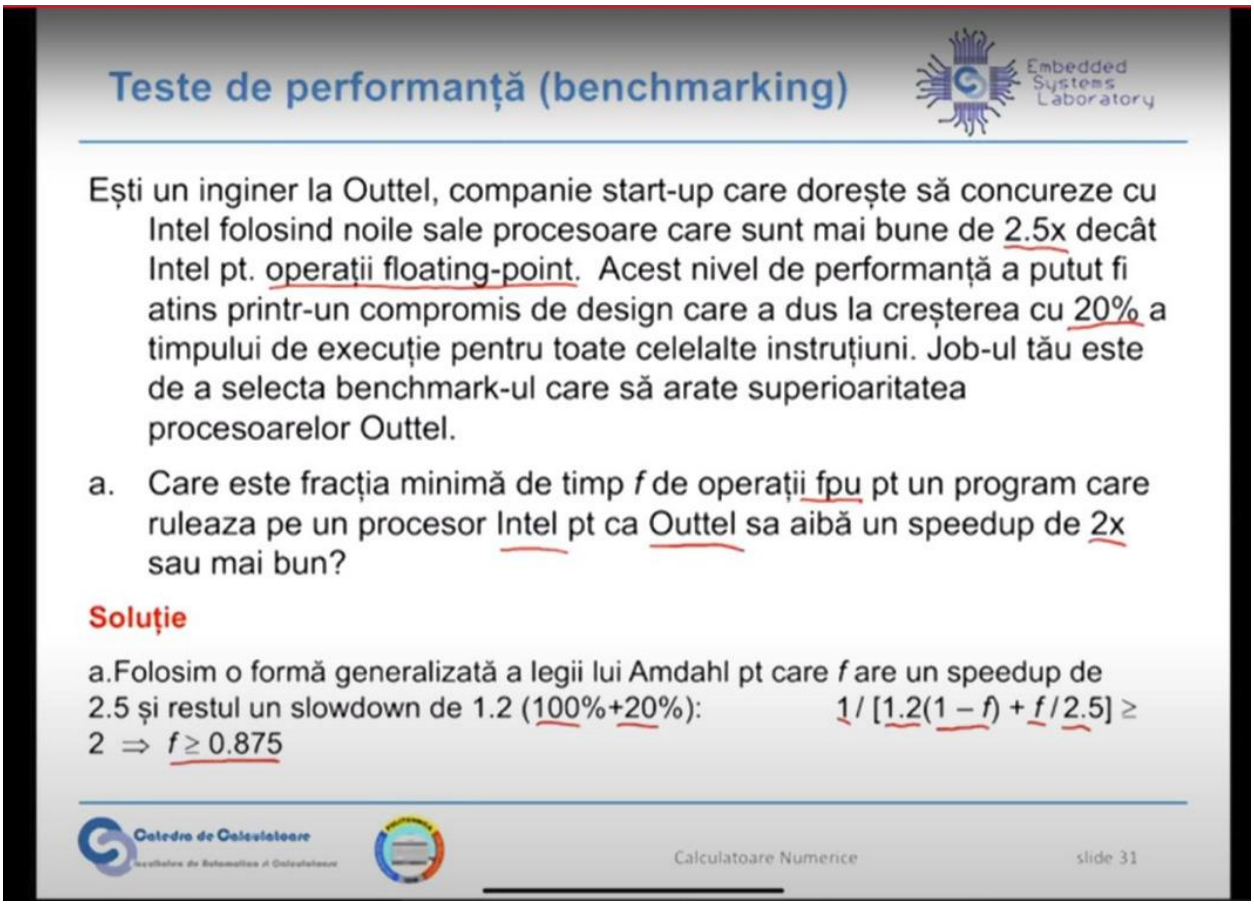
~~~ by Daiana ~~~

4. Numarul 0xCAFEBABE, reprezentat in cod IEEE 754 single precision este:
R: Negativ

5. Presupuneti ca un procesor in b.a. Executa urmatoarea operatie addi r1, r1, 0xbeef. In care etapa este actualizat r1?
R: Write Back

6. Un procesor petrece 80% din timp cu operatii UAL. Daca puteti imbunatati performanta UAL-ului de 4 ori, dar ca rezultat veti incetini executia celorlalte instructiuni cu 50%, ce speedup obtineti?
R: 1 / [(0.8/4) + (0.2 * (1 + 1 * 50%)) = 2
(verificat de Daiana. E bene)

Exemplu curs pentru formula:
~~~



7. Hazardele structurale:
R: Sunt rezolvate prin multiplicarea resurselor partajate

8. Care este avantajul primar al sumatorului CLA fata de un sumator ripple carry?

R: viteza : La ripple Carry se asteapta propagarea carry-ului prin toate sumatoarele full adder, pe cand la CLA, carry-urile sunt procesate simultan => CLA mai rapid.

Timp = nr_cicli_ceas / frecventa

Teorie Examen CN

1. De ce sunt folosite tranzistoare MOSFET pentru procesoarele moderne?

a) Consumul de putere este mai mic decat la tranzistoarele bipolare

b) Au un factor de amplificare mai mare

c) Pot sa aiba doar doua stari: 1 si 0 logic

d) Toate de mai sus

2. Adevarat sau fals: Procesorul cu banda de asamblare executa toate instructiunile intr-un singur ciclu de ceas.

M: (aici as zice fals pt ca daca le-ar executa pe toate intr-un ciclu nu ar mai avea ce tipuri de instructiuni sa separe a.i. sa se execute in paralel)

Nu cred ca ai dreptate cu motivatia. Cu raspunsul sunt de acord

FALS

3. De ce este important sa stim daca o masina este big-endian sau little-endian?

a) Pentru ca anumite programe scriu date in memorie intr-o anumita ordine (toate scriu

dupa cum vrea procesoru; ma rog, daca esti tu smecher si faci un program C care sa iti scrie rezultatele in ce endianness vrei tu e altceva)

b) Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers (asta ar putea fi, dar procesorul nu o sa adune el gresit, el aduna corect dar interpreteaza numerele invers)

c) Pentru ca partajarea folderelor si informatiei intre doua calculatoare poate cauza erori de interpretare - cred ca asta e

d) Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare

4. Numarul 0xC000FEFE, stocat ca signed int este:

a) Prea mare pentru a putea fi reprezentat (are 32 biti deci nu e prea mare)

b) Pozitiv (C e 1100 deci incepe cu 1 si e negativ)

c) Impar (E este 1010 deci se termina cu 0 si e par)

d) Nicio optiune

5. Presupuneti ca aveti de ales intre doua operatii echivalente= {op1}:mul r1, 4 si {op2}: shl r1,2.

Din punctul de vedere al timpului de executie, pentru majoritatea procesoarelor:

a) t(op1) > t(op2) Explicatie: shiftarea e mereu mai rapida

b) t(op1) < t(op2)

c) Ambele dureaza la fel

d) Nu poate fi determinat

6 . Un procesor petrece 50% din timp cu operatii UAL si 40% in operatii cu memoria. Daca puteti imbunatati timpul de acces la memorie de 4 ori, dar, ca rezultat veti incetini executia celorlate instructiuni cu 50%, ce speedup obtineti?

a) 5.3

b) 1 Explicatie: S = 1 / (0.6 * 1.5 + 0.4 / 4) = 1 / 1 = 1

c) 2

d) 3.33

e) Nu poate fi determinat

7. Hazardele de date:

a) Sunt cauzate de UAL

b) Pot fi solutionate prin bypassing

c) Au latenta minima

d) Apar numai la RISC

8 . De ce se foloseste deseori pentru procesoarele RISC denumirea de “arhitecturi load/store”?

Arhitectura load/store are caracteristica de a imparti instructiunile in 2 categorii: operatii cu memoria si operatii cu UAL. Arhitectura procesoarelor RISC face posibile operatiile de orice fel doar intre valori care se afla in registrii(operanzii trb sa fie adusi intr-un registru pentru a putea fi folositi), spre deosebire de CISC(operatiile memorie-registru sunt posibile).

9. Dati cel putin trei caracteristici ale ISA pentru un procesor RISC.

Complet, de mici dimensiuni, open source.

Bonus: o instructiune pe ciclu, dimensiune constanta a instructiunii, mai multe instructiuni per program.

10. Este obligatorie ca o exceptie sa fie tratata de catre procesor atunci cand se produce? Este acelasi lucru valabil si pentru intreruperi? Justificati raspunsurile voastre.

Exceptie este o intrerupere sincrona: tratata dupa terminarea rutinei actuale, lanseaza rutina de tratare dupa ce a golit toata banda de asamblare (exceptia este o intrerupere sincrona).

Intreruperi asioncrone: este tratata in momentul in care e detectata, se salveaza current state-ul, se executa rutina de tratare, se revine in current state (intrerupere asincrona).

Examen CN1 11.09.2015

1 . Descrieti functionalitatea unui circuit logic programabil de tipul PLA sau PAL (schema si scurta explicatie a functionalitatii).

Un circuit combinațional programabil poate să înlocuiască funcționalitatea mai multor porți logice. Programate prin întreruperea unor conexiuni existente (fuses) sau stabilirea de noi conexiuni(antifuses).

2. Proiectati un circuit de tip decodifier 1:2. Aveti la dispozitie numai parti logice de tip NOT, AND si OR cu doua intrati.

https://www.allaboutcircuits.com/textbook/digital/chpt-9/decoder/

3. Un procesor petrece 30% din timp cu adunari flp, 60% cu inmultiri flp si 5% cu impartiri flp. Evaluati imbunatariea performantei pentru:

a) Sumatorul flp este de 3 ori mai rapid

S = 1 / (0.3/3 + 0.6 + 0.05 + 0.05) = 1/0.8 = 1.25

b) Multiplicatorul flp este de 2 ori mai rapid

S = 1 / (0.3 + 0.6/2 + 0.05 + 0.05) = 1/0.7 = 1.43

c) Unitatea de impartire flp este de 5 ori mai rapid

S = 1 / (0.3 + 0.6 + 0.05/5 + 0.05) = 1/0.96 = 1.04

4. Este vizibila, din perspectiva software-ului, diferenta dintre o masina CISC si una RISC? Explicati.

5. Descrieti cei 5 pasi din executia unei instructiuni (fetch, decode, UAL, MEM, WB).

6. Care este principalul dezavantaj al sumatorului select?

Este mai complex. Ocupa mai mult spatiu, cost mai mare.

7. Implementati un CLA pe 4 biti. Scrieti ecuatiile pentru P si G si desenati schema acestuia.

8. Care dintre termenii din cadrul ***Legii de fier*** se minimizeaza la CISC? Dar la RISC?

CISC: Numarul de instructiuni per program

RISC: Cycles per instruction

9. De ce sunt preferati tranzistorii MOSFET in locul celor bipolari? Dati 3 argumente.

1 - Consumul de putere este mai mic decat la tranzistoarele bipolare

2 - Un MOSFET e mai tolerant la caldura decat un tranzistor bipolar

3 - MOSFETs sunt mai usor de fabricat (??)

4 - MOSFETs sunt mai usor de scalat

5 - MOSFET - impedanta de intrare mare

10. Cu tehnologia disponibila in zilele noastre, ce implementare ati alege pentru ISA-ul vostru: CISC sau RISC? Motivati.

Daca se doreste o implementare mai usoara, eficienta din punctul de vedere al memoriei folosite si fara limitari de hardware, putem folosi CISC.

Daca dorim o implementare mai rapida, fara constrangeri de memorie, putem folosi RISC.

+ Alt bullshit, ce iti mai vine prin cap

Examen CN1 12.06.2017

1. Un procesor cu arhitectura Harvard:

a) Are memorii de date si instructiuni separate

b) Executa orice operatie intr-un singur ciclu de ceas - nu e bn ca e ciclu masina, nu ciclu ceas

c) Are o banda de asamblare cu cinci etape - idk

d) Toate de mai sus

2. Adevarat sau fals: O intrerupere ne-mascabila se executa intotdeauna de catre procesor. **ADEVARAT**

3. Pentru a putea rula la 3GHz, procesorul unui calculator trebuie sa fie alimentat la o tensiune de 2V, iar curentul de mers in gol al acestuia este 100mA. Stiind ca procesorul are o capacitate parazita echivalenta de 10nF, care este puterea necesara pentru ca acesta sa funcioneza?

a) 60.1W b) 60W **c) 60.2W** d) 120.2W e) 120W f) 120.1W

P = 1⁄2 * C * (VDD^2) * f + IDD * VDD

P = ½ * 10 * 10^(-9) * (2^2) * 3 * 10^9 + 100 * 10^(-3) * 2

P = ½ * 10 * 4 * 3 + 0.2

P = 60.2W

4. Numarul 0xCAFEBABE, reprezentat in cod IEEE 754 single precision este:

a) Prea mare pentru a putea fi reprezentat (CAFEBABE are 32 de biti, deci poate fi reprezentat in format IEEE 754)

b) Negativ (C este 1100, deci bitul de semn o sa fie 1, deci numar negativ)

c) Are exponentul negativ (Exponentul este 1001 0101, care este 149 > 127, pozitiv)

d) Nicio optiune

CAFEBABE = 1100 1010 1111 1110 1011 1010 1011 1110

-> primul bit de semn

-> urmatorii 8 biti exponentul

-> restul de biti sunt mantissa

5. Presupuneti ca un procesor in b.a. Executa urmatoareae operatie: addi r1, r1, 0xbeef, in care etapa este actualizat r1?

a) Instruction Fetch

b) Decode

c) Execute

d) Memory

e) Write Back

6. Un procesor petrece 80% din timp cu operatii UAL. Daca puteti imbunatati performanta UAL-ului de 4 ori, dar, ca rezultat veti incetini executia celorlalte instructiuni cu 50%, ce soeedup obtineti?MOSFET

a) 5.3

b) 3.5

c) 2 **S = 1 / (0.8 / 4 + 1.5 * 0.2) = 1 / (0.2 + 0.3) = 1 / 0.5 = 2**

d) 3.33

7. Hazardele structurale:

a) Sunt intalnite doar la risc

b) Produc bule in b.a.

c) Sunt rezolvate prin multiplicarea resurselor partajate

8. Care este avantajul primar al sumatorului CLA fata de sumatorul ripple-carry?

Sumatorul CLA foloseste o structura auxiliara care poate sa determine unde vor aparea biti de carry, nemaifiind nevoie astfel sa se astepte rezultatul fiecarui sumator de dinainte, lucru care va duce la eficienta mai buna atunci cand este vorba de un numar mare de sumatoare (N > 16).

9. Care sunt pasii pe care un procesor trebuie sa ii execute pentru tratarea unei intreruperi?

10. Dati exemplu de un hazard de date de tip load/store si un hazard de control.

- Hazard load/store

-> RAW

```
Mov r1, r2
Add r3, r4, r1
```

-> WAR

```
Add r3, r1, r2
Add r2, r4, r5
```

- Hazard de control

```
Cmp eax, ebx ; eax = 100, ebx = 1
```

Jg label1

Jl label2

...

Label1: ...

Label2:...[1] [2]

1. Un coleg va propune sa adaugati instructiunea add3 la ISA-ul procesorului de la curs: Add3 r1, r2, r3, r4 -> R[r1] = R[r2] + R[r3] + R[r4]. Se schimba ISA in acest caz? Are aceasta schimbare vreun impact asupra arhitecturii procesorului?

Ar trebui sa se schimbe, dar nu garantez :)) (teoretic ar trebui sa percepem altfel modul de stocare al instructiunii, adica nu vom mai putea avea in aceeasi 32 de biti 4 registrii, cum aveam 3 la RISC V in curs).

2. Dati cel putin un avantaj si un dezavantaj daca, in loc de instructiuni de lungime fixa(de ex. 32 de biti), am folosi un ISA cu instructiuni de lungime variabila?

Avantaj : Daca am avea memorie variabila, e posibil sa scapam de memoria redundanta folosita (vezi cei doi biti de 11 de la RISC-V).

Dezavantaj : Ar lua mai mult timp sa fie interpretate / sa fie prelucrate (nu dati cu hate daca nu e asa) :(

3. Dati trei caracteristici ale ISA pentru un procesor de tip RISC.

1. Lucreaza doar cu registrii.
2. Lungime fixa a instructiunii.
3. Se executa instructiunile mai rapid ca la cisc (depinde ???)

4. Dacă aplicăm optimizarea B, îmbunătățim viteza de execuție cu 20% din programul A. Care este speed-up-ul maxim teoretic oferit de B?

1,2.

5. Completati

Exercitiu lipsa

6. Victor decide sa isi cumpere urmatoarea tableta bazandu-se pe metrica MIPS^3/(Watt^2 * Greutate)

a) Care sunt caracteristicile pe care Victor le cauta la tableta sa?

- Consum al bateriei mic si greutate mica, putere de procesare mare.

b) Care sunt caracteristicile pe care Victor doreste sa le evite?

- Vrea sa nu fie grea si sa nu **moara** repede bateria

c) Aranjati aceste caracteristici de la cea mai importanta la cea mai putin importanta.

- Pai cu cat e puterea mai mare cu atat e mai importanta. Deci aia cu la a treia e cea mai importanta, dupa aia la a doua, dupa aia simpla. (mi se pare corect :))

7. Un procesor petrece 60% din timp executand operatii cu UAL-ul si 20% din timp in operatii cu memoria. Daca puteti optimiza de 2 ori viteza de executie a operatiilor cu UAL si de 4 ori viteza celor cu memorie, care o sa fie speedup-ul obtinut?

S = 1 / (0.6 / 2 + 0.2 / 4 + 0.2) = 1 / (0.3 + 0.05 + 0.2) = 1/ 0.55 = 1.8

8. Procedeul de bypassing(sau forwarding) elimina hazardele structurale sau cele de date?

Elimina hazardele de date

9. Conform Legii lui Moore, ce fel de rata de creștere în raport cu timpul o are numărul de tranzistoare dintr-un circuit integrat?

Crescătoare (se dubleaza)

9. Întreruperile mascabile:

Alegeți o opțiune:

a. Poartă și numele de excepții

b. Pot să nu fie tratate de procesor

c. Produc bule de exact o instrucțiune lungime în b.a.

d. Sunt întotdeauna sincrone cu ceasul sistemului

e. Pot fi implementate doar de procesoarele RISC

10. Urmatorul cod se executa pe un procesor in b.a. cu 5 etape. Care sunt registrele care sunt citite la inceputul celui de-al 5-lea ciclu de ceas si care sunt cele scrise la sfarsitul aceluiasi ciclu de ceas? Luati in calcul numai registrele din tabela de registre generale(ex. R1, R2, R3..etc.)

ADD R1, R2, R3

ADD R4, R5, R6

ADD R7, R8, R9

ADD R10, R11, R12

ADD R13, R14, R15

Fie numerotarea instructiunilor I1, I2, I3, I4, I5

	T1	T2	T3	T4	**T5**	T6	T7	T8	T9
I1	IF	ID	E	MEM	**WB**				
I2		IF	ID	E	MEM	WB			
I3			IF	ID	E	MEM	WB		
I4				IF	**ID**	E	MEM	WB	
I5					IF	ID	E	MEM	WB

In ciclul 5 de ceas, se citeste la ID, deci de la instructiunea I4: R11 si R12. Se scrie la WB, adica la instructiunea I1: R1.

T1	T2	T3	T4	T5

IF I1 I2 I3 I4 I5

ID I1 I2 I3 I4 -> DECODE LA INCEPUTUL INSTRUCTIUNII

E I1 I2 I3

MA I1 I2

WB I1 -> SCRIERE LA FINALUL INSTRUCTIUNII

CISC VS RISC

CISC	**RISC**
Emphasis on hardware	Emphasis on software
Includes multi-clock complex instructions	Single-clock, reduced instruction only
Memory-to-memory: "LOAD" and "STORE" incorporated in instructions	Register to register: "LOAD" and "STORE" are independent instructions
Small code sizes, high cycles per second	Low cycles per second, large code sizes
Transistors used for storing complex instructions	Spends more transistors on memory registers

Procesoare RISC vs CISC

- RISC (Reduced Instruction Set Computer)
  - » set redus de instructiuni, multe registre
  - » operanzii sunt registre
  - » instrucțiuni simple: aritmetico-logice, comparații, salturi
  - » doar load/store cu memoria
  - » n++ în MIPS:

```
ldw r1, $n
add r1,1,r1
stw $n, r1
```

- CISC (Complex Instruction Set Computer)
  - » puține registre
  - » instrucțiuni mai complexe
  - » multe instrucțiuni cu operanzi în memorie
  - » o instrucțiune de pe un procesor CISC se poate descompune într-o suită de instrucțiuni RISC
  - » n++ în x86:

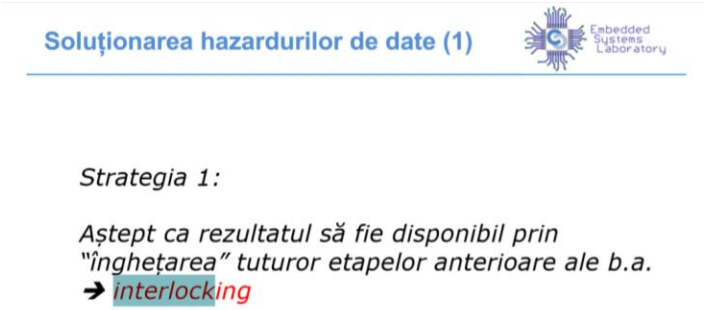
```
inc [n]
```


Procesoare RISC vs CISC

- RISC
  - » Hardware mai ușor de realizat
  - » Durata instrucțiunilor este relativ egală
  - » Lungimea instrucțiunilor este egală
  - » Codul mașină generat de compilator este mai mare
  - » Exemple: ARM, MIPS, Atmel, POWER7
- CISC
  - » Durata de execuție mai mică pentru operațiile frecvente
  - » Hardware mai complicat
  - » Pot exista instrucțiuni foarte scurte, sau foarte lungi
  - » Exemplu: x86, x86_64

Numar Procedeul de interlocking elimina hazardele structurale sau de date?

R: ask @DN. date



Numar: Ce se intampla cu latenta executiei unei instructiunii daca marim lungimea b.a(adaugam noi etape)?

R: probabil creste, ask @DN.

Numar: Ca sa elimine hazardele de date, banda de asamblare A se opreste (stall). Aceasta are un CPI de baza de 1 si timpul mediu de asteptare dupa un hazard este de 3 cicii (adica CPI mediu total este 4). Banda de asamblare B foloseste hardware pt data forwarding si elimna producerea hazardelor de date ot 50% din instructiuni.

Care este speedup B fata de _? = _ / B

50% - 1

50% - 4

Cpi med pt B: 0.5 + 2 = 2.5

1 / (0.5 + 0.5 / 4) = 1 / 0.625 = 1.6

R: @ask DN, intuim 4 / 2.5.

Numar: Aveti urmatorul listing de program in assembler:

1. MOV R3, R7
2. LD R8, (R3)
3. ADD R3,R3,4
4. LOAD R9, (R3)
5. BNE R8, R9, L3

a) Acest program are dependinte WAW, RAW, WAR. Care sunt acestea?

b) Care este diferenta dintre o dependinta de date si un hazard?

I think a **dependency** is something you see by looking at the code and trying to figure out possible WAW, WAR, RAW hazards that could happen. **Actual hazards** instead are a property of the pipeline which means that a dependency you found earlier may or may not generate an hazard depending on the actual code execution in the processor.

Link: https://cs.stackexchange.com/questions/66489/what-is-the-difference-between-data-hazard-and-dependencies-in-pipelining

Numar:

Aveti doua procesoare, care implementeaza ISA diferite. Primul implementeaza ISA A si executa cod cu o rata medie de 10 IPC (Instructions Per Cycle) la o frecventa de 500 MHz. Al doilea ruleaza ISA B cu 2 IPC la o frecventa de 600 MHz.

Ask @DN

a) Care este performanta in MIPS pentru fiecare procesor?

R: pt A = 500MHZ * 10 IPC = 500 M c / s * 10 inst / c = 5000 MIPS (inst / s)

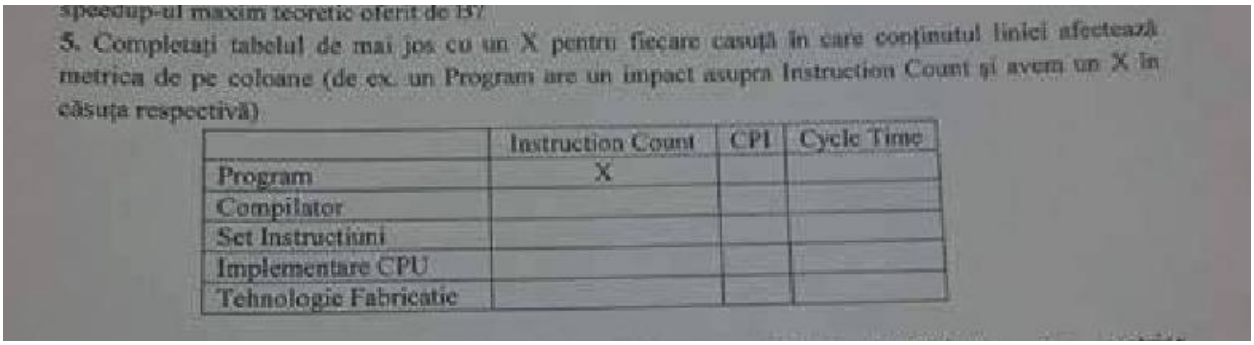
Pt B = 600MHZ * 2 IPC = 1200 MIPS

b) Care este procesorul cu performanta cea mai mare dintre cele 2? Depinde :) (primul are mai multe IPC, dar cred ca mai depind de alti parametrii).

Se schimba numarul de dependente de date daca se adauga 5 stagii in plus la procesorul de la curs?

Numar:

Ask @DN



speedup-ul maxim teoretic oferit de IS?

5. Completați tabelul de mai jos cu un X pentru fiecare casuță în care conținutul liniei afectează metrica de pe coloane (de ex. un Program are un impact asupra Instruction Count și avem un X în căsuța respectivă)

	Instruction Count	CPI	Cycle Time
Program	X		
Compilator			
Set Instructiuni			
Implementare CPU			
Tehnologie Fabricatie			

@all, daca mai e ceva ce nu e adaugat, puneti voi.

Intrebari prinse

1. Unde se scrie BIOS-ul?

   R: ROM

2. Cand se aduce de pe magistrala in instruction register valoarea?

   R: la frontul pozitiv al ceasului

3. Ce e ala asamblor?
4. Ce e ala compilator?

TODO before EXAM

1. **Curs0 -> Weierstrass**
2. **Curs0 -> slide 46**
3. **Curs1 -> Putere dinamica**
4. **Curs1 -> Putere statica**
5. **Curs4 -> slide 35 (setul de instructiuni)**
6. **Curs6 -> important**

7. **Curs6 -> slide 47**
8. **In general, fiecare bullet e o grila.**

Introducem o etapă suplimentară în banda de asamblare pentru procesorul de la curs. Care este efectul asupra ISA?

Select one:

a. Nici un efect

b. Câmpul *opcode* primește un bit suplimentar pentru noua etapă

c. Se modifică toate instrucțiunile ce fac acces la memorie

d. Instrucțiunile se vor executa mai rapid

e. Toate instrucțiunile se vor modifica

Stie cnv?

ELEMENTE INTRODUCTIVE PRIVIND INFORMAȚIA

*1.*PROBLEME PRIVIND MĂSURAREA INFORMAȚIEI.

1.1 Într-o urnă sunt plasate bilete cu numele a 5 femei şi 3 bărbați. Dacă se comunică faptul că a fost extras un bilet cu numele unui bărbat, câtă informație a fost transmisă?

1.2 Dispunând de un pachet de 52 de cărți, plasate în ordine aleatoare, cu fețele în jos, câți biți de informație se vor obține la extragerea:

a. primei cărți,
b. celei de-a cincea cărți,
c. a ultimei cărți.

1.3 Fie **X** un număr binar de **n** biți (**n** > 3). Se comunică faptul că primii 3 biți ai numărului **X** sunt 001. Câți biți de informație despre numărul **X** au fost transmişi ?

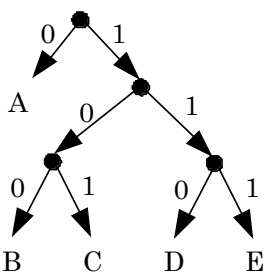
2 PROBLEME PRIVIND DETECTAREA ŞI CORECTAREA ERORILOR, COMPRESIA

2.1 Diferitele scheme de codificare, inclusiv metoda Huffman, dedică mai mulți biți pentru codificarea:

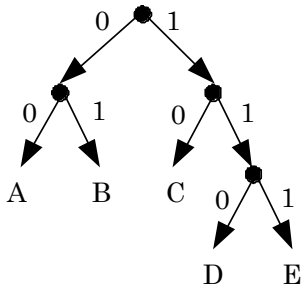
- a simbolurilor care transportă mai multă informație,
- a simbolurilor care transportă cea mai puțină informație,
- a simbolurilor care par a se repeta consecutiv,
- a simbolurilor care conțin informație redundantă ?

2.2 Se dau următorii arbori de decodificarea Huffman, pentru codurile de lungime variabilă corespunzătoare următoarelor 5 simboluri: A, B, C, D şi E:

Arborele 1



Arborele 2



Utilizând **Arborele 1** să se decodifice următorul mesaj: "01000111101".

2.3 Se presupune că are loc codificarea de mesaje cu următoarele probabilități de apariție pentru fiecare dintre cele 5 simboluri: p(A) = 0,5; p(B) = p(C) = p(D) = p(E) = 0,125. Care dintre cele două posibilități de codificare de mai sus (**Arborele 1** sau **Arborele 2**) va genera cele mai scurte mesaje codificate?

2.4 Utilizând probabilitățile pentru A, B, C, D şi E date mai sus, să se construiască un arbore binar de decodificare pentru coduri de lungime variabilă folosind un algoritm simplu "greedy", după cum urmează:

- Se începe cu setul de S simboluri, care trebuie codificate, împreună cu probabilitățile P(x) pentru fiecare simbol x. Suma tuturor probabilităților este 1. Probabilitățile măsoară frecvențele de apariție ale fiecărui simbol în şirul de intrare.
- Se repetă următorii paşi până când în S rămâne un singur simbol:

Se selectează doi membri din S, care au asociate cele mai mici probabilități. În exemplul dat, D şi E pot fi primele noduri alese.

Se înlătură simbolurile selectate din S şi se creează un nou nod în arborele de decodificare ai cărui copii (subnoduri) sunt simbolurile înlăturate. Se marchează ramura din dreapta cu "0" şi ramura din stânga cu "1". Astfel, la prima iterație, se va forma primul nod intern, de pe cel mai jos nivel (care conduce la D şi E).

Se adaugă la S noul simbol (de exemplu "DE"), care va reprezenta un nou nod. Se atribuie acestui nou simbol o probabilitate egală cu suma probabilităților simbolurilor pe care le înlocuieşte.

2.5. Codificarea Huffman este utilizată pentru a codifica compact o serie de păsări, dintr-o rezervație. Daca 50% dintre păsări sunt grauri, iar restul îl reprezintă alte specii, câți biți se vor folosi pentru a codifica graurii?

2.6 Studentul **Bitescu**, de la Calculatoare, a inventat un nou limbaj, AEI,care constă numai în 5 vocale: "A", "E", "I", "O" şi "U". Acestea apar în mesaje cu următoarele probabilități:

Litera	Probabilitatea de apariție
A	p(A) = 0,15
E	p(E) = 0,4
I	p(I) = 0,15
O	p(O) = 0,15
U	p(U) = 0,15

2.6.1 Dacă se afirmă că prima litera a unui mesaj este “A”, să se stabilească expresia pentru numărul de biți de informație furnizați.

2.6.2 Bitescu încearcă să găsească un cod de lungime fixă pentru AEI, care să-i permită să detecteze şi să corecteze erorile la nivelul unui singur bit. Să se prezinte restricțiile pe care le va întâmpina Bitescu la codificarea fiecărei litere, în vederea realizării dezideratelor propuse.

2.6.3 Bitescu este interesat ca să transmită mesaje AEI cu cât mai puțini biți posibil. Se consideră că fiecare literă este codificată separat. Ajutați-l să creeze un cod de lungime variabilă care să minimizeze numărul mediu de biți transmişi pentru fiecare literă a mesajului.

2.7 Pentru protecția informației stocate sau transmise se pot adăuga datelor biți de verificare, care vor facilita detectarea şi corectarea erorilor. Astfel, o schema pentru detectarea erorilor singulare de 1 bit constă în adăugarea unui bit de paritate **p**:

$$b_0\, b_1\, b_2\, b_{N-1}\, p$$

Când se utilizează paritatea pară, **p** se alege astfel încât numărul de biți egali cu “1“, inclusiv în bitul de paritate să fie par; la paritatea impară numărul de biți va fi impar. În continuare se va considera paritatea pară.

Care dintre şirurile de biți următoare, prevăzute cu bit de paritate au erori detectabile?

(1) 11101101111011011

(2) 11011110101011011

(3) 10111110111011110

(4) 00000000000000000

2.8 Pentru a construi un cod cu corectarea erorii (ECC) se vor adăuga biți suplimentari, pentru a depista pozițiile biților eronați. Există, în acest sens, numeroase scheme. Una dintre acestea, pentru detectarea şi corectarea unei erori singulare, consta în organizarea datelor pe linii şi coloane, adăugând apoi biții de paritate pară pentru fiecare linie şi fiecare coloană. Următoarea organizare va asigura protecția a 9 biți de date:

b0,0 b0,1 b0,2 plinie0

b1,0 b1,1 b1,2 plinie1

b2,0 b2,1 b2,2 plinie2

pcol0 pcol1 pcol2

O eroare singulară de 1 bit de date $(b_{I,J})$ va genera două erori de paritate, una în linia **I** şi cealaltă în coloana **J.** O eroare singulară într-unul din biții de paritate va genera numai o singură eroare de paritate, pentru linia sau coloana corespunzătoare. Astfel, după calculul parității, pentru fiecare linie şi coloană, în cazul apariției unei erori pentru linia şi coloana dată, inversând valoarea bitului de la intersecția acestora se va realiza corectarea erorii. Dacă se detectează o singură eroare de paritate, datele sunt corecte iar eroarea a fost într-unul dintre biții de paritate.

Să se furnizeze datele corecte pentru fiecare dintre blocurile de date protejate cu ECC, la nivel de linii şi coloane

(1) 1011 (2) 1100 (3) 000 (4) 0111

0110 0000 111 1001

0011 0101 10 0110

011 100 100

2.9 Liniile şi coloanele ECC pot detecta, de asemenea, erori de câte doi biți (ex., 2 biți de date sunt biți de verificare modificați). Să se evidențieze tipurile de erori la nivelul a 2 biți, care nu se pot detecta.

2.10 În perioada în care se utilizau cartele perforate, cifrele zecimale erau reprezentate prin codurile speciale *2 din 5*. După cum arată şi numele două dintre cele 5 poziții binare erau încărcate cu unități după cum se vede din tabelul de mai jos:

Codul 2 din 5	Zecimal
11000	1
10100	2
10010	3
10001	4
01100	5
01010	6
01001	7
00110	8
00101	9
00011	0

Care este cea mai mică distanță Hamming între două coduri *2 din 5?.*

2.10.a Să se caracterizeze tipurile de erori (ex., de unul sau doi biți), care pot fi sigur/fiabil detectate în *codul 2 din 5*.

2.10.b Paritatea pară reprezintă o altă schemă de detectare a erorilor. Daca se trece de la codul *2 din 5* la un cod de 5 biți, care include un bit de paritate pară , câte date pot fi suplimentar codificate?

2.11 Codul Hamming pentru corectarea unei erori singulare necesita aproximativ $\log_2(N)$ biți de verificare, pentru a corecta erori singulare, de un bit. Se începe cu reenumerarea biților de date cu indici care nu sunt puteri ale lui 2:

Indicii pentru16 biți de date = 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21

Ideea este aceea de a calcula biții de verificare pentru subseturile de date, astfel încât o eroare de 1 bit să producă un set de erori de paritate, care în mod unic indică indicele bitului eronat:

p0 = paritate para pentru biții de date 3, 5, 7, 9, 11, 13, 15, 17, 19, 21

p1 = paritate para pentru biții de date 3, 6, 7, 10, 11, 14, 15, 18, 19

p2 = paritate para pentru biții de date 5, 6, 7, 12, 13, 14, 15, 20, 21

p3 = paritate para pentru biții de date 9, 10, 11, 12, 13, 14, 15

p4 = paritate para pentru biții de date 17, 18, 19, 20, 21

De remarcat faptul că fiecare bit de date apare în cel puțin două calcule de bit de paritate, astfel încât o eroare de un singur bit de date va produce cel puțin două erori de paritate. Atunci când se verifică câmpul de date protejat, dacă numărul de erori de paritate este egal cu 0 sau 1, biții de date sunt în regulă (mai exact, o eroare de paritate singulară arată că un bit de paritate este incorect). Dacă sunt detectate două sau mai multe erori de paritate, atunci erorile identifică exact care bit a fost eronat.

2.11.1 Care este relația între indicele unui bit particular de date şi subsetul de verificare în care acesta apare? (Indicație: se consideră reprezentarea binară a indicelui)

2.11.2 Dacă calculele de paritate referitoare la **p0**, **p2** şi **p3** indică erori, considerând o eroare de un bit, care este indicele bitului eronat de date?

2.11.3 SECC Hamming nu detectează toate erorile de câte doi biți. Să se caracterizeze tipurile de erori la nivelul a doi biți, care nu vor fi detectate. Să se sugereze o extensie la SECC Hamming, care permite detectarea tuturor erorilor la nivelul a doi biți.

RĂSPUNSURI

1.1 Cunoscând că, în total sunt **n** = 8 nume şi că extragerea a constatat că este vorba de numele unui bărbat, numărul selecțiilor posibile s-a redus la **m** = 3. Conform formulei $\log_2\left(\frac{n}{m}\right)$ s-au obținut $\log_2\left(\frac{8}{3}\right)$ biți de informație. Probabilitatea de a extrage numele unui bărbat este $p_b = \frac{3}{8}$ astfel cantitatea de informație obținută va fi: $\log_2\left(\frac{1}{p_b}\right) = \log_2\left(\frac{1}{\frac{3}{8}}\right) = \log_2\left(\frac{8}{3}\right)$.

1.2 (a) $\log_2\left(\frac{52}{1}\right)$ biți de informație.

(b) $\log_2\left(\frac{48}{1}\right)$ biți de informație.

(b) $\log_2\left(\frac{1}{1}\right)$ = 0 biți de informație.

1.3 Intuitiv, întrucât s-a dat informație despre 3 biți din X, se apreciază că au fost dați 3 biți de informație. Trecând la formule: există 2^n numere binare de câte **n** biți şi 2^{n-3} numere binare care încep cu 001. Astfel au fost transmişi: $\log_2\left(\frac{2^n}{2^{n-3}}\right) = \log_2(2^3)$ = 3 biți de informație.

2.1 Pentru a menține mesajele cât mai scurte posibil, simbolurile care apar cu frecvența mare sunt codificate cu mai puțini biți, în timp ce simbolurile care apar rar sunt codificate cu mai mulți biți.

2.2 Pentru decodificare se porneşte de la rădăcina arborelui, parcurgând rangurile pe măsura traversării acestuia pentru a atinge un nod frunză. Se repetă procedura până la epuizarea tuturor rangurilor numărului. Prelucrarea se efectuează de la stânga la dreapta.

"0" => A; "100" => B; "0" => A; "111" => E; "101" => C

2.3 Folosind **Arborele 1**, lungimea aşteptată a codificării pentru un simbol este:

$$1 * p(A) + 3 * p(B) + 3 * p(C) + 3 * p(D) + 3 * p(E) = 2{,}0$$

Folosind **Arborele 2**, lungimea aşteptată a codificării pentru un simbol este:

$$2 * p(A) + 2 * p(B) + 2 * p(C) + 3 * p(D) + 3 * p(E) = 2{,}25$$

În concluzie, codificarea pe baza Arborelui 1 va conduce, în medie, la cele mai scurte mesaje.

2.4 $S = \left\{ \frac{A}{0,5}, \frac{B}{0,125}, \frac{C}{0,125}, \frac{D}{0,125}, \frac{E}{0,125} \right\}$

Se aleg arbitrar D & E

Se codifică: "0" => D, "1" => E

$$S = \left\{ \frac{A}{0,5}, \frac{B}{0,125}, \frac{C}{0,125}, \frac{DE}{0,25} \right\}$$

Se aleg B & C

Se codifică: "0" => B, "1" => C

$$S = \left\{ \frac{A}{0,5}, \frac{BC}{0,25}, \frac{DE}{0,25} \right\}$$

Se aleg BC & DE

Se codifică: "00" => B, "01" => C, "10" => D, "11" => E

$$S = \left\{ \frac{A}{0,5}, \frac{BCDE}{0,5} \right\}$$

Se aleg A & BCDE

Codificarea: "0" => A, "100" => B, "101" => C, "110" => D, "111" => E

$$S = \left\{ \frac{ABCDE}{1,0} \right\}$$

Acesta este **Arborele 1** de mai sus. Selectarea lui D şi E ca prime simboluri, care se combină, a fost arbitrară. Puteau fi alese oricare alte două simboluri dintre B, C, D şi E. Astfel, există multe codificări plauzibile pentru acest exemplu, date fiind probabilitățile egale ale acestor 4 simboluri.

2.5 Folosind algoritmul de mai sus - 1 bit.

2.6.1 Folosind formula dată în cadrul cursului de va obține: $\log_2 \left(\frac{1}{p(A)} \right) = \log_2 \frac{1}{0,15}$

2.6.2 Fiecare cod trebuie să difere de celelalte coduri prin cel puțin 3 poziții, adică să aibă distanța Hamming ≥ 3. Aceasta va asigura faptul că fiecare cuvânt-cod recepționat (chiar şi acelea cu un singur bit eronat) vor fi asociate cu o sursă dată de cod.

2.6.3. Folosind algoritmul “greedy”, descris mai sus se va obține:

$$S = \left\{ \frac{A}{0,15}, \frac{E}{0,4}, \frac{I}{0,15}, \frac{O}{0,15}, \frac{U}{0,15} \right\}$$

Se aleg arbitrar O & U

Se codifică: "0" => O, "1" => U

$$S = \left\{ \frac{A}{0,15}, \frac{E}{0,4}, \frac{I}{0,15}, \frac{OU}{0,3} \right\}$$

Se aleg A & I

Se codifică: "0" => A, "1" => I

$$S = \left\{ \frac{AI}{0,3}, \frac{E}{0,4}, \frac{OU}{0,3} \right\}$$

Se aleg AI & OU

Se codifică: "00" => A, "01" => I, "10" => O, "11" => U

$$S = \left\{ \frac{AIOU}{0,6}, \frac{E}{0,4} \right\}$$

Se aleg E & AIOU

Se codifică: "0" => E, "100" => A, "101" => I, "110" => O, "111" => U

$$S = \left\{ \frac{AEIOU}{0,1} \right\}$$

Se poate observa că atribuirile simbolurilor pentru codificările “0” şi”1” au fost arbitrare şi că, la fiecare nivel, ele au putut fi interschimbate. Astfel, interschimbarea la nivelul ultimului pas ar fi dus la următoarea codificare:

"1" => E, "000" => A, "001" => I, "010" => O, "011" => U,

care conduce la aceiaşi valoare medie de biți/simbol ca şi codificarea anterioară.

2.7 Şirurile 1 şi 3 au erori detectabile. De observat că paritatea permite detectarea erorilor de un bit (sau a erorilor de un număr impar de biți).

2.8 (1**)** **0**011 (2) 1100 (3) 000 (4) 011**0**

0110	0000	1**0**1	1001
0011	0101	10	0110
011	100		100

(1) şi (3) au erori de un singur bit, (2) nu are erori detectabile, iar (4) are o eroare în bitul de paritate. Rangurile reprezentate în roşu constituie valorile corecte.

2.9 Dacă biții de paritate detectează o eroare pentru o singură coloană şi o singură linie, aceasta se va interpreta ca o eroare în poziția corespunzătoare a datei. Apariția a două erori nu a fost detectată, şi mai rău, valoarea din poziția corespunzătoare a datei a fost modificată, de la cea corectă la una incorectă.

2.10 Distanța Hamming între două coduri este numărul de poziții de bit în care cele două coduri diferă. În cazul de față distanța Hamming este egală cu 2.

2.10.a Codurile Hamming cu distanța 2 pot detecta o eroare de 1 bit. Codul *2 din 5* poate detecta erori de 3 biți, dar nu erori de 2 biți. În mod normal, când se spune că un cod detectează erori de **n** biți, se consideră că detectează erori de **m** biți, unde **m < n**. Conform acestei convenții, se poate spune că, *codul 2 din 5* detectează erori de 1 bit.

2.10.b La un cod cu 4 biți de date şi 1 bit de paritate există 16 posibilități de a codifica datele, cu 6 mai mult decât în cazul codului *2 din 5*.

2.11.1 Rangul **i** în reprezentarea binară a indicelui bitului de date indică faptul că indicele trebuie să fie inclus în calculul lui **pi.** De exemplu, primul bit de date (al cărui indice este 3 = 00011) va fi utilizat în calculul parității pentru biții de paritate **p0** şi **p1**. În mod asemănător bitul 6 de date (care are indicele 10 = 01010) va fi utilizat pentru calculul biților de paritate **p1** şi **p3.** Întrucât nici unui bit de date nu i-a fost

atribuit un indice, care să fie o putere a lui 2, se poate garanta faptul că fiecare bit de date este utilizat în calculul a cel puțin 2 biți de paritate diferiți.

2.11.2 Trebuie să se determine indicele datei, care apare în calculele pentru **p0**, **p2** şi **p3,** dar şi in calculele pentru **p1** şi **p4**. Indicii 13 şi 15 apar în **p0**, **p2** şi **p3**, dar indicele 15 apare în calculul pentru **p1**. Astfel, indicele bitului de date eronat este 13.

Se poate construi indicele bitului eronat de date direct din calculul de paritate folosind **ei** = 1, dacă **pi** este eronat sau **ei** =0, dacă nu este eronat. Astfel, dacă **p0**, **p2** şi **p3** sunt eronați, iar alți nu sunt, indicele este **e4e3e2e1e0** = 01101 = 13_{10}.

2.11.3 Dacă apare o eroare în doi biți de paritate separați, aceasta se va interpreta greşit ca o eroare la nivelul unui bit singular de date. De asemenea, când anumite perechi de biți de date au erori, eroarea la nivelul celor doi biți se insinuează ca fiind eroare la nivelul unui singur bit într-un alt bit, fără nici o legătură (ex. Biții cu indicii 19 şi 21 sunt eronați)

Aceste situații pot fi detectate prin adăugarea unui bit suplimentar de paritate, **p5**, care este bitul de paritate pentru ceilalți biți de date şi paritate. Dacă un bit singular de date este eronat, acest bit va indica o eroare. Dacă exact doi biți sunt eronați, **p5** nu va indica eroare, dar **p0, p1,…p4** vor indica prezența unor erori (cu toate că indicația se referă la un bit eronat de date). Astfel, dacă examinarea lui **p0, p1, ..., p4** sugerează apariția unei erori, se va verifica **p5**, pentru a fi siguri ca a apărut numai o eroare. Dacă **p5** nu indică o eroare, atunci a apărut o eroare la nivelul a doi biți.

Examen CN1 - Partea II - 09.06.2018

Nume: …… Grupa: …….

1. (7p) Fie următorul cod în assembly (x86):

```
       0: mov eax, 0     ; eax = 0
       1: mov ecx, 4     ; ecx = 4
       2: xor edi, edi   ; edi = 0
_loop:
       3: cmp eax, ecx   ; eax ? ecx
       4: je _out
       5: mov ebx, eax   ; ebx = eax
       6: and ebx, 0x01  ; ebx &= 0x01
       7: cmp ebx, 1     ; ebx ? 1
       8: je _inc
       9: add eax, 1     ; eax += 1
      10: jmp _loop
_inc:
      11: inc edi        ; edi += 1
      12: add eax, 1     ; eax += 1
      13: jmp _loop
_out:
      ; se afiseaza continutul lui edi
```

Procesorul ce execută acest cod rulează la frecvența de **2Ghz**. Metoda de *branch-prediction* folosită de fiecare este **1-bit counter** (**taken/not taken**), starea inițială fiind ***not taken***. Predictorul este unul **local**, astfel încât există un state-machine pentru fiecare branch.

Știm numărul de cicli de ceas pentru fiecare tip de instrucțiune: operațiile cu date și memorie (**mov**) durează **2** cicli, operațiile aritmetice/logice (**cmp**, **add**, **inc, dec, xor, and**) durează **1** ciclu, operațiile de salt necondiționat (**jmp**) durează **10** cicli, operațiile de salt condiționat (**jg, jl, je**) durează **50** cicli (dacă nu a fost prezis corect) sau **10** cicli (dacă a fost prezis corect).

Se cer următoarele:

a. **(1p)** Ce valoare afișează programul? Justificați!
b. **(5p)** Evidențiați prin ce stări trec branch-predictor-ele, semnalând și care branch-uri au fost prezise corect, respectiv incorect.
c. **(1p)** Care este timpul de rulare al programului?

2. (3p) Fie următorul cod în assembly (x86). Indentificați hazardele de date. Pentru fiecare menționați intrucțiunile implicate, arătând cauzele fiecărui hazard.

```
0: mov eax, ebx          ; eax = ebx
1: mov ebx, [0xCAFEBABE]       ; ebx = MEM[0xCAFEBABE]
2: add ecx, eax          ; ecx += eax
3: mov ecx, 0xCAFEBABE   ; ecx = 0xCAFEBABE
4: mov eax, ecx          ; eax = ecx
```

Rezolvare

1.

a. 2, fiindca numara numerele impare in [0, 4)
b. je (linia 4): NT (corect), NT (corect), NT (corect), NT (corect), NT (gresit), T;
   je (linia 8): NT (corect), NT (gresit), T (gresit), NT (gresit), T
c. 322 cicli = 161 ns

2. I0 - I2: RAW (eax); I2 - I4: RAW (ecx); I3 - I4: RAW (ecx);

Examen CN1 - Partea II - 09.06.2018

Nume: …… Grupa: …….

1. (7p) Fie următorul cod în assembly (x86):

```
       0: mov eax, 4    ; eax = 4
       1: mov ecx, 0    ; ecx = 0
       2: xor esi, esi  ; esi = 0
_loop:
       3: cmp eax, ecx  ; eax ? ecx
       4: je _out
       5: mov ebx, eax  ; ebx = eax
       6: and ebx, 0x01 ; ebx &= 0x01
       7: cmp ebx, 0    ; ebx ? 0
       8: je _inc
       9: sub eax, 1    ; eax -= 1
      10: jmp _loop
_inc:
      11: inc esi       ; esi += 1
      12: sub eax, 1    ; eax -= 1
      13: jmp _loop
_out:
      ; se afiseaza continutul lui esi
```

Procesorul ce execută acest cod rulează la frecvența de **3Ghz**. Metoda de *branch-prediction* folosită de fiecare este **1-bit counter** (**taken/not taken**), starea inițială fiind ***not taken***. Predictorul este unul **local**, astfel încât există un state-machine pentru fiecare branch.

Știm numărul de cicli de ceas pentru fiecare tip de instrucțiune: operațiile cu date și memorie (**mov**) durează **2** cicli, operațiile aritmetice/logice (**cmp**, **add**, **inc, dec, xor, and**), durează **1** ciclu, operațiile de salt necondiționat (**jmp**) durează **10** cicli, operațiile de salt condiționat (**jg, jl, je**) durează **60** cicli (dacă nu a fost prezis corect) sau **10** cicli (dacă a fost prezis corect).

Se cer următoarele:

a. **(1p)** Ce valoare afișează programul? Justificati!
b. **(5p)** Evidențiați prin ce stări trec branch-predictor-ele, semnalând și care branch-uri au fost prezise corect, respectiv incorect.
c. **(1p)** Care este timpul de rulare al programului?

2. (3p) Fie următorul cod în assembly (x86). Indentificați hazardele de date. Pentru fiecare menționați intrucțiunile implicate, arătând cauzele fiecărui hazard.

```
0: mov ebx, eax          ; ebx = eax
1: mov ebx,[0xDEADBEEF] ; ebx = MEM[0xDEADBEEF]
2: add eax, ecx          ; eax += ecx
3: mov [0xDEADBEEF], eax       ; MEM[0xDEADBEEF] = eax
4: mov ecx, eax          ; ecx = eax
```

Rezolvare

1.

a. 2, fiindca numara numerele pare in [4, 0)
b. je (linia 4): NT (corect), NT (corect), NT (corect), NT (corect), NT (gresit), T
   je (linia 8): NT (gresit), T (gresit), NT (gresit), T (gresit), NT
c. 412 cicli = 137.34 ps

2. I2 - I3: RAW (eax); I2 - I4: RAW (eax);

Problemă CN1
9 iunie

Fie următorul cod în assembly (simplificat):

```
mov ax, $a
mov bx, $b
_while:    cmp ax, bx
   jg _greater
   jl _less
   je _final
   jmp _while

_greater:
   sub ax, bx ;ax = ax - bx
   jmp _while

_less:
   sub bx, ax ;bx = bx - ax
   jmp _while

_final: ;se afiseaza conținutul lui ax
```

Avem 2 procesoare, fiecare având frecvența de **1Ghz,** care execută acest cod, procesoarele A și B. Metoda de *branch-prediction* folosită de fiecare este:

A. 1-bit counter (taken/not taken)
B. 2-bit counter (strongly taken/weak taken/weak not taken/strongly not taken)

Ambele pornesc cu o abordare pesimistă, counter-ul/state-machine-ul fiecăreia având starea inițială pentru procesorul A *not taken* și pentru B *strongly not taken*. De altfel, ambele implementează un *branch-predictor global*, astfel că există un singur counter/state-machine global, pentru toate branch-urile.

Se știu numărul de cicli de ceas pentru fiecare tip de instrucțiune: **mov** - 1 ciclu, **cmp** - 1 ciclu, **sub** - 1 ciclu, **branch** - 20 cicli (dacă nu a fost prezis corect) sau 2 cicli (dacă a fost prezis corect).

Se cer următoarele:

1. Pentru input-ul **a=26** și **b=32** să se evidențieze prin ce stări trec branch-predictor-ele celor 2 procesoare, semnalându-se și care branch-uri au fost prezise corect, respectiv incorect. **(4p)**
2. Să se calculeze timpul de rulare al programului pe cele 2 procesoare, pentru input-ul de la punctul 1. **(3p)**
3. FIe un procesor C, cu frecvență de 1Ghz, care are aceeași metoda de *branch-prediction* pe 2 biți ca procesorul B, implementând însă un *branch-predictor local* (câte un counter/state-machine pentru fiecare branch). Execută procesorul C codul dat mai repede decât procesorul A sau B, pentru input-ul de la punctul 1? Argumentați. Care este speed-up față de B? **(3p)**

Problemă CN1
10 iunie

Fie următorul cod assembly :

0		load R0, #n
1		mov R1,0
2		mov R2, 1
3	loop:	cmp R0, 1
4		je Dan
5		mov R3, R1
6		add R1, R2
7		mov R2,R3
8		sub R0,1
9		j loop
10	Dan:	;print R1

Avem 2 procesoare, A și B:

- A = mașină, fără pipeline, cu branch prediction pe 1 bit.
  Numărul de cicli /instrucțiuni este: load - 10 cicli, mov 1 ciclu, cmp 1 ciclu, je - 10 cicli la BP failed, je - 1 ciclu dacă a fost prezis corect , j - 1 ciclu , add/sub 4 cicli
- B = mașină cu pipeline în 5 stagii(IF,ID,E,M,W), 1 ciclu/stagiu
  În cazul hazardelor de date, procesorul introduce ‘bubbles’.

Se presupune că [n] = 7.
Se cer următoarele:

A. **(0.5p)** Ce valoare se afișează?
B. **(3.5p)** Pentru inputul de mai sus să se evidențieze prin ce stări trece branch-predictorul procesorului A, semnalându-se și care branch-uri au fost prezise corect, respectiv incorect.
   Câți cicli durează execuția programului pe procesorul A?
C. **(4.5p)** Evidențiați execuția instrucțiunilor în pipeline pentru prima iterație pe procesorul B(ex: cu ajutorul unei diagrame pipeline). Cum se modifică distribuția hazardelor după prima iterație?
   Câți cicli durează execuția programului pe procesorul B?
D. **(1.5p)** Presupunând că procesorul A rulează la 500 MHz și procesorul B la 400Mhz câte microsecunde durează execuția pe fiecare procesor.

Problemă CN1
11 iunie

Fie următorul cod în assembly:

```
mov ax, $a
mov bx, $b
mov cx, $c
loop:   cmp cx, 0
   je out
   add ax, 2
   add bx, 3
   mov dx, ax
   mov ax, bx
   mov bx, dx
   dec cx
   jmp loop

out: ;se afiseaza conținutul lui ax si bx
```

Procesorul ce execută acest cod rulează la frecvența de **1Ghz**. Metoda de *branch-prediction* folosită de fiecare este 1-bit counter (taken/not taken). Predictorul pornește cu o abordare pesimistă, counter-ul/state-machine-ul având starea inițială *not taken*. Predictorul este unul local, astfel că există un counter/state-machine pentru fiecare branch.

Se știu numărul de cicli de ceas pentru fiecare tip de instrucțiune: acces la memorie (**mov**) - 2 cicli, aritmetice/logice (**cmp**, **add**, **dec**) - 1 ciclu, de salt (**je**, **jmp**) - 40 cicli (dacă nu a fost prezis corect) sau 10 cicli (dacă a fost prezis corect).

Se știu **$a = 10**, **$b = 4**, **$c = 5**. Se cer următoarele:

1. Ce valori se afișează? **(1p)**
2. Să se evidențieze prin ce stări trec branch-predictor-ele semnalându-se și care branch-uri au fost prezise corect, respectiv incorect. **(4p)**
3. Să se calculeze timpul de rulare al programului. **(2p)**
4. Dacă ați putea îmbunătăți viteza de rulare a unei singure categorii de instrucțiuni (aritmetice/logice, acces la memorie sau de salt) astfel încât timpul de execuție al acelei categorii să scadă la jumătate, ce categorie ați alege pentru a obține cel mai bun speed-up în cazul programului vostru? Cât este acest speed-up? **(3p)**

Început la	vineri, 19 iunie 2020, 09:01
Status	Terminat
Completat la	vineri, 19 iunie 2020, 09:41
Timp luat	40 min 1 sec
Notează	**10,00** din maxim 10,00 (**100**%) posibil

1 întrebare

Corect

Marcat 3,00 din 3,00

Fie următorul cod în assembly (x86). Indentificați hazardele de date. Pentru fiecare menționați intrucțiunile implicate, arătând cauzele fiecărui hazard.

Treceți hazardele în ordinea crescătoare a instrucțiunilor.

Exemplu: Dacă există:

- **un hazard de tipul RAW între liniile x și y (x < y) din cauza registrului eax**
- **un hazard de tipul RAW între liniile x și z (x < z, y < z) din cauza registrului eax**
- **un hazard de tipul RAW între liniile t și z (x < t, t < z) din cauza registrului eax**

atunci veți scrie:

Ix - Iy: RAW (eax)

Ix - Iz: RAW (eax)

It - Iz: RAW (eax)

0: mov ebx, edx ; ebx = edx

1: add ecx, ebx ; ecx = ecx + ebx

2: mov eax, [0xCAFEBABE] ; eax = MEM[0xCAFEBABE]

3: mov edx, eax ; edx = eax

4: xor edi, eax ; edi = edi ^ eax

Procesor single-core:

I 0 ✔ - I 1 ✔ : RAW ✔ (ebx ✔)

I 2 ✔ - I 3 ✔ : RAW ✔ (eax ✔)

I 2 ✔ - I 4 ✔ : RAW ✔ (eax ✔)

In plus, la multi-core:

I 0 ✔ - I 3 ✔ : WAR ✔ (edx ✔)

I Nu mai exista ✔ - I Nu mai exista ✔ : Nu mai exista ✔ (Nu mai exista ✔)

I Nu mai exista ✔ - I Nu mai exista ✔ : Nu mai exista ✔ (Nu mai exista ✔)

Vom puncta hazardele identificate corect chiar daca nu au fost puse in ordinea crescatoare a numarului instructiunii.

2 întrebare

Corect

Marcat 7,00 din 7,00

Avem un procesor cu frecvența de **3GHz**, care execută acest cod.

Știm numărul de cicli de ceas pentru fiecare tip de instrucțiune:

- operațiile cu date și memorie (**mov**): **3 cicli**;
- operațiile aritmetice/logice (**cmp, add, inc, dec, xor, and, test, shl**): **1 ciclu**;
- operațiile de salt necondiționat (**jmp**): **5 cicli**;
- operațiile de salt condiționat (**jg, jl, je/jz, jge**): **30 cicli** (dacă nu a fost prezis corect) sau **5 cicli** (dacă a fost prezis corect);
- afisarea: **1 ciclu**.

Fie procesorul B care implementează un branch predictor **global**, iar metoda de branch prediction folosită este **2-bit counter** (strongly taken/weak taken/weak not taken/strongly not taken). Starea inițială este **strongly not taken**.

Se consideră inputul **a = 18**, **b = 1**.

1. Ce valoare afișează programul?

✖

2. Enumerați prin ce stări trece branch predictorul, semnalând dacă branch-ul a fost prezis corect sau greșit.

	Iteratia 1	Iteratia 2	Iteratia 3	Iteratia 4	Iteratia 5
`xor ecx, ecx`	ecx: 0 ✔ 1 ✔				
`mov eax, $a`	eax: 18 ✔ 3 ✔				
`mov ebx, $b`	ebx: 1 ✔ 3 ✔				
`count_loop:`					
`test eax, ebx`	1 ✔	1 ✔	1 ✔	1 ✔	1 ✔
`jz no_inc`	30 ✔ Gresit ✔ WNT ✔	5 ✔ Corect ✔ SNT ✔	30 ✔ Gresit ✔ SNT ✖	5 ✔ Corect ✔ WT ✖	30 ✔ Gresit ✔ SNT ✖
`inc ecx`	ecx: 0 ✔ Nu se executa ✔	ecx: 1 ✔ 1 ✔	ecx: 0 ✔ Nu se executa ✔	ecx: 0 ✔ Nu se executa ✔	ecx: 1 ✖ 1 ✔
no_inc:					
`shl ebx, 1`	ebx: 2 ✔ 1 ✔	ebx: 4 ✔ 1 ✔	ebx: 8 ✔ 1 ✔	ebx: 16 ✔ 1 ✔	ebx: 32 ✔ 1 ✔
`test ebx, ebx`	1 ✔	1 ✔	1 ✔	1 ✔	1 ✔

	Iteratia 1	Iteratia 2	Iteratia 3	Iteratia 4	Iteratia 5
	5 ✔	5 ✔	30 ✔	30 ✔	30 ✔
`je done`	Corect ✔	Corect ✔	Gresit ✔	Gresit ✔	Gresit ✔
	SNT ✔	SNT ✔	SNT ✖	WNT ✖	SNT ✖
`cmp eax, ebx`	1 ✔	1 ✔	1 ✔	1 ✔	1 ✔
	30 ✔	30 ✔	30 ✔	30 ✖	5 ✔
`jge count_loop`	Gresit ✔	Gresit ✔	Gresit ✔	Gresit ✖	Corect ✔
	WNT ✔	WNT ✔	WNT ✖	WNT ✖	WNT ✖

done:

`;se afiseaza conținutul lui ecx (trebuie trecuti cicli)`	Nu se executa ✔	Nu se executa ✔	Nu se executa ✔	Nu se executa ✔	1 ✔

3. Câte branch-uri au fost prezise corect?

6 ✔

4. Câte branch-uri au fost prezise incorect?

9 ✔

5. Care este timpul de rulare al programului (in ns)?

✖

Din cauza multiplelor probleme cu acest subiect (Branch Predictor), va fi acordat punctajul intreg.

Comentariu:

◄ Examen - Partea I Salt la... Model Colocviu ►

- Registrul Program Counter genereaza adresa instructiunii curente din memorie.
R: Adevărat

- Unitatea de comanda este responsabila pentru generarea adresei instructiunii urmatoare din program.
R: Fals

- Ce numar va fi afisat la finalul executiei urmatorului program SAP-1?

```
0H: LDA 5H
1H: ADD 7H
2H: SUB 6H
3H: OUT
4H: HLT
5H: 0x03
6H: 0x02
7H: 0x05
```

R: 0x06

- Arhitectura CISC presupune secventierea executiei instructiunilor in mai multi pasi.
R: Adevărat

- Orice procesor va secventia executia instructiunilor in 6 pasi (T1, T2, .. T6).
R: Fals

- Numarul -1.5 este reprezentat in IEE754 pe 32 ca:
R: 0xbfc00000

- Comparatia de mai jos este adevarata pentru orice valoare a variabilei f, declarata initial ca float:
f == (float)(double) f
R: Adevărat

- Functia divide1() se executa in medie mai rapid decat divide2() pe o arhitectura x86.

```
uint32_t divide1(uint32_t a, uint32_t b){
    return a / b;
}
uint32_t divide2(uint32_t a, uint32_t b){
   double da = (double) a;
   double db = (double) b;
   double q = da/db;
   return (uint32_t) q;
}
```

R: Fals

- 2/3 == 2/3.0
R: Fals

- Pentru orice valofi ale lui double d, urmatoarea expresie este adevarata: d == (float) d
R: Fals

- O instructiune cu operand imediat are prioritate in executie fata de restul instructiunilor.
R: Fals

- O arhitectura RISC executa majoritatea instructiunilor in un ciclu de ceas.
R: Adevărat

- Unitatea de comanda a unui procesor in arhitectura CISC este mult mai complexa decat unitatea de comanda a unui procesor in arhitectura RISC.
R: Adevărat

- Un procesor petrece 50% din timp in operatii cu memoria. Care este speedup-ul obtinut daca timpul de acces la memorie este imbunatatit de 4 ori?
R: 1,6

- Un program contine 30% operatii cu intregi, 20% operatii floating point si 40% operatii cu memoria. Care este speedup-ul obtinut daca masina pe care ruleaza acest program primeste un upgrade al procesorului ce-i permite executia de 3x mai rapida a operatiilor pe intregi si de 2x al operatiilor in flp, dar acest lucru duce cu sine la scaderea de 2x a timpilor de acces la memorie.
R: 0,91

- Un program contine 20% instructiuni cu intregi ce se executa in 1 ciclu de ceas, 30% instructiuni in virgula mobila ce se executa in 3 cicli de ceas si 10% instructiuni conditionale ce necesita 2 cicli de ceas pentru executie. Restul instructiunilor se executa in 1 ciclu de ceas. Care este CPI-ul mediu?
R: 1,7

- Un program contine doua clase de instructiuni A si B si este rulat pe doua masini de calcul diferite M1 si M2.
Pe M1, clasa A are un CPI de 2 si clasa B un CPI de 3.
Pe M2, clasa A are un CPI de 3 si clasa B un CPI de 1.
Care este procentajul instructiunilor de clasa A in program daca CPI-ul mediu in urma rularii pe masina M2 este de 2 ori mai mic decat CPI-ul mediu in urma rularii programului pe masina M1?
R: 20

- Procesoarele CISC disipa in general mai putina caldura decat procesoarele RISC.
R: Fals

- Un set de instructiuni RISC va tinde in medie sa minimizeze numarul de instructiuni dintr-un program.
R: Fals

- In etapa de Write Back sunt scrise in memoria RAM rezultatele instructiunii curente.
R: Fals

- Instructiunile Load si Store pentru RISC V presupun calculul adresei prin adunarea unui deplasament la adresa de baza.
R: Adevărat

- Un procesor in b.a. executa instructiuni intr-un singur ciclu de ceas, dar dupa fiecare instructiune va insera doua bule in banda de asamblare. Care este CPI efectiv?
R: 3

- Banda de asamblare mareste performanta prin micsorarea numarului de cicli pe instructiune.
R: Fals

- Un hazard structural poate fi rezolvat prin bypassing.

R: Fals

- Hazardurile de control se produc doar in cazul instructiunilor de salt conditional.
R: Fals

Început la	vineri, 19 iunie 2020, 08:08
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:43
Timp luat	35 min 24 secs
Notează	**7,00** din maxim 10,00 (**70**%) posibil

1 întrebare

Incorect

Marcat 0,00 din 1,00

Dacă îmbunătățim timpul de acces la memorie, atunci în ce sens putem opera asupra frecvenței procesorului?

Alegeți o opțiune:

- a. **Se apropie de 3GHz**
- b. **Scădere**
- c. **Creștere**
- d. **Nu se poate determina** ✖
- e. **Rămâne neschimbată**

Răspunsul corect este: **Creștere**

2 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor petrece 60% din timp cu operații UAL. De câte ori mai rapidă trebuie să fie unitatea aritmetică logică pentru ca procesorul să ruleze de două ori mai repede?

Alegeți o opțiune:

- a. **5**
- b. **3**
- c. **2**
- d. **1.5**
- e. **6** ✔

Răspunsul corect este: **6**

3 întrebare

Corect

Marcat 1,00 din 1,00

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	HLT	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **9** ✔
- b. **Nu se poate determina**
- c. **B**
- d. **A**
- e. **8**

Răspunsul corect este: **9**

4 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor cu arhitectura Harvard:

Alegeți o opțiune:

- a. **Toate de mai sus**
- b. **Are memorii de date si instructiuni separate** ✔
- c. **Executa orice operatie intr-un singur ciclu de ceas**
- d. **Are o banda de asamblare cu cinci etape**

Răspunsul corect este: **Are memorii de date si instructiuni separate**

5 întrebare

Corect

Marcat 1,00 din 1,00

Hazardele de date:

Alegeți o opțiune:

- a. **Au latenta minima**
- b. **Sunt cauzate de UAL**
- c. **Apar doar la arhitecturile RISC**
- d. **Pot fi solutionate prin bypassing** ✔

Răspunsul corect este: **Pot fi solutionate prin bypassing**

6 întrebare

Corect

Marcat 1,00 din 1,00

Pentru a putea rula la 0.5GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 2V, iar curentul de mers în gol al acestuia este 10mA. Știind că procesorul are o capacitate parazită echivalentă de 200nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:

- a. **200.02W** ✔
- b. **200W**
- c. **100.1W**
- d. **100.01W**
- e. **50W**

Răspunsul corect este: **200.02W**

7 întrebare

Incorect

Marcat 0,00 din 1,00

Dacă CPU-ul este implementat în logică rapidă, în timp ce accesul la memorie este de cel puțin 10x mai lent, ce fel de ISA alegi pentru a contracara pe cât de mult posibil limitările hardware?

Alegeți o opțiune:

- a. **Mașină virtuală Java**
- b. **VLIW**
- c. **CISC**
- d. **RISC** ✖
- e. **MISC**

Răspunsul corect este: **CISC**

8 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că un procesor fără b.a. execută următoarea operație: *ldi r16, 0xdeadc0de*. În care etapă este actualizat *r16*?

Alegeți o opțiune:

- a. **Instruction Fetch**
- b. **Decode**
- c. **Memory**
- d. **Execute**
- e. **Write Back** ✔

Răspunsul corect este: **Write Back**

9 întrebare

Corect

Marcat 1,00 din 1,00

Numărul 2.25, poate fi reprezentat în cod IEEE 754 single precision ca:

Alegeți o opțiune:

- a. **0x40100000** ✔
- b. **0x40200000**
- c. **0x40600000**
- d. **Nicio opțiune**
- e. **0x40800000**

Răspunsul corect este: **0x40100000**

10 întrebare

Incorect

Marcat 0,00 din 1,00

Adevarat sau fals: Un procesor CISC este în medie mai lent decât un procesor RISC.

Selectați o opțiune:

- Adevărat ✖
- Fals

Răspunsul corect este "Fals".

◄ Inregistrare curs 11&12

Salt la...

Examen - Partea II ►

Examen CN1 - Partea II - 12.06.2017

Nume: …….. Grupa: …….

Fie următorul cod în assembly:

```
mov eax, 0
mov ecx, $n
mov esi, 0
mov edi, 0
process_element:
      cmp ecx, 0
      je pwp
      mov al, byte [v + ecx - 1]
      cmp al, byte [X]
      jg add_greater
      jl add_less
      continue_loop:
            dec ecx
            jmp process_element
add_greater:
      add esi, eax
      jmp continue_loop
add_less:
      add edi, eax
      jmp continue_loop
pwp: ; Print esi, edi
```

Procesorul ce execută acest cod rulează la frecvența de **1Ghz**. Metoda de *branch-prediction* folosită de fiecare este 1-bit counter (taken/not taken), starea inițială fiind *not taken*. Predictorul este unul local, astfel încât există un state-machine pentru fiecare branch.

Stim numărul de cicli de ceas pentru fiecare tip de instrucțiune: operațiile cu date și memorie (**mov**) durează 2 cicli, operațiile aritmetice/logice (**cmp**, **add**, **dec**) durează 1 ciclu, operațiile de salt necondiționat (**jmp**) durează 10 cicli, operațiile de salt condiționat (**jg, jl, je**) durează 40 cicli (dacă nu a fost prezis corect) sau 10 cicli (dacă a fost prezis corect).

Considerăm **$n=7** (dword), **$X=2** (byte), iar v este adresa de început a vectorului de octeți (1, 3, 3, 3, 1, 1, 3). Se cer următoarele:

1. **(1p)** Ce valori se afișează in labelul **pwp**.
2. **(5p)** Să se evidențieze prin ce stări trec branch-predictor-ele semnalându-se și care branch-uri au fost prezise corect, respectiv incorect.
3. **(2p)** Să se calculeze timpul de rulare al programului.
4. **(2p)** Se face următorul experiment:

n = 10, v = (1, 4, 4, 2, 3, 4, 5, 1, 1, 1), X = 2 ⇒ un timp de rulare t1.
n = 10, v = (1, 1, 1, 1, 2, 3, 4, 4, 4, 5), X = 2 ⇒ un timp de rulare t2 (t2 mult mai mic decât t1).
Ce proprietate al celui de-al doilea vector a dus la scăderea timpului de execuție? Justificați acest speed-up.

Rezolvare

TODO: Rezolvare

Examen CN1 - Partea II - 16.06.2017

Nume: …….. Grupa: ……..

Fie următorul cod în assembly

```
                mov ax, $a
                mov bx, $b

_loop:          cmp ax, bx
                jg _greater
                jl _less
                je _final

_greater:       sub ax, bx
                jmp _loop

_less:          sub bx, ax
                jmp _loop

_final:         ; se afiseaza conținutul lui ax
```

Avem 2 procesoare, fiecare rulând la frecvența de **1Ghz**, care execută acest cod, procesoarele A și B. Metoda de branch-prediction folosită de procesorul A este **1-bit counter** (taken/not taken), iar metoda folosită de procesorul B este **2-bit counter** (strongly taken/weakly taken/weakly not taken/strongly not taken). Ambele pornesc cu o abordare pesimistă, counter-ul/state-machine-ul având starea inițială not taken, pentru procesorul A, respectiv strongly not taken, pentru B. Ambele implementează un branch-predictor global, astfel încât există un singur counter/state-machine global, pentru toate branch-urile.

Se știu numărul de cicli de ceas pentru execuția fiecărui tip de instrucțiune: operațiile cu date și memorie (**mov**) durează 2 cicli, operațiile aritmetice/logice (**cmp**, **sub**) durează 1 ciclu, operațiile de salt necondiționat (**jmp**) durează 10 cicli, operațiile de salt condiționat (**jg, jl, je**) durează 40 cicli (dacă nu a fost prezis corect) sau 10 cicli (dacă a fost prezis corect).

Fie **$a = 26**, **$b = 32**. Se cer următoarele:

1. **(4p)** Să se evidențieze prin ce stări trec branch-predictor-ele celor 2 procesoare, semnalându-se și care branch-uri au fost prezise corect, respectiv incorect.
2. **(3p)** Să se calculeze timpul de rulare al programului pe fiecare dintre cele 2 procesoare.
3. **(3p)** FIe un procesor C, rulând la frecvența de **1Ghz**, care folosește metoda de branch-prediction **2-bit counter**, implementând însă un branch-predictor local (câte un counter/state-machine pentru fiecare branch). Să se calculeze timpul de rulare al programului pe procesorul C. Care este speed-up-ul față de A și B? Argumentați ce a determinat acest speed-up.

Rezolvare

TODO: rezolvare

Început la	vineri, 19 iunie 2020, 08:08
Status	Terminat
Completat la	vineri, 19 iunie 2020, 08:46
Timp luat	38 min
Notează	**9,00** din maxim 10,00 (**90**%) posibil

1 întrebare

Corect

Marcat 1,00 din 1,00

Ce afișează următorul program SAP-1?

0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
LDA 8H	SUB 9H	ADD AH	SUB BH	OUT	HLT	0xDE	0xCE	0x10	0x04	0x03	0x06	0xCA	0xFE	0xBA	0xBE

Alegeți o opțiune:

- a. **9** ✔
- b. **Nu se poate determina**
- c. **8**
- d. **A**
- e. **B**

Răspunsul corect este: **9**

2 întrebare

Corect

Marcat 1,00 din 1,00

Adevarat sau fals: Procesorul cu banda de asamblare executa toate instructiunile intr-un singur ciclu de ceas.

Selectați o opțiune:

- Adevărat
- Fals ✔

Răspunsul corect este "Fals".

3 întrebare

Incorect

Marcat 0,00 din 1,00

Dacă îmbunătățim timpul de acces la memorie, atunci în ce sens putem opera asupra frecvenței procesorului?

Alegeți o opțiune:

- a. **Rămâne neschimbată**
- b. **Nu se poate determina** ✖
- c. **Se apropie de 3GHz**
- d. **Creștere**
- e. **Scădere**

Răspunsul corect este: **Creștere**

4 întrebare

Corect

Marcat 1,00 din 1,00

De ce este important sa stim daca o masina este big-endian sau little-endian?

Alegeți o opțiune:

- a. **Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare**
- b. **Pentru ca anumite programe scriu date in memorie intr-o anumita ordine**
- c. **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare** ✔
- d. **Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers**

Răspunsul corect este: **Pentru ca partajarea fisierelor si informatiei intre doua calculatoare poate cauza erori de intrepretare**

5 întrebare

Corect

Marcat 1,00 din 1,00

Numarul 0xC000FEFE, stocat ca *signed int (32 de biți)* este:

Alegeți o opțiune:

- a. **Nicio opțiune**
- b. **Pozitiv**
- c. **Prea mare pentru a putea fi reprezentat**
- d. **Negativ** ✔

Răspunsul corect este: **Negativ**

6 întrebare

Corect

Marcat 1,00 din 1,00

Care este, în medie, efectul pe care dublarea numărului de registre generale îl are asupra ratei apariției hazardelor de date?

Alegeți o opțiune:

- a. **Va crește**
- b. **Ramâne constantă**
- c. **Va scade** ✔
- d. **Imposibil de determinat**
- e. **Le elimină complet**

Răspunsul corect este: **Va scade**

7 întrebare

Corect

Marcat 1,00 din 1,00

Presupuneți că un procesor în b.a. execută următoarea operație: *addi r1, 0xbeef*. În ce etapă este actualizat *r1*?

Alegeți o opțiune:

- a. **Execute**
- b. **Decode**
- c. **Memory**
- d. **Write Back** ✔
- e. **Instruction Fetch**

Răspunsul corect este: **Write Back**

8 întrebare

Corect

Marcat 1,00 din 1,00

Dacă trebuie să proiectați un procesor performant (frecvență mare de ceas, superscalar) folosind tehnologia actuală, ce fel de ISA ați alege pentru acesta?

Alegeți o opțiune:

- a. **Mașină virtuală Java**
- b. **RISC** ✔
- c. **MISC**
- d. **CISC**
- e. **VLIW**

Răspunsul corect este: **RISC**

9 întrebare

Corect

Marcat 1,00 din 1,00

Un procesor petrece 40% din timp cu operații UAL, 30% în operații cu memoria și 30% din timp cu alte operații. Dacă puteți îmbunătăți performanța UAL-ului de 5 ori, accesul la memorie de 10 ori dar, ca rezultat veți încetini execuția celorlalte instrucțiuni cu 30%, ce speedup obțineți?

Alegeți o opțiune:

- a. **2** ✔
- b. **1.5**
- c. **3.33**
- d. **5**
- e. **10**

Răspunsul corect este: **2**

10 întrebare

Corect

Marcat 1,00 din 1,00

Pentru a putea rula la 1GHz, procesorul unui calculator trebuie să fie alimentat la o tensiune de 1V, iar curentul de mers în gol al acestuia este 50mA. Știind că procesorul are o capacitate parazită echivalentă de 100nF, care este puterea necesară pentru ca acesta să funcționeze?

Alegeți o opțiune:

- a. **100.1W**
- b. **50.05W** ✔
- c. **50W**
- d. **100.05W**
- e. **100W**

Răspunsul corect este: **50.05W**

◄ Inregistrare curs 11&12 Salt la... Examen - Partea II ►

9 iunie

ex 3

1 si 2

			cicli						**stare3**	
ax	**bx**	**instructiune**	**cpu1**	**cpu2**	**stare1**	**stare2**	**cpu3**	**jg _greater**	**jl _less**	**je _final**
26		mov ax 26	1	1	NT	SNT	1	SNT	SNT	SNT
	32	mov bx 32	1	1			1			
		cmp ax bx	1	1			1			
		jg _greater	2	2			2			
		jl _less	20	20	T	WNT	20		WNT	
	8	sub bx,ax	1	1			1			
		jmp	2	2			2			
		cmp ax bx	1	1			1			
26	8	jg _greater	2	20		WT	20	WNT		
18		sub ax,bx	1	1			1			
		jmp	2	2			2			
18	8	cmp ax bx	1	1			1			
		jg _greater	2	2		ST	20	WT		
10		sub ax,bx	1	1			1			
		jmp	2	2			2			
10	8	cmp ax bx	1	1			1			
		jg _greater	2	2			2	ST		
2		sub ax,bx	1	1			1			
		jmp	2	2			2			
2	8	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	20	WT		
		jl _less	20	2	T	ST	20		WT	
	6	sub bx,ax	1	1			1			
		jmp	2	2			2			
2	6	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	20	WNT		
		jl _less	20	2	T	ST	2		ST	
	4	sub bx,ax	1	1			1			
		jmp	2	2			2			
2	4	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	2	SNT		
		jl _less	20	2	T	ST	2		ST	
	2	sub bx,ax	1	1			1			
		jmp	2	2			2			
2	2	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	2	SNT		
		jl _less	2	20		WNT	20		WT	
		je _final	20	20	T	WT	20			WNT
2	2	print ax	1	1			1			
FINAL			#NAME? ns	#NAME? ns	T	WT	#NAME? ns	SNT	WT	WNT

3) speedup este 0

9 iunie

ex 3

1 si 2

			cicli						**stare3**	
ax	**bx**	**instructiune**	**cpu1**	**cpu2**	**stare1**	**stare2**	**cpu3**	**jg _greater**	**jl _less**	**je _final**
26		mov ax 26	1	1	NT	SNT	1	SNT	SNT	SNT
	32	mov bx 32	1	1			1			
		cmp ax bx	1	1			1			
		jg _greater	2	2			2			
		jl _less	20	20	T	WNT	20		WNT	
	8	sub bx,ax	1	1			1			
		jmp	2	2			2			
		cmp ax bx	1	1			1			
26	8	jg _greater	2	20		WT	20	WNT		
18		sub ax,bx	1	1			1			
		jmp	2	2			2			
18	8	cmp ax bx	1	1			1			
		jg _greater	2	2		ST	20	WT		
10		sub ax,bx	1	1			1			
		jmp	2	2			2			
10	8	cmp ax bx	1	1			1			
		jg _greater	2	2			2	ST		
2		sub ax,bx	1	1			1			
		jmp	2	2			2			
2	8	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	20	WT		
		jl _less	20	2	T	ST	20		WT	
	6	sub bx,ax	1	1			1			
		jmp	2	2			2			
2	6	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	20	WNT		
		jl _less	20	2	T	ST	2		ST	
	4	sub bx,ax	1	1			1			
		jmp	2	2			2			
2	4	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	2	SNT		
		jl _less	20	2	T	ST	2		ST	
	2	sub bx,ax	1	1			1			
		jmp	2	2			2			
2	2	cmp ax bx	1	1			1			
		jg _greater	20	20	NT	WT	2	SNT		
		jl _less	2	20		WNT	20		WT	
		je _final	20	20	T	WT	20			WNT
2	2	print ax	1	1			1			
FINAL			#NAME? ns	#NAME? ns	T	WT	#NAME? ns	SNT	WT	WNT

3) speedup este 0

Teorie Examen CN

1. De ce sunt folosite tranzistoare MOSFET pentru procesoarele moderne?
   a) **Consumul de putere este mai mic decat la tranzistoarele bipolare**
   b) Au un factor de amplificare mai mare
   c) Pot sa aiba doar doua stari: 1 si 0 logic
   d) Toate de mai sus

2. Adevarat sau fals: Procesorul cu banda de asamblare executa toate instructiunile intr-un singur ciclu de ceas (aici as zice fals pt ca daca le-ar executa pe toate intr-un ciclu nu ar mai avea ce tipuri de instructiuni sa separe a.i. sa se execute in paralel) **FALS**

3. De ce este important sa stim daca o masina este big-endian sau little-endian?
   a) Pentru ca anumite programe scriu date in memorie intr-o anumita ordine (toate scriu
   b) dupa cum vrea procesoru; ma rog, daca esti tu smecher si faci un program c care sa iti scrie rezultatele in ce endianness vrei tu e altceva)
   c) Pentru ca operatiile aritmetice vor genera rezultate gresite daca procesorul citeste numerele invers (asta ar putea fi, dar procesorul nu o sa adune el gresit, el aduna corect dar interpreteaza numerele invers)
   d) **Pentru ca partajarea folderelor si informatiei intre doua calculatoare poate cauza erori de interpretare - cred ca asta e**
   e) Pentru ca o eroare de citire inversa poate cauza bule in banda de asamblare

4. Numarul 0xC000FEFE(covfefe lel), stocat ca signed int este:
   a) Prea mare pentru a putea fi reprezentat (are 32 biti deci nu e prea mare)
   b) Poztiv (C e 1100 deci incepe cu 1 si e negativ)
   c) Impar (E este 1010 deci se termina cu 0 si e par)
   d) **Nicio optiune**

5. Presupuneti ca aveti de ales intre doua operatii echivalente= {op1}:mul r1, 4 si {op2}: shl r1,2. Din punctul de vedere al timpului de executie, pentru majoritatea procesoarelor:
   a) **t(op1) > t(op2)** **Explicatie: shiftarea e mereu mai rapida**
   b) t(op1) < t(op2)
   c) Ambele dureaza la fel
   d) Nu poate fi determinat

6 . Un procesor petrece 50% din timp cu operatii UAL si 40% in operatii cu memoria. Daca puteti imbunatati timpul de acces la memorie de 4 ori, dar, ca rezultat veti incetini executia celorlate instructiuni cu 50%, ce speedup obtineti?

a) 5.3
b) 1 Explicatie: S = 1 / (0.6 * 1.5 + 0.4 / 4) = 1 / 1 = 1
c) 2
d) 3.33
e) Nu poate fi determinat

7. Hazardele de date:

a) Sunt cauzate de UAL
b) Pot fi solutionate prin bypassing
c) Au latenta minima
d) Apar numai la RISC

8 . De ce se foloseste deseori pentru procesoarele RISC denumirea de “arhitecturi load/store”?

Arhitectura load/store are caracteristica de a imparti instructiunile in 2 categorii: operatii cu memoria si operatii cu UAL. Arhitectura procesoarelor RISC face posibile operatiile de orice fel doar intre valori care se afla in registrii(operanzii trb sa fie adusi intr-un registru pentru a putea fi folositi), spre deosebire de CISC(operatiile memorie-registru sunt posibile).

9. Dati cel putin trei caracteristici ale ISA pentru un procesor RISC.

Complet, de mici dimensiuni, open source.s

10. Este obligatorie ca o exceptie sa fie tratata de catre procesor atunci cand se produce? Este acelasi lucru valabil si pentru intreruperi? Justificati raspunsurile voastre.

Exceptie este o intrerupere sincrona: tratata dupa terminarea rutinei actuale, lanseaza rutina de tratare dupa ce a golit toata banda de asamblare(exceptia este o intrerupere sincrona)

Intreruperi asioncrone: este tratata in momentul in care e detectata, se salveaza current state-ul, se executa rutina de tratare, se revine in current state(intrerupere asincrona)

Examen CN1 11.09.2015

1 . Descrieti functionalitatea unui circuit logic programabil de tipul PLA sau PAL (schema si scurta explicatie a functionalitatii).

Un circuit combinaţional programabil poate să înlocuiască funcţionalitatea mai multor porţi logice. Programate prin întreruperea unor conexiuni existente (fuses) sau stabilirea de noi conexiuni(antifuses).

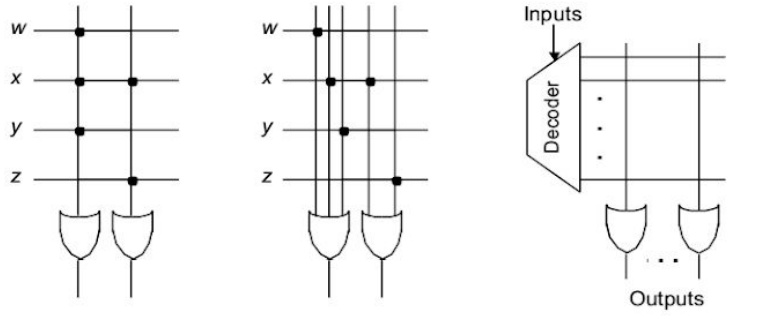
Logică combinațională programabilă

Un circuit combinațional programabil poate să înlocuiască funcționalitatea mai multor porți logice

Programate prin întreruperea unor conexiuni existente (*fuses*) sau stabilirea de noi conexiuni(*antifuses*)

- Programmable ROM (PROM)
- Programmable array logic (PAL)
- Programmable logic array (PLA)

Circuite PROM



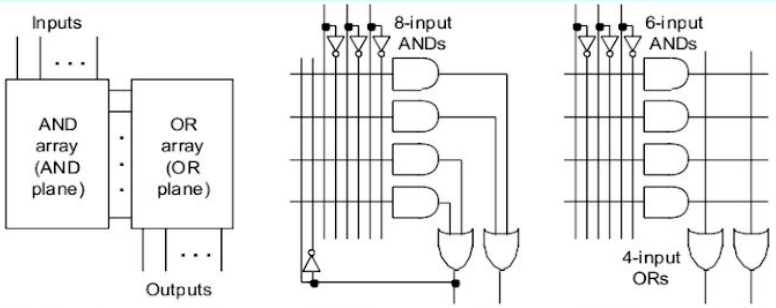
(a) Programmable OR gates

(b) Logic equivalent of part a

(c) Programmable read-only memory (PROM)

Conexiuni programabile și folosirea lor într-un circuit PROM.

Circuite PAL & PLA



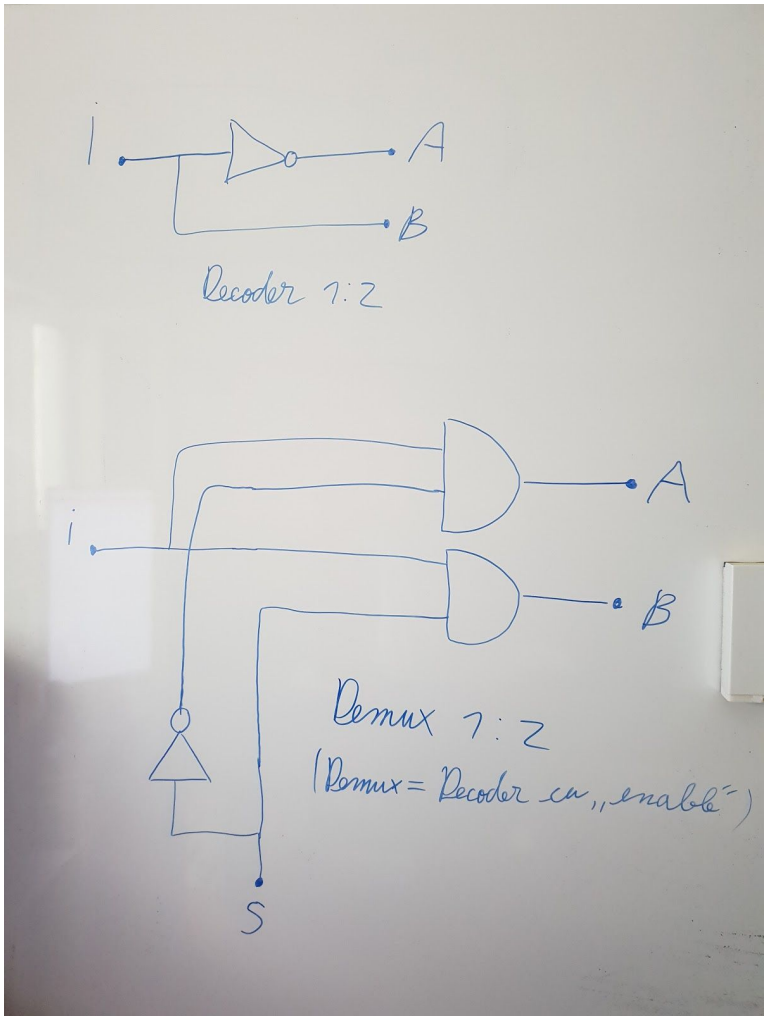
(a) General programmable combinational logic

(b) PAL: programmable AND array, fixed OR array

(c) PLA: programmable AND and OR arrays

Logică combinațională programabilă: structură generală și două clase de circuite (PAL și PLA). În același circuit este nevoie și de un PROM cu un plan AND fix (rol de decodificator) și un plan OR programabil.

2. Proiectati un circuit de tip decodifier 1:2. Aveti la dispozitie numai parti logice de tip NOT, AND si OR cu doua intrati.
https://www.allaboutcircuits.com/textbook/digital/chpt-9/decoder/



3. Un procesor petrece 30% din timp cu adunari flp, 60% cu inmultiri flp si 5% cu impartiri flp. Evaluati imbunatariea performantei pentru:

a) Sumatorul flp este de 3 ori mai rapid
   S = 1 / (0.3/3 + 0.6 + 0.05 + 0.05) = 1/0.8 = 1.25
b) Multiplicatorul flp este de 2 ori mai rapid
   S = 1 / (0.3 + 0.6/2 + 0.05 + 0.05) = 1/0.7 = 1.43
c) Unitatea de impartire flp este de 5 ori mai rapid
   S = 1 / (0.3 + 0.6 + 0.05/5 + 0.05) = 1/0.96 = 1.04

4. Este vizibila, din perspectiva software-ului, diferenta dintre o masina CISC si una RISC? Explicati.

5. Descrieti cei 5 pasi din executia unei instructiuni (fetch, decode, UAL, MEM, WB).

6. Care este principalul dezavantaj al sumatorului select?

7. Implementati un CLA pe 4 biti. Scrieti ecuatiile pentru P si G si desenati schema acestuia.

8. Care dintre termenii din cadrul Legii de fier se minimizeaza la CISC? Dar la RISC?
CISC: Numarul de instructiuni per program
RISC: Numarul de clocks per instruction

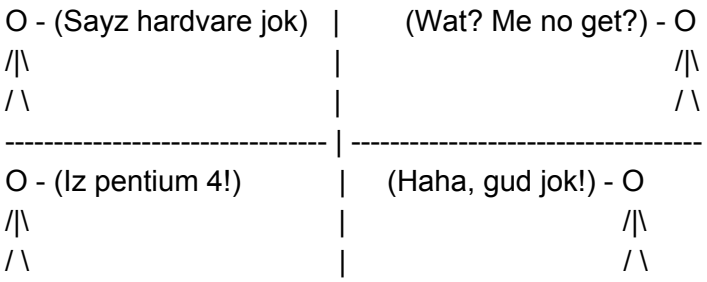
9. De ce sunt preferati tranzistorii MOSFET in locul celor bipolari? Dati 3 argumente.

1 - Consumul de putere este mai mic decat la tranzistoarele bipolare
2 - Un MOSFET e mai tolerant la caldura decat un tranzistor bipolar
3 - MOSFETs sunt mai usor de fabricat (??)
4 - MOSFETs sunt mai usor de scalat
5 - MOSFET - impedanta de intrare mare

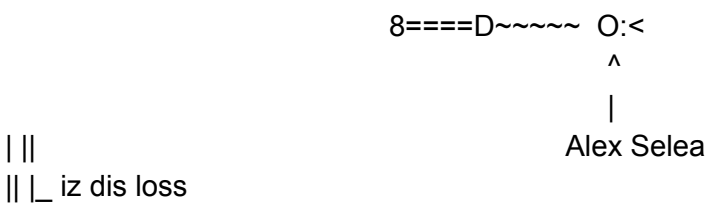
10. Cu tehnologia disponibila in zilele noastre, ce implementare ati alege pentru ISA-ul vostru: CISC sau RISC? Motivati.
Daca se doreste o implementare mai usoara, eficienta din punctul de vedere al memoriei folosite si fara limitari de hardware, putem folosi CISC.
Daca dorim o implementare mai rapida, fara constrangeri de memorie, putem folosi RISC.

+ Alt bullshit, ce iti mai vine prin cap

11. Desenati un comic pentru CN2.



\- Comic by Graet Artiste, 2018



Examen CN1 12.06.2017

1. Un procesor cu arhitectura Harvard:
a) Are memorii de date si instructiuni separate
b) Executa orice operatie intr-un singur ciclu de ceas - nu e bn ca e ciclu masina, nu ciclu ceas
c) Are o banda de asamblare cu cinci etape - idk
d) Toate de mai sus

2. Adevarat sau fals: O intrerupere ne-mascabila se executa intotdeauna de catre procesor.
ADEVARAT

3. Pentru a putea rula la 3GHz, procesorul unui calculator trebuie sa fie alimentat la o tensiune de 2V, iar curentul de mers in gol al acestuia este 100mA. Stiind ca procesorul are o capacitate parazita echivalenta de 10nF, care este puterea necesara pentru ca acesta sa funcioneza?
a) 60.1W b) 60W **c) 60.2W** d) 120.2W e) 120W f) 120.1W

P = 1⁄2 * C * (VDD^2) * f + IDD * VDD
P = ½ * 10 * 10^(-9) * (2^2) * 3 * 10^9 + 100 * 10^(-3) * 2
P = ½ * 10 * 4 * 3 + 0.2
P = 60.2W

4. Numarul 0xCAFEBABE, reprezentat in cod IEEE 754 single precision este:
a) Prea mare pentru a putea fi reprezentat (CAFEBABE are 32 de biti, deci poate fi reprezentat in format IEEE 754)
b) Negativ (C este 1100, deci bitul de semn o sa fie 1, deci numar negativ)
c) Are exponentul negativ (Exponentul este 1001 0101, care este 149 > 127, pozitiv)
d) Nicio optiune

CAFEBABE = 1100 1010 1111 1110 1011 1010 1011 1110
-> primul bit de semn
-> urmatorii 8 biti exponentul
-> restul de biti sunt mantissa

5. Presupuneti ca un procesor in b.a. Executa urmatoareae operatie: addi r1, r1, 0xbeef, in care etapa este actualizat r1?
a) Instruction Fetch
b) Decode
c) Execute
d) Memory
e) Write Back

6. Un procesor petrece 80% din timp cu operatii UAL. Daca puteti imbunatati performanta UAL-ului de 4 ori, dar, ca rezultat veti incetini executia celorlalte instructiuni cu 50%, ce soeedup obtineti?

a) 5.3
b) 3.5
c) 2 **S = 1 / (0.8 / 4 + 1.5 * 0.2) = 1 / (0.2 + 0.3) = 1 / 0.5 = 2**
d) 3.33

7. Hazardele structurale:

a) Sunt intalnite doar la risc
b) Produc bule in b.a.
c) Sunt rezolvate prin multiplicarea resurselor partajate

8. Care este avantajul primar al sumatorului CLA fata de sumatorul ripple-carry?

Sumatorul CLA foloseste o structura auxiliara care poate sa determine unde vor aparea biti de carry, nemaifiind nevoie astfel sa se astepte rezultatul fiecarui sumator de dinainte, lucru care va duce la eficienta mai buna atunci cand este vorba de un numar mare de sumatoare (N > 16).

9. Care sunt pasii pe care un procesor trebuie sa ii execute pentru tratarea unei intreruperi?

Întreruperi asincrone: invocarea rutinei de tratare a întreruperii



- Un dispozitiv I/O cere atenție prin semnalizarea pe o linie dedicată de interrupt request (IRQ)
- Când un procesor decide să proceseze întreruperea
  - Oprește execuția programului curent la instrucțiunea I_i, completând execuția tuturor instrucțiunilor până la I_{i-1} *(întrerupere precisă)*
  - Salvează PC-ul instrucțiunii I_i într-un registru special (EPC)
  - Maschează toate întreruperile și transferă controlul unei rutine speciale pentru tratarea întreruperii (pentru un OS ca Linux/Windows aceasta e situată de obicei în kernel)

Rutina de tratare a întreruperii



- Salvează EPC înainte să permită execuția altor întreruperi (întreruperi imbricate) ⇒
  - Avem nevoie de o instrucțiune pentru a muta EPC în GPR-uri
  - Avem nevoie de un mecanism pentru a masca toate întreruperile până când EPC este salvat
- Trebuie să citească un Registru de Stare care indică ce periferic sau subsistem a cauzat întreruperea
- Folosește o instrucțiune specială de salt indirect RFE (*return-from-exception*) care
  - Demaschează întreruperile
  - Întoarce procesorul înapoi în modul utilizator
  - Reface statusul inițial hardware și starea unității de control

Întreruperi sincrone



- O întrerupere sincronă (excepție) este cauzată de o anumită instrucțiune
- În general, instrucțiunea nu poate fi completată și trebuie să fie restartată după tratarea excepției pe care a generat-o
  - Necesită refacerea efectului unei sau mai multor instrucțiuni executate parțial
- În cazul unui system call trap, se consideră că instrucțiunea și-a incheiat execuția
  - O instrucțiune de jump specială ce face o trecere a procesorului în modul kernel

10. Dati exemplu de un hazard de date de tip load/store si un hazard de control.

- Hazard load/store
  -> RAW
  ```
  Mov r1, r2
  Add r3, r4, r1
  ```
  -> WAR
  ```
  Add r3, r1, r2
  Add r2, r4, r5
  ```

- Hazard de control
  ```
  Cmp eax, ebx ; eax = 100, ebx = 1
  Jg label1
  Jl label2
  ...
  Label1: ...
  Label2:...
  ```

1. Un coleg va propune sa adaugati instructiunea add3 la ISA-ul procesorului de la curs: Add3 r1, r2, r3, r4 -> R[r1] = R[r2] + R[r3] + R[r4]. Se schimba ISA in acest caz? Are aceasta schimbare vreun impact asupra arhitecturii procesorului?

2. Dati cel putin un avantaj si un dezavantaj daca, in loc de instructiuni de lungime fixa(de ex. 32 de biti), am folosi un ISA cu instructiuni de lungime variabila?

3. Dati trei caracteristici ale ISA pentru un procesor de tip RISC.
Lucreaza doar cu registrii
Numar fix de parametrii
Se executa instructiunile mai rapid ca la cisc???

4. Daca aplicam optimizarea B, imbunatatim viteza de executie cu 20% din programul A. Care este speedup-ul maxim teoretic oferit de B?

5. Completati
   Completez
   Eu nu vreau sa completez. Fight me >:(

6. Victor decide sa isi cumpere urmatoarea tableta bazandu-se pe metrica MIPS^3/(Watt^2 * Greutate)
Coaie victor efectiv cauta sa isi ia bataie nu tableta

a) Care sunt caracteristicile pe care Victor le cauta la tableta sa?
- Vrea sa fie puternica, i guess

b) Care sunt caracteristicile pe care Victor doreste sa le evite?

- Vrea sa nu fie grea si sa nu se duca repede bateria

c) Aranjati aceste caracteristici de la cea mai importanta la cea mai putin importanta.

- Pai cu cat e puterea mai mare cu atat e mai importanta. Deci aia cu la a treia e cea mai importanta, dupa aia la a doua, dupa aia simpla

7. Un procesor petrece 60% din timp executand operatii cu UAL-ul si 20% din timp in operatii cu memoria. Daca puteti optimiza de 2 ori viteza de executie a operatiilor cu UAL si de 4 ori viteza celor cu memorie, cvare o sa fie speedup-ul obtinut?

S = 1 / (0.6 / 2 + 0.2 / 4 + 0.2) = 1 / (0.3 + 0.05 + 0.2) = 1/ 0.55 = 1.8

8. Procedeul de bypassing(sau forwarding) elimina hazardele structurale sau cele de date?
Elimina hazardele de date

9. Conform Legii lui Moore, ce fel de rata de crestere in raport cu timpul o are numarul de tranzistoare dintr-un circuit integrat?
Crescatoare (se dubleaza)

10. Urmatorul cod se executa pe un procesor in b.a. cu 5 etape. Care sunt registrele care sunt citite la inceputul celui de-al 5-lea ciclu de ceas si care sunt cele scrise la sfarsitul aceluiasi ciclu de ceas? Luati in calcul numai registrele din tabela de registre generale(ex. R1, R2, R3..etc.)
ADD R1, R2, R3
ADD R4, R5, R6
ADD R7, R8, R9
ADD R10, R11, R12
ADD R13, R14, R15

Vreti sa invatati chitara si pentru asta va trebuie o chitara acordata. Pentru ca nu v-ati antrenat inca urechea incat sa faceti asta singuri, va propuneti sa faceti un tuner automat bazat pe FPGA care va da un feedback legat de cat de "aproape" e o coarda de o nota muzicala corecta.

Aveti la dispozitie: - un FPGA cu 6 LED-uri (de ce 6, explicatii imediat) - 12 valori intregi, care reprezinta frecventele echidistante ale notelor corecte (in ordinea frecventei: A, A#, B, C, C#, D, D#, E, F, F#, G, G#); distanta dintre doua note adiacente este si ea cunoscuta - un modul gata implementat (prin microfon etc) care da frecventa pe care o emite coarda; el arata astfel: module microphone (output frequency[15:0]); - un modul gata implementat module interval closest

(input values[11:0][15:01 input x_val[15:0], output closest_low[15:0], closest_high[15:0], closest[15: 0]); care va da 2 valori din "vectorul" valuescele mai apropiate de valoarea x_val (in scenariul adevarat, x val va fi mereu intre aceste doua valori) si valoarea cea mai apropiata (evident, va fi una dintre closest low si closest high)

Modulul pe care it yeti implementa va arata astfel: module tuner 10; si va contine ca si constante frecventele notelor corecte (va trebui doar sa le declarati ca atare, valorile for efective nu sunt importante in aceasta problema).
Veti afisa pe cele 6 LED-uri cat de aproape este frequency (data de modulul-microfon) de una din constantele pe care le aveti (cea mai apropiata). Unul dintre LED-uri se va aprinde sau nu, cu semnificatia "frecventa corzii este mai mare/mai mica decat frecventa corecta". Celelalte 5 LED-uri se vor aprinde in functie de diferenta de frecventa intre coarda si nota corecta. Frecventa corzii va fi tot timpul intre A si G#.
Exemplu: sa zicem ca vreti sa acordati o coarda pentru nota B, dar deocamdata coarda e mai aproape de A# decat de B. Atunci LED-utile (x = aprins) vor arata, sa zicem, xxxx00 (primul aprins, ca frecventa e mai mare decat A#, cea mai apropiata, iar celelalte 3 indicand ca diferenta e destul de mare). Mai strangeti coarda (LED-urile se vor aprinde pe treptat in configuratiile xxxxx0, xxxxxx, apoi Oxxxxx, Oxxxx0, Oxxx00) si verificati tuner-ul, iar acesta va afisa Oxx000, indicand ca nota e mai jos decat urmatoarea (B), cu o diferenta mai mica. Strangand coarda putin cate putin, cele 2 LED-uri aprinse se vor stinge treptat (0x0000, apoi 000000), indicand ca ati ajuns fix pe nota corecta.
Indicatii: - folositi-va de modulele pe care be aveti la dispozitie, instantiati-le corespunzator - va puteti crea un mic modul auxiliar care va da bitii corespunzatori celor 5 LED-uri care se aprind proportional; de exemplu, primul din ele va fi setat daca diferenta dintre coarda si cea mai apropiata nota e mai mare de 1/5 din distanta_note_adiacente/2 (supra 2 din cauza alegerii notei mai apropiate, diferenta maxima pana la o nota corecta va fi tot timpul mai mica decat distanta_note_adiacente/2) - alegeti-va de la inceput pe ce logica lucrati (secventiala/combinationala) si pastrati-o - comentati si explicati-va metoda

CISC	RISC
Emphasis on hardware	Emphasis on software
Includes multi-clock complex instructions	Single-clock, reduced instruction only
Memory-to-memory: "LOAD" and "STORE" incorporated in instructions	Register to register: "LOAD" and "STORE" are independent instructions
Small code sizes, high cycles per second	Low cycles per second, large code sizes
Transistors used for storing complex instructions	Spends more transistors on memory registers

Felicitari ca ai ajuns pana aici :). Esti pe drumul cel drum in a deveni Hokage. Share la 3 prieteni pwp